CHAPTER

UX Design Guidelines

32

To err is human; forgive by design.

– Anonymous

Highlights

- Human memory limitations.
- Selected UX design guidelines and examples for user actions within the Interaction Cycle:
  - Planning.
  - Translation.
  - Physical actions.
  - Outcomes.
  - Assessment.
  - Overall.

32.1 INTRODUCTION

When properly interpreted in context, UX (or any) design guidelines help channel design per the collective wisdom of empirical studies and extensive practitioner experience.

32.1.1 Scope and Universality

Lots of design and design guidelines out there. There have been many books and articles on design guidelines for graphical user interfaces (GUIs) and other user interfaces and their widgets—how to create and employ windows, buttons, pull-down menus, pop-up menus, cascading menus, icons, dialogue boxes, check boxes, radio buttons, options menus, forms, and so on. But we want you to think about UX design and design guidelines much more broadly than that, well beyond GUIs, webpages, mobile devices, particular platforms, media, and devices.

There is a world of design out there, and, as Don Norman (1990) says, seeing the guidelines applied to the design of everyday things helps us understand the application of guidelines to interaction by humans with almost any kind of device or system.

Design principles behind the guidelines don't change over time. Although guidelines may change somewhat to remain specific to technology, **design principles have not changed over the years** (Soon, 2013). Changes in technology only affect the ways these design principles are applied.

Universality of design guidelines. The principles and guidelines in this chapter are universal; you will see in this chapter that the same issues apply to ATMs, elevator controls, hair dryers, and even highway signage. We, too, have a strong flavor of *The Design of Everyday Things* (Norman, 1990) in our guidelines and examples. We agree with Jokela (2004) that usability and a quality user experience are also essential in everyday consumer products.

Not everything is covered. We hope you will forgive us for excluding guidelines about internationalization or accessibility (as we advised in the Preface). This book, especially this chapter, is already large, and we cannot cover everything. Beyond that, because the collection of guidelines and examples is so large and diverse, we make no claim for completeness or even that they are consistent among themselves.

For more about where UX design guidelines came from, see Section 33.6.

32.1.2 Some of Our Examples Are Intentionally Old

We have been collecting examples of good and bad interaction and other kinds of design for decades. This means that some of these examples are old. Some of these systems no longer exist. Certainly, some of the problems have been fixed over time, but they are still good examples, and their age shows how we as a community have advanced and improved our designs. Many readers may think the interfaces of modern commercial software applications have always been as they are. Read on.

32.2 USING AND INTERPRETING UX DESIGN GUIDELINES

Are most UX design guidelines not obvious? When we teach these design guidelines, we usually get nods of agreement upon our statement of each guideline. There is very little controversy about most UX design guidelines stated absolutely, out of context. It's obvious; how else would you do it?

However, it's not always so easy to apply those same guidelines in specific usability design and evaluation situations. People are often unsure about which guidelines apply or how to apply, tailor, or interpret them in a specific design situation (Potosnak, 1988). Practitioners don't even agree on the meaning of some guidelines. As Lynn Truss (2003) says in the context of English grammar, that even considering people who are rabidly in favor of using the rules of grammar, it's impossible to get them all to agree on the rules and their interpretation and to pull in the same direction.

Bastien and Scapin (1995, p. 106) quote a study by de Souza and Bevan (1990), who "found that designers made errors, that they had difficulties with 91% of the guidelines, and that integrating detailed design guidelines with their existing experience was difficult for them."

You will not see guidelines here of the type: "Menus should not contain more than X items." That is because such guidelines are meaningless without interpretation within a design and usage context. In the presence of sweeping statements about what is right or wrong in UX design, we can only think of our long-time friend Jim Foley who said, "The only correct answer to any UX design question is: it depends."

We believe much of the difficulty stems from the broad generality, vagueness, and even contradiction within most sets of design guidelines. One of the guidelines near the top of almost any list is "be consistent," an all-time favorite UX platitude. But what does it mean? Consistency at what level; what kind of consistency? Consistency of layout or semantic descriptors such as labels or system support for workflow?

Another such overly general maxim is "keep it simple," certainly a shoo-in to the UX design guidelines hall of fame. But, again, what is simplicity? Minimize the things users can do? It depends on the kind of users, the complexity of their work domain, their skills and expertise.

To address this vagueness and difficulty in interpretation at high levels, we have organized the guidelines in a particular way. Rather than organize the guidelines by the obvious keywords such as consistency, simplicity, and the language of the user, we have tried to associate each guideline with a specific part of the user's Interaction Cycle (previous chapter). This allows specific guidelines to be linked to user actions for planning, knowing what actions to take, making physical actions, and assessing feedback.

Finally, we warn you to use your head and not follow guidelines blindly. While design guidelines and custom style guides are useful in supporting UX design, remember that there is no substitute for a competent, careful, and experienced practitioner.

Interaction cycle

Our adaptation of Norman's (1986) "stages-of-action" model that characterizes sequences of user actions typically occurring in interaction between a human user and almost any kind of machine (Section 31.1.1).

Style guide

A document fashioned and maintained by designers to capture and describe details of visual and other general design decisions, especially about screen designs, font choices, iconography, and color usage, which can be applied in multiple places. A style guide helps with consistency and reuse of design decisions (Section 17.8.1).

32.3 HUMAN MEMORY LIMITATIONS

Because some of the guidelines and much of practical user performance depend on the concepts of human working memory, we interject a short discussion of the same here, before we get into the guidelines themselves. We treat human memory here because:

- It applies to most of the Interaction Cycle parts.
- It's one of the few areas of psychology that has solid empirical data supporting knowledge that is directly usable in UX design.

Our discussion of human memory here is by no means complete or authoritative. Seek a good psychology book for that. We present a potpourri of concepts that should help your understanding in applying the design guidelines related to human memory limitations.

32.3.1 Short-Term or Working Memory

Short-term memory, which we usually call working memory, is the type we are primarily concerned with in UX and has a duration of about 30 seconds.[1] With a little rehearsal, you can stretch this duration out to 2 minutes or longer. Other intervening activities, sometimes called proactive interference, will cause the contents of working memory to fade even faster.

Working memory is a buffer storage that carries information of immediate use in performing tasks. Most of this information is called "throw-away data" because its usefulness is short term, and it's not even desirable to keep it longer. In his famous paper, George Miller (1956) showed experimentally that under certain conditions, the typical capacity of human short-term memory is about seven, plus or minus two items; often it's less.

32.3.1.1 Chunking

The items in short-term memory are often encodings that Simon (1974) has labeled "chunks." A chunk is a basic human memory unit containing one piece of data that is recognizable as a single gestalt. That means for spoken expressions, for example, a chunk is a word, not a phoneme, and in written expressions a chunk is a word or even a single sentence, not usually a letter.

Random strings of letters can be divided into groups which are remembered more easily. If the group is pronounceable, it's even easier to remember, even if it

[1] All of these quantifications are admittedly approximations and can vary with respect to the situation.

has no meaning. Duration trades off with capacity; all else being equal, the more chunks involved, the less time they can be retained in short-term memory.

Example: Phone Numbers Are Designed to be Remembered

Not counting the area code, a phone number has seven digits—not a coincidence that this exactly meets the Miller estimate of working memory capacity. If you look up a number in the phone book, you are loading your working memory with seven chunks. You should be able to retain the number if you use it within the next 30 seconds or so.

A telephone number is a classic example of working memory usage in daily life. If you get distracted between memory loading and usage, you may have to look the number up again, a scenario we all have experienced. If the prefix (the first three digits) is familiar, it's treated as a single chunk, making the task easier.

Sometimes items can be grouped or recoded into patterns that reduce the number of chunks. When grouping and recoding is involved, storage can trade off with processing, just as it does in computers. For example, think about keeping this pattern in your working memory:

001010110111000

On the surface, this is a string of 15 digits, beyond the working memory capacity of most people. But a clever user might notice that this is a binary number and the digits can be grouped into threes:

001 010 110 111 000

By converting this to octal digits: 12670, we have traded off a modicum of processing against memory requirements.

32.3.1.2 Stacking

One way user working memory limitations affect task performance is when task context stacking (storing away context information in the middle of a task, Section 31.4.3) is required due to a new situation arising in the middle of task performance. Before the user can continue with the original task, its context (a memory of where the user was in the task) must be put on a "stack" in the user's memory.

This same thing happens to the context of execution of a software program when an interrupt must be processed before proceeding: the program execution context is stacked in a last-in-first-out (LIFO) data structure. Later, when the system returns to the original program, its context is "popped" from the stack, and execution continues.

It's pretty much the same for a human user whose primary task is interrupted, only the stack is implemented in human working memory. This means that user task stacks are small in capacity and short in duration; people have leaky stacks. After enough time and interruptions, they forget what they were doing.

32.3.1.3 Cognitive load

Cognitive load is the load on working memory at any point in time (Cooper, 1998; Sweller, 1988, 1994). Cognitive load theory (Sweller, 1988, 1994) has been aimed primarily at improvement in teaching and learning through attention to the role and limitations of working memory, but, of course, it also applies directly to UX design. While working with the computer, users are often in danger of having their working memory overloaded. Users can get lost easily in cascading menus with lots of choices at each level or tasks that lead through large numbers of webpages.

If you could chart the load on working memory as a function of time through the performance of a task, you would be looking at variations in the cognitive load across the task steps. When users get to "pop" task context back off the stack, memory load reaches zero, and they get "closure," a feeling of cognitive relief they get from not having to retain information in their working memories.

By organizing tasks into smaller operations instead of one large hierarchical structure, you will reduce the average user cognitive load over time and achieve task closure more often.

32.3.1.4 Recognition versus recall

Because we know that computers are better at memory and humans are better at pattern recognition, we design interaction to play to each other's strengths. You hear people say, in many contexts, "I cannot remember it exactly, but I will recognize it when I see it." That is the basis for the guideline to use recognition over recall. In essence, it means letting the user choose from a list of possibilities rather than having to come up with the choice entirely from memory.

Recognition over recall does work better for initial or intermittent use where learning and remembering are the operational factors, but what happens to people who do learn? They migrate from novice to experienced userhood. In terms of the Interaction Cycle, they begin to remember how to make translations of frequent intentions into actions. They focus less on cognitive actions to know what to do and more on the physical actions of doing it. The cognitive affordances to help new users make these translations can now begin to become barriers to performance of the physical actions.

Moving the cursor and clicking to select items from lists of possibilities becomes more effortful than just typing short memorized commands. When more experienced users do recall the commands they need by virtue of their frequent usage, they find command typing a (legal) performance enhancer compared to the less efficient and, eventually, more boring and irritating physical actions required by those once-helpful GUI affordances.

Even command users get some memory help through command completion mechanisms, the "hum a few bars of it" approach. The user has to remember only the first few characters and the system provides possibilities for the whole command.

32.3.1.5 Shortcuts

When expert users get stuck with a GUI designed for translation affordances, it's time for shortcuts to come to the rescue. In GUIs, these shortcuts are physical affordances, mainly "hot key" equivalents of menu, icon, and button command choices, such as Ctrl-S for the Save command.

Physical affordance

A design feature that helps, aids, supports, facilitates, or enables user physical actions: clicking, touching, pointing, gesturing, and moving things (Section 30.3).

The addition of an indication of the shortcut version to the pull-down menu choice, for example, Ctrl+S added to the Save choice in the File menu, is a simple and subtle but remarkably effective design feature to remind all users of the menu about the corresponding shortcuts. All users can migrate seamlessly from using the shortcuts on the menus to learning and remembering the commands, bypassing the menus to use the shortcuts directly. This is true "as-needed" support of memory limitations in design.

32.3.2 Other Kinds of Human Memory

These other kinds of human memory are usually not important in UX, but we describe them briefly here for completeness.

32.3.2.1 Sensory memory

Sensory memory is of very brief duration. For example, the duration of visual memory ranges from a small fraction of a second to maybe 2 seconds and is primarily about visual (and other) patterns observed, not anything about identifying what was seen or what it means. It is raw sensory data that allow direct comparisons between stimuli, such as might occur in detecting voice inflection. Sensory persistence is the phenomenon of storage of the stimulus in the sensory organ, not the brain.

For example, visual persistence allows us to integrate the fast-moving sequences of individual image frames in movies or television, making them appear as a smooth integrated motion picture.

32.3.2.2 Muscle memory

Muscle memory is a little bit like sensory memory in that it's mostly stored locally (in the muscles in this case), and not the brain. Muscle memory is important for repetitive physical actions; it's about getting in a "rhythm." Thus, muscle memory is an essential aspect of learned skills of athletes. In UX, it's important in physical actions such as typing.

Example: Muscling Light Switches

In the United States at least, we use an arbitrary convention that moving an electrical switch up means "on," and down means "off." Over a lifetime of usage, we develop muscle memory because of this convention and hit the switch in an upward direction as we enter the room without pausing.

However, if you have lights on a three-way switch, "on" and "off" cannot be assigned consistently to any given direction of the switch. It depends on the state of the whole set of switches. If, out of habit, you find yourself hitting a three-way switch in an upward direction, sometimes the light fails to turn on because the switch was already up with the lights off. No amount of practice or trying to remember can overcome this conflict between muscle memory and this design.

32.3.2.3 Long-term memory

Information stored in short-term memory can be transferred to long-term memory by "learning," which may involve the hard work of rehearsal and repetition. Transfer to long-term memory relies heavily on organization and structure of information already in the brain. Items transfer more easily if associations exist with items already in long-term memory.

The capacity of long-term memory is almost unlimited—a lifetime of experiences. The duration of long-term memory is also almost unlimited, but retrieval is not always guaranteed. Learning, forgetting, and remembering are all intertwined with the vagaries of long-term memory. Sometimes, items can be classified in more than one way. Maybe one item of a certain type goes in one place, and another item of the same type goes elsewhere. As new items and new types of items come in, the classification system is revised to accommodate. Retrieval depends on being able to reconstruct the structural encoding used for storage.

When we forget, items become inaccessible, but probably not lost. Sometimes forgotten or repressed information can be recalled. Electric brain stimulation can trigger reconstructions of visual and auditory memories of past events. Hypnosis can help recall vivid experiences of years ago. Some evidence indicates that hypnosis increases willingness to recall rather than the ability to recall.

32.4 REVIEW OF THE INTERACTION CYCLE STRUCTURE

The selected UX design guidelines in this section are generally organized by the Interaction Cycle (previous chapter) structure.

To review the structure of the Interaction Cycle from the previous chapter, we show the simplest view of this cycle in Fig. 32-1, featuring these parts:

- Planning: How the UX design supports users in determining what to do?
- Translation: How the UX design supports users in determining how to do actions on objects?
- Physical actions: How the UX design supports users in doing those actions?
- Outcomes: How the noninteraction functionality of the system helps users achieve their work goals?
- Assessment of outcomes: How the UX design supports users in determining whether the interaction is turning out right?

32.5 PLANNING

Support for user planning (Fig. 32-2) is often a missing component of UX designs.

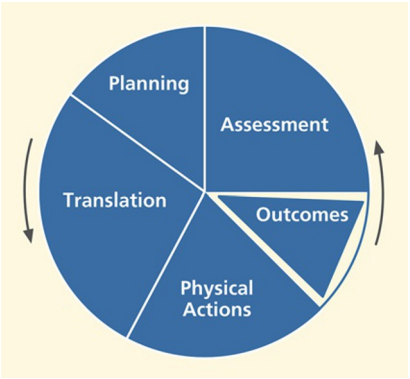


Fig. 32-1
Simplest view of the Interaction Cycle.

Planning guidelines are to support users as they plan how they will use the system to accomplish work in the application domain, including cognitive user actions to determine what tasks or steps to do. It's also about helping users understand what tasks they *can* do with the system and how well the design supports learning about the system for planning. If users cannot determine how

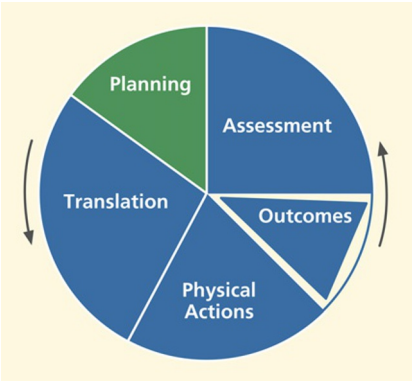


Fig. 32-2
The planning part of the Interaction Cycle.

to organize several related tasks in the work domain because the system doesn't help them understand exactly how it can help do these kinds of tasks, the design needs improvement in planning support.

32.5.1 Clear System Task Model for User

Support the user's ability to acquire an overall understanding of the system at a high level, including the system model, conceptual design, and metaphors.[2]

Help users plan goals, tasks by providing a clear model of how users should view system in terms of tasks.

Support user task decomposition by matching the design to users' concept of task decomposition and organization.

Example: Get Organized

The top of Fig. 32-3 shows tabs that occur on every page of a particular digital library website. These tabs are not well organized by task, having information-seeking tasks mixed in with other kinds of tasks. Part of the new tab bar in our suggested new design is shown in the bottom of Fig. 32-3.

Help users understand what system features exist and how they can be used in their work context.

[2]This special green font denotes a guideline.

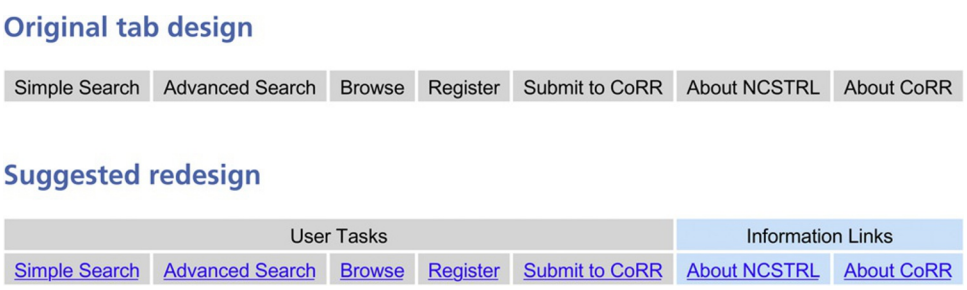


Fig. 32-3
Tab reorganization to match task structure.

Support user awareness of specific system features capabilities and understanding of how they can use those features to solve work domain problems in different work situations. Support user ability to attain awareness of specific system feature or capability.

Example: Mastering the Master Document Feature

Consider the case of the Master Document feature in Microsoft Word. For convenience and to keep file sizes manageable, users of Microsoft Word can maintain each part of a document in a separate file. At the end of the day, they can combine those individual files to achieve the effect of a single document for global editing and printing.

However, we have found this ability to treat several chapters in different files as a single document almost impossible to understand. The system doesn't help the user determine what can be done with it or how it might help with this task. Further, one wrong move, and "it can corrupt your entire document at the most inconvenient time possible."[3]

Help users decompose tasks, logically breaking long, complex tasks into smaller, simpler pieces.

Make clear all possibilities for what users can do at every point.

Keep users aware of system state for planning the next task.

Maintain and clearly display system state indicators when next actions are state dependent.

Keep the task context visible to minimize memory load.

To help users compare outcomes with goals, maintain and clearly display user request along with results.

[3]A comment on the Internet (and it has happened to us).

Example: Library Search by Author

In the search mode within a library information system, users can find themselves deep down many levels and screens into the task where card catalog information is being displayed. By the time they dig into the information structure that deeply, there is a chance users may have forgotten their exact original search intentions. Somewhere on the screen, it would be helpful to have a reminder of the task context, such as "You are *searching by author* for: Stephen King."

32.5.2 Planning for Efficient Task Paths

Help users plan the most efficient ways to complete their tasks.

Example: The Helpful Printing Command

This is an example of good design, rather than a design problem, and it's from an old version of Borland's 3-D Home Architect. Using this house-design program, when a user tries to print a large house plan, it results in an informative message in a dialogue box that says: "Current printer settings require 9 pages at this scale. Switching to Landscape mode allows the plan to be drawn with 6 pages. Click on Cancel if you wish to abort printing."

This tip can be most helpful, saving the time and paper involved in printing it incorrectly the first time, making the change, and printing again. As an aside here, this message still falls short of the mark. First, the term "abort" has overtones that could be interpreted as violent. Plus, the design could provide a button to change to landscape mode directly, without forcing the user to find out how to make that switch.

32.5.3 Progress Indicators

Support user planning with task progress indicators to help users manage task sequencing and keep track of what parts of the task are done and what parts are left to do.

During long tasks with multiple and possibly repetitive steps, users can lose track of where they are in the task. For these situations, task progress indicators or progress maps can be used to help users with planning based on knowing where they are in the task.

Example: Turbo-Tax Keeps You on Track

Filling out income tax forms is a good example of a lengthy multiple-step task. The designers of Turbo-Tax by Intuit used a "wizard-like" step-at-a-time prompter

to help users understand where they are in the overall task, showing the user's progress through the steps while summarizing the net effect of the user's work at each point.

32.5.4 Avoiding Transaction Completion Slips

A transaction completion slip is a kind of error in which the user omits or forgets a final action, often a crucial action for consummating the task.

Provide cognitive affordances at the end of critical tasks to remind users to complete the transaction.

Example: Hey, Don't Forget Your Tickets

A transaction completion slip can occur in the Ticket Kiosk System when the user gets a feeling of closure at the end of the interaction for the transaction and fails to take the tickets just purchased. In this case, special attention is needed to provide a cognitive affordance to remind the user of the final step in the task plan and help prevent this kind of slip: "Please take your tickets (and your bank card and receipt)."

Example: Another Forgotten Email Attachment

As another example, almost everyone has sent or tried to send an email for which an attachment was intended but forgotten. Recent versions of Google's Gmail (and others, like the Mac) have a simple solution. If any variation of the word "attach" appears in an email but it's sent without an attachment, the system asks if the sender intended to attach something, as you can see in Fig. 32-4. Similarly, if the email author says something such as "I am copying …", and there is no address in the Copy field, the system could ask about that, too.

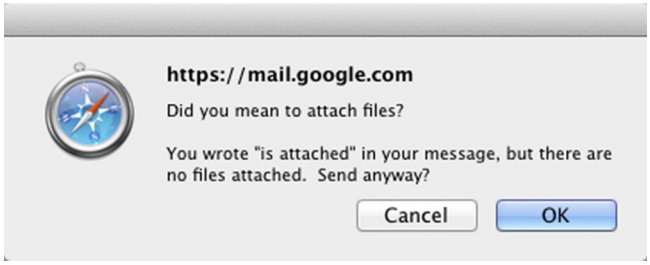


Fig. 32-4
Reminder to attach a file.

Example: Oops, You Didn't Finish Your Transaction

On one banking site, when users transfer money from their savings accounts to their checking accounts, they often think the transaction is complete when it's actually not. This is because, at the bottom right-hand corner of the last page in this transaction workflow, just below the "fold" on the screen, there is a small button labeled **Confirm** that is often completely missed.

Users close the window and go about their business of paying bills, unaware that they are possibly heading toward an overdraft. At least they should have gotten a pop-up message reminding them to click the Confirm button before letting them log out of the website.

Later, when one of the users called the bank to complain, they politely declined his suggestion that they should pay the overdraft fee because of their liability due to poor usability. We suspect they must have gotten other such complaints, however, because the flaw was fixed in the next version.

Example: Microwave Is Anxious to Help

As a final example of avoiding transaction completion slips, we cite a microwave oven. Because it takes time to defrost or cook the food, users often start it and do something else while waiting. Then, depending on their level of hunger, it's possible to forget to take out the food when it's done.

So, microwave designers usually include a reminder. At completion, the microwave usually beeps to signal the end of its part of the task. However, a user who has left the room or is otherwise occupied when it beeps may still not be mindful of the food waiting in the microwave. As a result, most oven designs call for the beep to repeat periodically until the door is opened to retrieve the food.

The design for one particular microwave, however, took this too far. It didn't wait long enough for the follow-up beep. Sometimes a user would be on the way to remove the food and it would beep. Some users found this so irritating that they would hurry to rescue the food before that "reminder" beep. To them, this machine seemed to be "impatient" and "bossy" to the point that it had been controlling the users by making them hurry.

32.6 TRANSLATION

Translation guidelines are to support users in sensory and cognitive actions needed to determine how to do a task step in terms of what actions to make on which objects and how. Translation, along with assessment, is one of the places in the Interaction Cycle where cognitive affordances play a major role.

Many of the principles and guidelines apply to more than one part of the Interaction Cycle and, therefore, to more than one section of this chapter. For example, "Use consistent wording" is a guideline that applies almost everywhere. Rather than repeat, we will put them in the most pertinent location and hope that our readers recognize the broader applicability.

Translation issues include:

- Existence (of cognitive affordance).
- Presentation (of cognitive affordance).
- Content and meaning (of cognitive affordance).
- Task structure.

32.6.1 Existence of Cognitive Affordance

Fig. 32-5 highlights the "existence of cognitive affordance" part within the breakdown of the translation part of the Interaction Cycle.

"Existence of a cognitive affordance" is about whether a needed cognitive affordance is provided in the first place. If UX designers don't provide needed cognitive affordances, such as labels and other cues, users will lack the support they need for learning and knowing what actions to make on what objects in order to carry out their task intentions. The existence of cognitive affordances is necessary to:

- Show which user interface object to manipulate.
- Show how to manipulate an object.

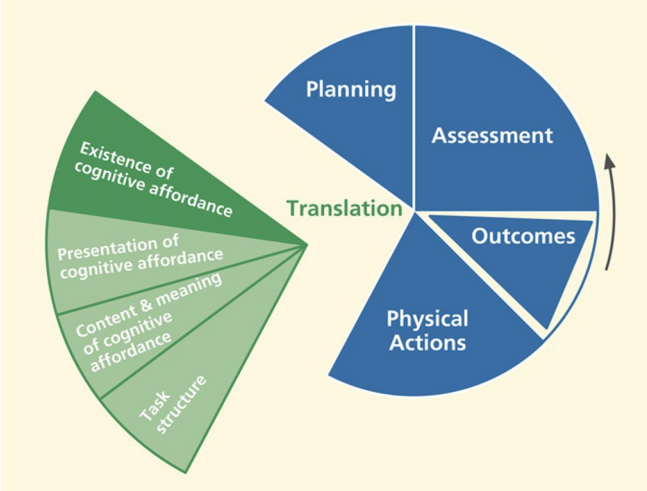


Fig. 32-5
Existence of a cognitive affordance within translation.

- Help users get started in a task.
- Guide data entry in formatted fields.
- Indicate active defaults to suggest choices and values.
- Indicate system states, modes, and parameters.
- Remind about steps the user might forget.
- Avoid inappropriate choices.
- Support error recovery.
- Help answer questions from the system.
- Deal with idioms that require rote learning.

Provide effective cognitive affordances that help users get access to system functionality.

Support users' cognitive needs to determine how to do something by ensuring the existence of an appropriate cognitive affordance.

It's possible to build in effective cognitive affordances that help novice users and don't get in the way of experienced users.

Help users know how to do something at the action/object level, to know/learn what actions are needed to carry out intentions.

Users get their operational knowledge from experience, training, *and cognitive affordances in the design*. It's our job to provide this latter source of user knowledge.

Be predictable; help users predict outcome of actions with feed-forward information in cognitive affordances.

Users need *feed-forward* cognitive affordances, such as labels, data field formats, and icons that explain the effects of physical actions, such as clicking on a button. Not giving feed-forward cues is what Cooper (2004, p. 140) calls "uninformed consent"; the user must proceed without understanding the consequences and possibly go down a rabbit hole. Predictability helps both learning and error avoidance.

Help users determine what to do to get started.

Users need support in understanding what actions to take for the first step of a particular task, the "getting started" step, often the most difficult part of a task.

Example: Helpful PowerPoint

In Fig. 32-6, we see a start-up screen of an early version of Microsoft PowerPoint. In applications where there is a wide variety of things a user can do, it's difficult to know what to do to get started when faced with a blank screen. The addition of one simple cognitive affordance: The advice to **Click to add first slide**, provides an easy way for an uncertain user to get started in creating a presentation.

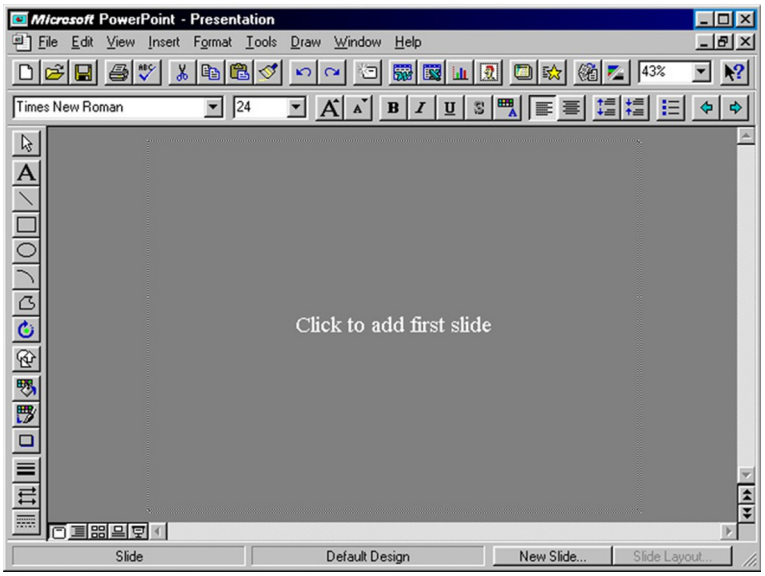


Fig. 32-6
Help in getting started in PowerPoint (screen image courtesy of Tobias Frans-Jan Theebe).

Similarly, in Fig. 32-7, we show other such helpful cues to continue, once a new slide is begun.

Provide a cognitive affordance for a step the user might forget.

Support user needs with cognitive affordances as prompts, reminders, cues, or warnings for a particular needed action that might get forgotten.

32.6.2 Presentation of Cognitive Affordance

In Fig. 32-8, we highlight the "presentation of cognitive affordance" portion of the translation part of the Interaction Cycle.

Presentation of cognitive affordances is about how cognitive affordances *appear* to users, not how they convey meaning. Users must be able to sense, for example, see or hear, a cognitive affordance before it can be useful to them. Therefore, presentation of cognitive affordances is primarily about sensory affordances.

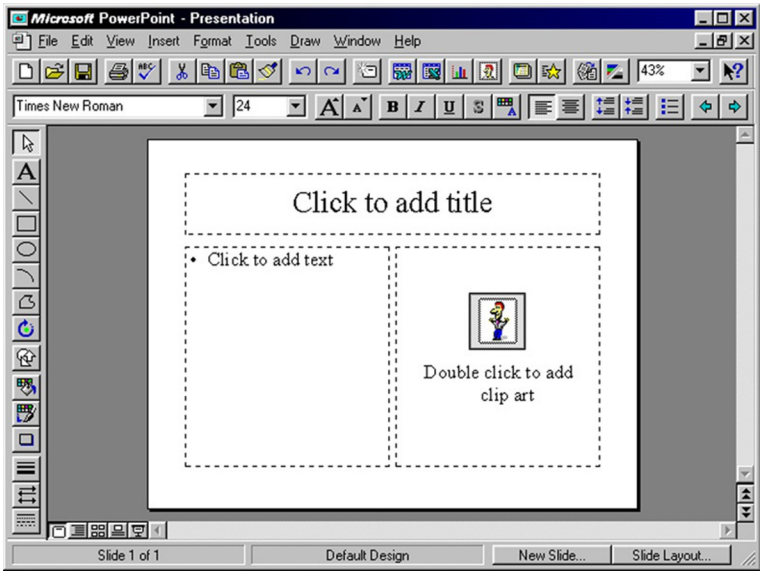


Fig. 32-7

More help in getting started (screen image courtesy of Tobias Frans-Jan Theebe).

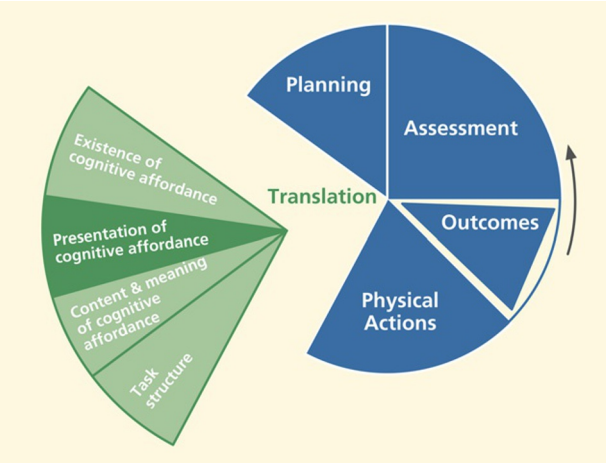


Fig. 32-8

Presentation of cognitive affordances within translation.

Support user sensory needs in seeing and hearing cognitive affordances by effective presentation or appearance.

This category is about issues such as legibility, noticeability, timing of presentation, layout, spatial grouping, complexity, consistency, and presentation medium, for example, audio, when needed. Sensory affordance issues also

include text legibility and content contained in the appearance of a graphical feature, such as an icon, but only about whether the icon can be seen or discerned easily. For an audio medium, the volume and sound quality are presentation characteristics.

32.6.2.1 Cognitive affordance visibility

Obviously, a cognitive affordance cannot be an effective cue if it cannot be seen or heard when it's needed. Our first guideline in this category is conveyed by the sign in Fig. 32-9, if only we could be sure what it means.

Fig. 32-9
Good advice anytime.

Make cognitive affordances visible.

If a cognitive affordance is invisible, it could be because it's not (yet) displayed or because it's occluded by another object. A user aware of the existence of the cognitive affordance can often take some actions to summon an invisible cognitive affordance into view. It's the designer's job to be sure each cognitive affordance is visible, or easily made visible, when it's needed in the interaction.

See the example of shopping for deodorant in a grocery store in Section 30.4.2.1.

32.6.2.2 Cognitive affordance noticeability

Make cognitive affordances noticeable.

When a needed cognitive affordance exists and is visible, the next consideration is its noticeability or likelihood of being noticed or sensed. Just putting a cognitive affordance on the screen is not enough, especially if the

user doesn't necessarily know it exists or is not necessarily looking for it. These design issues are largely about supporting awareness. Relevant cognitive affordances should come to users' attention without users seeking it. The primary design factor in this regard is location, putting the cognitive affordance within the users' focus of attention. It's also about contrast, size, and layout complexity and their effect on separation of the cognitive affordance from the background and from the clutter of other user interface objects.

Example: Status Lines Often Don't Work

Message lines, status lines, and title lines at the top or bottom of the screen are notoriously unnoticeable. Each user typically has a narrow focus of attention, usually near where the cursor is located. A pop-up message next to the cursor will be far more noticeable than a message in a line at the bottom of the screen.

Example: Where the Heck Is the Log-In?

For some reason, many websites have very small and inconspicuous log-in boxes, often mixed in with many objects most users don't even notice in the far top border of the page. Users have to waste time in searching visually over the whole page to find the way to log in.

32.6.2.3 Cognitive affordance legibility

Make text legible, readable.

Text legibility is about being discernable, not about the words being understandable. Text presentation issues include the way the text of a button label is presented so it can be read or sensed, including appearance or sensory characteristics of the text, such as font type, font size, font and background color, bolding, or italics, but it's not about the content or meaning of the words in the text. The meaning is the same regardless of the font or color.

32.6.2.4 Cognitive affordance presentation complexity

Control cognitive affordance presentation complexity with effective layout, organization, and grouping.

Support user needs to locate and be aware of cognitive affordances by controlling layout complexity of user interface objects. Screen clutter can obscure needed cognitive affordances such as icons, prompt messages, state indicators, dialogue box components, or menus and make it difficult for users to find them.

32.6.2.5 Cognitive affordance presentation timing

Support user needs to notice cognitive affordance with appropriate timing of appearance or display of cognitive affordances. Don't present a cognitive affordance too early or too late. Present with adequate persistence; that is, avoid "flashing."

Present cognitive affordance in time to help the user before the associated action.

Sometimes getting cognitive affordance presentation timing right means presenting at exactly the point in a task and under exactly the conditions when the cognitive affordance is needed.

See the example about the just-in-time towel dispenser message in Section 30.4.2.7.

Example: Special Pasting

When a user wishes to paste something from one Word document to another, there can be a question about formatting. Will the item retain its formatting, such as text or paragraph style, from the original document or will it adopt the formatting from the place of insertion in the new document? And how can the choice be controlled by the user? When you want more control of a paste operation, you might choose **Paste Special** … from the **Edit** menu.

But the choices in the **Paste Special** dialogue box say nothing about controlling formatting. Rather, the choices can seem system centered or too technical, for example, **Microsoft Office Word Document Object** or **Unformatted Unicode Text**, without an explanation of the resulting effect in the document. While these choices might be precise about the action and its results to some users, they are cryptic even to most regular users.

In some versions of Word, a small cognitive affordance, a tiny clipboard icon with a pop-up label **Paste Options** appears, but it appears *after* the paste operation. Many users don't notice this little object, mainly because by the time it appears, they have experienced closure on the paste operation and have already moved on mentally to the next task. If they don't like the resulting formatting, then changing it manually becomes their next task.

Even if users notice the little object, it's possible they might confuse it with something to do with undoing the action or something similar because Word uses that same object for in-context undo. However, if a user does notice this icon and does take the time to click on it, that user will be rewarded with a pull-down menu of useful options, such as **Keep Source Formatting**, **Match Destination**

Formatting, **Keep Text Only**, plus a choice to see a full selection of other formatting styles.

Just what users need! But it's made available too late; the chance to see this menu comes *after* the user action to which it applied. If choices on this after-the-fact menu were available on the **Paste Special** menu, it would be perfect for users.

32.6.2.6 Cognitive affordance presentation consistency

When a cognitive affordance is located within a user interface object that is also manipulated by physical actions, such as a label within a button, maintaining a consistent location of that object on the screen helps users find it quickly and helps them use muscle memory for fast clicking. Hansen (1971) used the term "display inertia" in reference to one of his top-level principles, optimize operations, to describe this business of minimizing display changes in response to user inputs, including displaying a given user interface object in the same place each time it's shown.

Give similar cognitive affordances consistent appearance in presentation.

Example: Archive Button Jumps Around

When users of an older version of Gmail were viewing the list of messages in the Inbox, the **Archive** button was at the far left at the top of the message pane, set off by the blue border, as shown in Fig. 32-10.

But on the screen for reading a message, Gmail had the **Archive** button as the second object from the left at the top. In the place where the **Archive** button was earlier, there was now a **Back to Inbox** link, as seen in Fig. 32-11. Using a link instead of a button in this position is a slight inconsistency, probably without

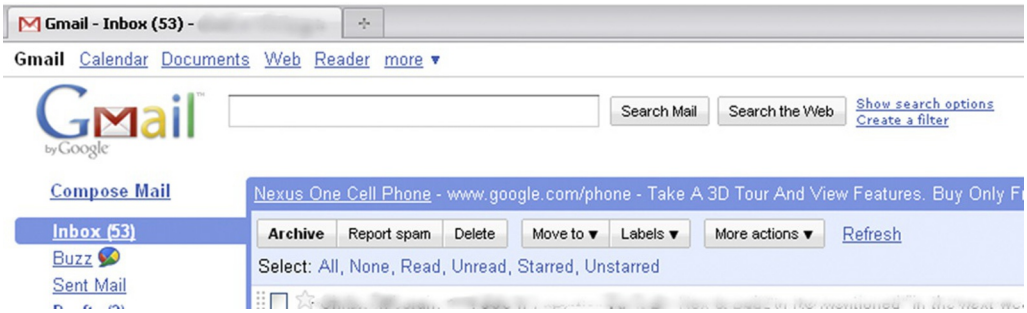


Fig. 32-10

*The **Archive** button in the Inbox view of an older version of Gmail.*

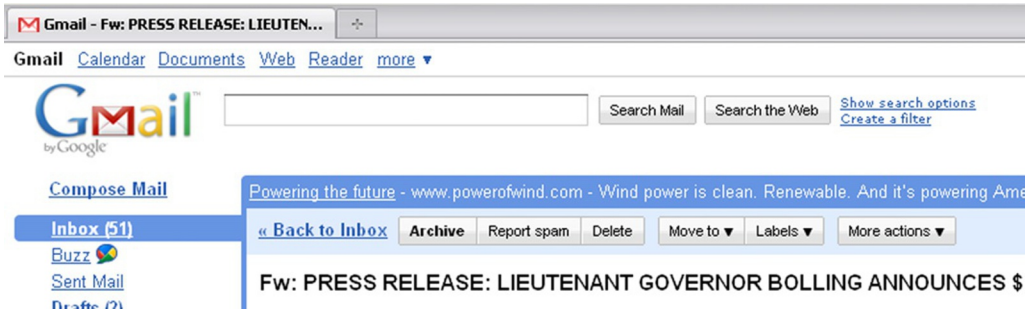


Fig. 32-11

*The **Archive** button in a different place in the message reading view.*

much effect on users. But users feel a larger effect from the inconsistent placement of the Archive button.

Selected messages can be archived from either view of the email by clicking on the **Archive** button. Further, when archiving messages from the Inbox list view, the user sometimes goes to the message-reading view to be sure. So, a user doing an archiving task could be going back and forth between the Inbox listing of Fig. 32-10 and message viewing of Fig. 32-11.

For this activity, the location of the **Archive** button is never certain. The user loses momentum and performance speed by having to look for the **Archive** button each time before clicking on it. Even though it moves only a short distance between the two views, it's enough to slow down users significantly because they cannot run the cursor up to the same spot every time to do multiple archive actions quickly. The lack of display inertia works against an efficient sensory action of finding the button, and it works against muscle memory in making the physical action of moving the cursor up to click.

It seems that Google people have fixed this problem in subsequent versions, as attested to by the same kinds of screens in Figs. 32-12 and 32-13.

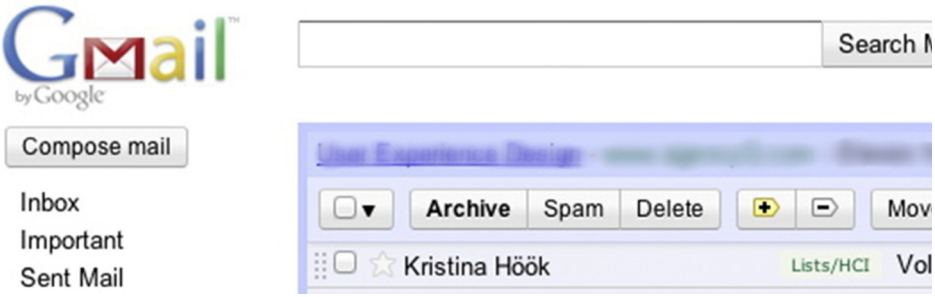


Fig. 32-12

*The **Archive** button in the Inbox view of a later version of Gmail.*

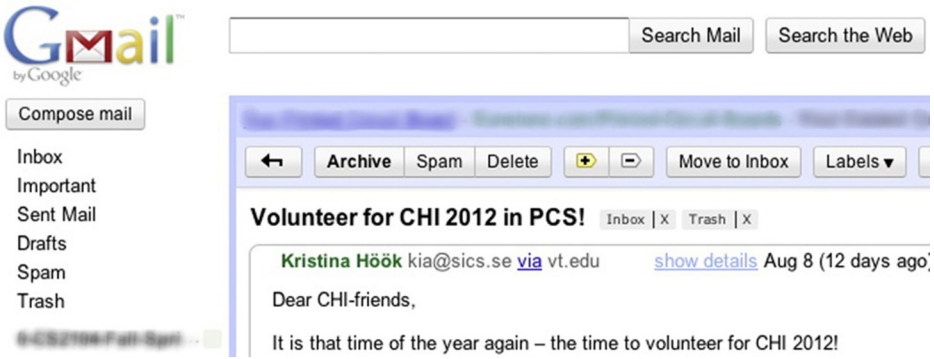


Fig. 32-13

*The **Archive** button in the same place in the new message reading view.*

32.6.3 Content and Meaning of Cognitive Affordance

> Just what part of quantum theory do you not understand?
>
> – Anonymous

Fig. 32-14 highlights the "content and meaning of cognitive affordance" portion of the translation part of the Interaction Cycle.

The content and meaning of a cognitive affordance are the knowledge that must be conveyed to users to be effective in helping them as affordances to think, learn, and know what they need to make correct actions. The cognitive affordance design concepts that support understanding of content and meaning include clarity, distinguishability from other cognitive affordances, consistency,

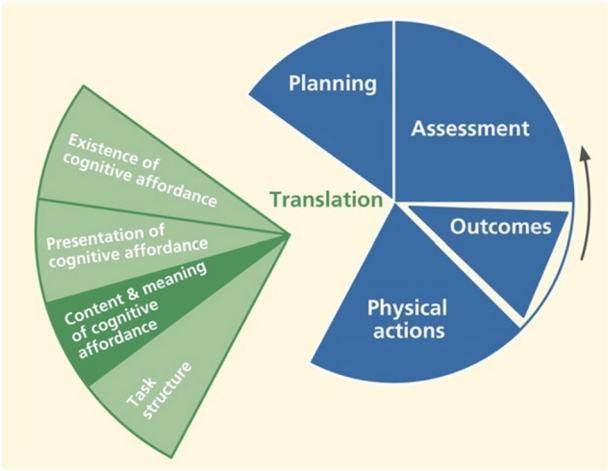


Fig. 32-14

Content/meaning within translation.

layout, and grouping to control complexity, usage centeredness, and techniques for avoiding errors.

Help user determine actions with effective content/meaning in cognitive affordances.

Support user ability to determine what action(s) to make and on what object (s) for a task step through understanding and comprehension of cognitive affordance content and meaning: what it says, verbally or graphically.

32.6.3.1 Clarity of cognitive affordances

Design cognitive affordances for clarity.

Use precise wording, carefully chosen vocabulary, or meaningful graphics to create correct, complete, and sufficient expressions of content and meaning of cognitive affordances.

32.6.3.2 Precise wording

Support user understanding of cognitive affordance content by precise expression of meaning through precise word choices.

Precise word choices are especially important for short, command-like text, such as is found in button labels, menu choices, and verbal prompts. For example, the button label to dismiss a dialogue box could say "**Return to** …" where appropriate, instead of just **OK**.

Use precise wording in labels, menu titles, menu choices, icons, data fields.

The imperative for clear and precise wording of button labels, menu choices, messages, and other text may seem obvious, at least in the abstract. Even experienced designers often don't take the time to choose their words carefully.

In our own evaluation experience, this guideline is among the most violated in real-world practice. Others have shared this experience, including Johnson (2000). Because of the overwhelming importance of precise wording in UX designs and the apparent unmindful approach to wording by many designers in practice, we consider this to be one of the most important guidelines in the whole book.

Part of the problem in the field is that wording is often considered a relatively unimportant part of UX design and is left to developers and software people not trained to construct precise wording and not even trained to think much about it.

Example: Wet Paint!

This is one of our favorite examples of precise wording, probably overdone: "Wet Paint. This is a warning, not an instruction."

This guideline represents a part of UX design where a great improvement can be accrued for only a small investment of extra time and effort. Even a few minutes devoted to getting just the right wording for a button label used frequently has an enormous potential payoff. Here are some related and helpful subguidelines:

Use a verb and noun and even an adjective in labels where appropriate.
Avoid vague, ambiguous terms.
Be as specific to the interaction situation as possible; avoid one-size-fits-all messages.
Clearly represent work domain concepts.

Example: Then, How Can We Use the Door?

As an example of matching the message to the reality of the work domain, signs such as "Keep this door closed at all times" (Fig. 32-15) probably should read something more like "Close this door immediately after use."

Use dynamically changing labels when toggling.

Fig. 32-15
Difficult to follow this instruction literally.

When using the same control object, such as a **Play**/**Pause** button on an mp3 music player, to control the toggling of a system state, change the object label to show that it's consistently a control the user can act on to get to the next state. Otherwise, the current system state can be unclear, and there can be confusion over whether the label represents an action the user can make or feedback about the current system state.

Example: Reusing a Button Label

In Fig. 32-16, we show an early prototype of a personal document retrieval system. The underlying model for deleting a document involves two steps: marking the document for deletion and later deleting all marked documents permanently. The small check box at the lower right is labeled: **Marked for Deletion**.

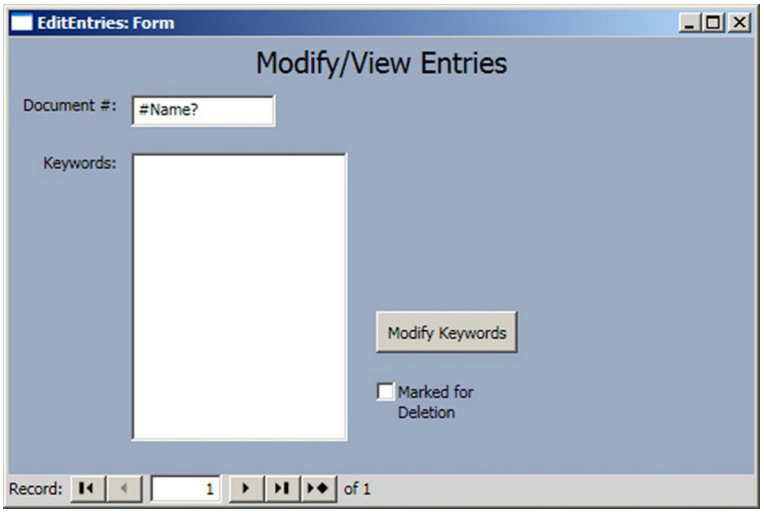


Fig. 32-16
The Marked for Deletion check box in a document retrieval screen (screen image courtesy of Raphael Summers).

The designer's idea was that users would check that box to signify the intention to delete the record. Thereafter, until a permanent purge of marked records, seeing a check in this box signifies that this record is, indeed, marked for deletion. The problem comes before the user checks the box.

The user wants to delete the record (or at least mark it for deletion), but this label seems to be a statement of system state rather than a cognitive affordance for an action, implying that it's already marked for deletion. However, because the check box is not checked, it's not entirely clear. Our suggestion was to relabel the box in the unchecked state to read **Check to Mark for Deletion**, making it a true cognitive affordance for action in this state. After checking, **Marked for Deletion** works fine.

Data value formats. **Provide cognitive affordances to indicate formatting within data fields.**

Support user needs to know how to enter data, such as in a form field, with a cognitive affordance or cue to help with format and kinds of values that are acceptable.

Data entry is a user work activity where the formatting of data values is an issue. Entry in the "wrong" format, meaning a format the user thinks is right but the system designers didn't anticipate, can lead to errors that the user must spend time to resolve or, worse, undetected data errors. It's relatively easy for designers to indicate expected data formats, with cognitive affordances associated with the field labels, with sample data values, or both.

Example: How Should I Enter the Date?

In Fig. 32-17, we show a dialogue box that appears in an application that is, despite the cryptic title **Task Series**, for scheduling events. In the **Duration** section, the **Effective Date** field doesn't indicate the expected format for data values. Although many systems are capable of accepting date values in almost any format, new or intermittent users may not know if this application is that smart. It

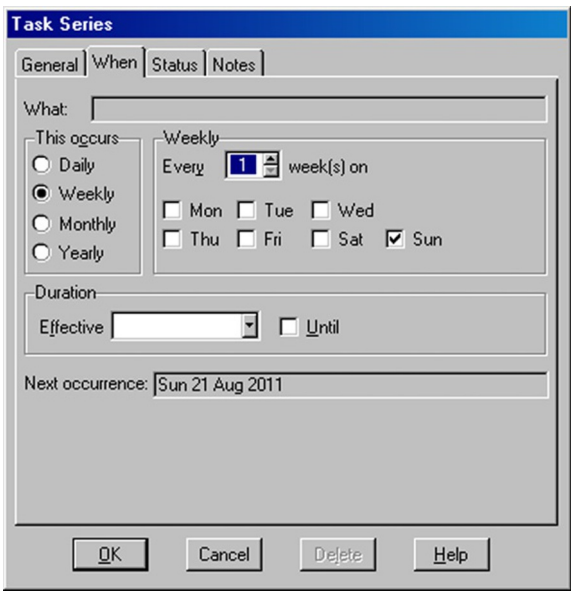


Fig. 32-17
*Missing cognitive affordance about **Effective Date** data field format (screen image courtesy of Tobias Frans-Jan Theebe).*

would have been easy for the designer to save users from hesitation and uncertainty by suggesting a format here.

Constrain the formats of data values to avoid data entry errors.

Sometimes rather than just show the format, it's more effective to constrain values to prevent erroneous entries.

An easy way to constrain the formatting of a date value, for example, is to use drop-down lists, specialized to hold values appropriate for the month, day, and year parts of the date field. Another approach that many users like is a "date picker," a calendar that pops up when the user clicks on the date field. A date can be entered into the field only by way of selection from this calendar.

A calendar with one month of dates at a time is perhaps the most practical. Side arrows allow the user to navigate to earlier or later months or years. Clicking on a date within the month on the calendar causes that date to be picked for the value to be used. By using a date-picker, you are constraining both the data entry method and the format the user must employ, effectively eliminating errors due to either allowing inappropriate values or formatting ambiguity.

Provide clearly marked exits.

Support user ability to exit dialogue sequences confidently by using clearly labeled exits. Include destination information to help users predict where the action will go upon leaving the current dialogue sequence. For example, in a dialogue box you might use **Return to XYZ After Saving** instead of **OK** and **Return to XYZ Without Saving** instead of **Cancel**.

To qualify this example, we have to say that the terms **OK** and **Cancel** are so well accepted and so thoroughly part of our current shared conventions that, even though the example shows potentially better wordings, the conventions now carry the same meaning at least to experienced users.

Provide clear "do it" mechanism.

Some kinds of choice-making objects, such as drop-down or pop-up menus, commit to the choice as soon as the user indicates the choice; others require a separate "commit to this choice" action. This inconsistency can be unsettling for some users who are unsure about whether their choices have "taken." Becker (2004) argues for a consistent use of a "Go" action, such as a click, to commit to choices, for example, choices made in a dialogue box or drop-down

menu. And we caution to make its usage clear to avoid task completion slips where users think they have completed making the menu choice, for example, and move on without committing to it with the Go button.

32.6.3.3 Distinguishability of choices in cognitive affordances

Make choices cognitively distinguishable.

Support user ability to differentiate two or more possible choices or actions by distinguishable expressions of meaning in their cognitive affordances. If two similar cognitive affordances lead to different outcomes, careful design is needed so users can avoid errors by distinguishing the cases.

Often distinguishability is the key to correct user choices by the process of elimination; if you provide enough information to rule out the cases not wanted, users will be able to make the correct choice. Focus on differences of meaning in the wording of names and labels. Make larger differences graphically in similar icons.

Example: Tragic Airplane Crash

This is an unfortunate, but true, story that evinces the reality that human lives can be lost due to simple confusion over labeling of controls. This is a very serious usability case involving the October 31, 1999, EgyptAir Flight 990 airliner crash (Acohido, 1999) possibly as a result of poor usability in design. According to the news account, the pilot may have been confused by two sets of switches that were similar in appearance, labeled very similarly, as "**Cut out"** and "**Cut off"**, and located relatively close to each other in the Boeing 767 cockpit design.

Exacerbating the situation, both switches are used infrequently, only under unusual flight conditions. This latter point is important because it means that the pilots would not have been experienced in using either one. Knowing pilots receive extensive training, designers assumed their users are experts. But because these particular controls are rarely used, most pilots are novices in their use, implying the need for more effective cognitive affordances than usual.

One conjecture is that one of the flight crew attempted to pull the plane out of an unexpected dive by setting the **Cut out** switches connected to the stabilizer trim, but instead accidentally set the **Cut off** switches, shutting off fuel to both engines. The black box flight recorder did confirm that the plane did go into a sudden dive, and that a pilot did flip the fuel system cutoff switches soon thereafter.

There seem to be two critical design issues, the first of which is the distinguishability of the labeling, especially under conditions of stress and

infrequent use. To us, not knowledgeable in piloting large planes, the two labels seem so similar as to be virtually synonymous.

Making the labels more complete would have made them much more distinguishable. In particular, adding a noun to the verb of the labels would have made a huge difference: **Cut out trim** versus **Cut off fuel**. Putting the all-important noun first might be an even better distinguisher: **Fuel off** and **Trim out**. Just this simple UX improvement might have averted the disaster.

The second design issue is the apparent physical proximity of the two controls, inviting the physical slip of grabbing the wrong one, despite knowing the difference. Surely stabilizer trim and fuel functions are completely unrelated. Regrouping by related functions—locating the **Fuel off** switch with other fuel-related functions and the **Trim out** switch with other stabilizer-related controls—might have helped the pilots distinguish them, preventing the catastrophic error.

Finally, we have to assume that safety (absolute error avoidance in this situation) was a top priority UX goal for this design. To meet this goal, the Fuel off switch could have been further protected from accidental operation by adding a mechanical feature that requires an additional conscious action by the pilot to operate this seldom-used but dangerous control. One possibility is a physical cover over the switch that has to be lifted before the switch can be flipped, a safety feature used in the design of missile launch switches, for example.

32.6.3.4 Consistency of cognitive affordances

Be consistent with cognitive affordances.

Use consistent wording in labels for menus, buttons, icons, fields.

Consistency is one of those concepts that everyone thinks they understand, but almost no one can define. Being consistent in wording has two sides: using the same terms for the same things and using different terms for different things.

Use similar names for similar kinds of things.

Don't use multiple synonyms for the same thing.

Example: Continue or Retry?

This example comes from the very old days of floppy disks, but could apply to external hard disks of today as well. It's a great example of using two different words for the same thing in the short space of the one little message dialogue box in Fig. 32-18.

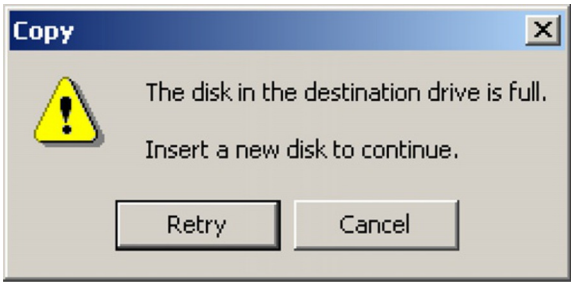


Fig. 32-18

*Inconsistent wording: **Continue** or **Retry**? (screen image courtesy of Tobias Frans-Jan Theebe).*

If, upon learning that the current disk is full, the user inserts a new disk and intends to continue copying files, for example, what should she click, **Retry** or **Cancel**? Hopefully she can find the right choice by the process of elimination, as **Cancel** will almost certainly terminate the operation. But **Retry** carries the connotation of starting over. Why not match the goal of continuing with a button labeled **Continue**?

Use the same term in a reference to an object as the name or label of the object.

If a cognitive affordance suggests an action on a specific object, such as **Click on Add Record**, the name or label on that object must be the same, in this case also **Add Record**.

Example: Press What?

From more modern days and a website for Virginia Tech employees, Fig. 32-19 is another clear example of how easy it is for this kind of design flaw to slip by designers, a type of flaw that is usually found by expert UX inspection.

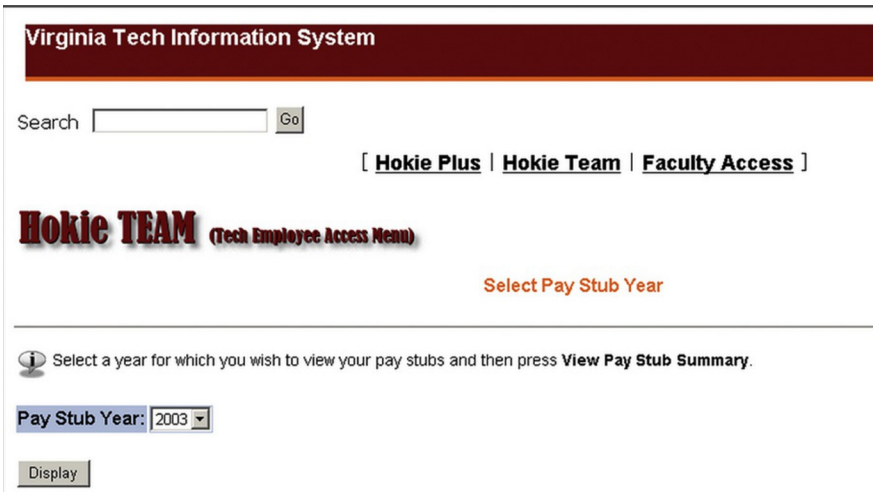


Fig. 32-19

*Cannot click on **View Pay Stub Summary**.*

This is another example of inconsistency of wording. The cognitive affordance in the line above the **Pay Stub Year** selection menu says press **View Pay Stub Summary**, but the label on the button to be pressed says **Display**. Maybe this big a difference in what is supposed to be the same is due to having different people working on different parts of the design. In any case, we noticed that in a subsequent version, someone had found and fixed the problem, as seen in Fig. 32-20.

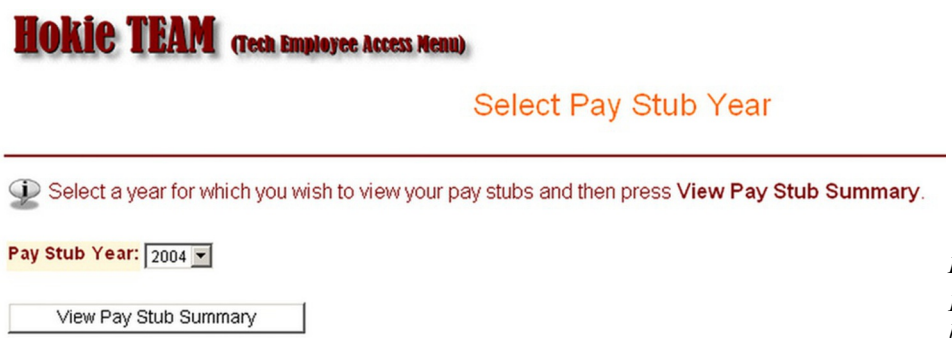


Fig. 32-20
Problem fixed with new button label.

In passing, we note an additional UX problem with each of these screens, the cognitive affordance **Select Pay Stub Year** above the line in the frame is redundant with **Select a year for which you wish to view your pay stubs** in the bottom section. We would recommend keeping the second one, as it's more informative and is grouped with the pull-down menu for year selection (which, in itself, might be sufficient without further instructions).

The first **Select Pay Stub Year** looks like some kind of title but is really kind of an orphan. The distance between this cognitive affordance and the user interface object to which it applies, plus the solid line, makes for a strong separation between two design elements that should be closely associated. Because it's unnecessary and separate from the year menu, it could be confusing.

Use different terms for different things, especially when the difference is subtle.

This is the flip side of the guideline that says to use the same terms for the same things. As we will see in the following example, terms such as **Add** can mean several different but closely related things. If you, the UX designer, don't distinguish the differences with appropriately precise terminology, it can lead to confusion for the user.

Example: The User Thought Files Were Already "Added"

When using Nero Express to burn CDs and DVDs for data transfer and backup, users put in a blank disc and choose the **Create a Data Disc** option and see the window shown in Fig. 32-21.

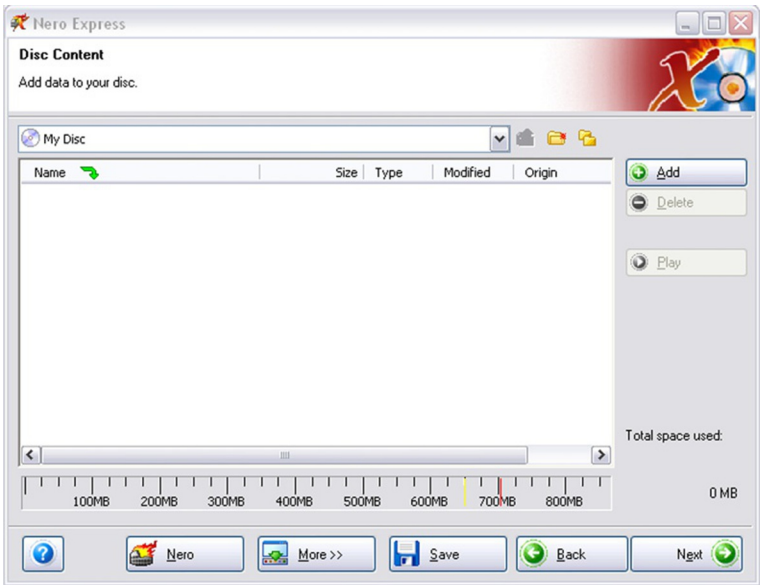


Fig. 32-21
*First Nero **Add** button.*

In the middle of this window is an empty space that looks like a file directory. Most users will figure out that this is for the list of the files and folders they want to put on the disc. At the top, where it will be seen only if the user looks around, it gives the cue: **Add data to your disc**. In the normal task path, there is really only one action that makes sense, which is clicking on the **Add** button at the top on the right-hand side.

This is taken by the user to be the way you add files and folders to this list. When users click on **Add**, they get the next window, shown in Fig. 32-22, overlapping the initial window.

This window also shows a directory space in the middle for browsing the files and folders and selecting those to be added to the list for the disc. The way that one commits the selected files to go on the list for the disc is to click on the **Add** button in this window. Here, the term **Add** really means to add the selected files to the disc list. In the first window, however, the term **Add** really meant proceed to file selection for the disc list, which is related but slightly different. Yes, the difference is subtle, but it's our job to be precise in wording.

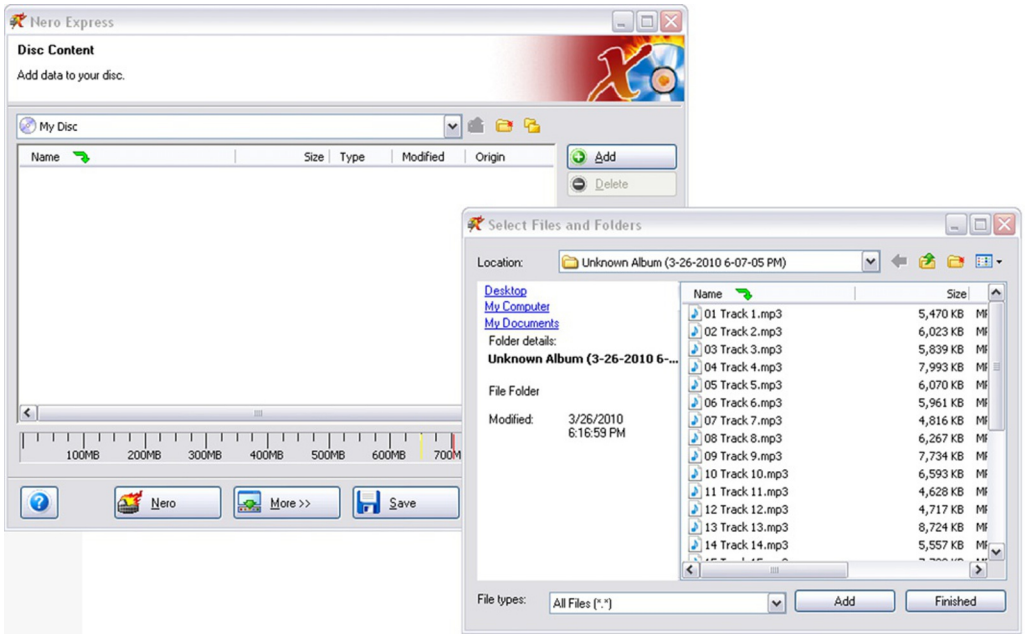


Fig. 32-22
*Another window and another **Add** button.*

Be consistent in the way that similar choices or parameter settings are made.

If a certain set of related parameters are all selected or set with one method, such as check boxes or radio buttons, then all parameters related to that set should be selected or set the same way.

Example: The Find Dialogue Box in Microsoft Word

Setting and clearing search parameters for the **Find** function are done with check boxes on the lower left-hand side (Fig. 32-23) and with pull-down menus at the bottom of the dialogue box for font, paragraph, and other format attributes and special characteristics. We have observed many users having trouble turning off the format attributes, which is because the "command" for that is different from all the others.

It's accomplished by clicking on the **No Formatting** button on the right-hand side at the bottom; see Fig. 32-23. Many users simply don't see that because nothing else uses a button to set or reset a parameter, so they are not looking for a button.

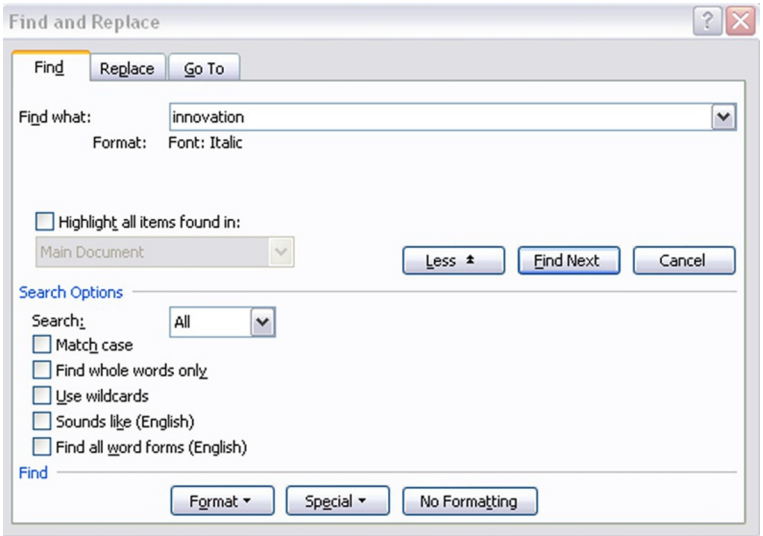


Fig. 32-23
Format with a menu but ***No Formatting*** *with a button.*

The following example is an instance of the same kind of inconsistency (not using the same kind of selection method for related parameters), only this example is from the world of food ordering.

Example: Circle Your Selections

In Fig. 32-24, you see an order slip for a sandwich at Au Bon Pain. Under **Create Your Own Sandwich** it says **Please circle all selections,** but the very next choice is between two check boxes for selecting the sandwich size. It's a minor thing that probably doesn't impact user performance, but, to a UX stickler, it stands out as an inconsistency in the design.

We wrap up this section on consistency of cognitive affordances with the following example about how many problems with consistency in terminology we found in an evaluation of one web-based application.

Example: Consistency Problems

In this example, we consider only problems relating to wording consistency from a lab-based UX evaluation of an academic web application for classroom support. We suspect this pervasiveness of inconsistency was due to having different people doing the design in different places and not having a project-wide custom style guide or not using one to document working terminology choices.

au bon pain

Name _____ ☐ Here ☐ To Go

CREATE YOUR OWN SANDWICH

☐ whole Please circle all selections (includes one meat, bread, spreads, and toppings) ☐ half

MEAT (select one; 1.75 each additional): Tuna; Smoked Ham; Roast Beef; Smoked Turkey; Grilled Chicken (not available as a 1/2 sandwich)

BREAD: Baguette; Multi-Grain Baguette; Sliced Country White; Sliced Multi-Grain; Sliced Tomato Herb; Lahvash; Croissant; Soft Roll; Bagel _____ (indicate choice of bagel)

ADD-ONS (.79 each): Swiss; Cheddar; Fresh Mozzarella; Bacon

SPREADS: Mayo; Herb Mayo; Dijon Mustard; Honey Dijon; Chili Dijon

TOPPINGS: Tomato; Romaine Lettuce; Red Onion; Cucumber

Fig. 32-24

***Circle all selections**, but size choice is by check boxes.*

In any case, when the design contains different terms for the same thing, it can confuse the user, especially the new user who is trying to conquer the system vocabulary. Here are some examples of our UX problem descriptions, "sanitized" to protect the guilty.

- The terms **Revise** and **Edit** were used interchangeably to denote an action to modify an information object within the application. For example, **Revise** is used as an action option for a selected object in the **Worksite Setup** page of **My Workspace**, whereas **Edit is** used inside the **Site Info** page of a given worksite.
- The terms "**worksite**" and "**site**" are used interchangeably for the same meaning. For example, many of the options in the menu bar of **My Workspace** use the term "**worksite**," whereas the **Membership** page uses "**site**," as in **My Current Sites**.
- The terms **Add** and **New** are used interchangeably, referring to the same concept. Under the **Manage Groups** option, there is a link for adding a group, called **New**. Most everywhere else, such as for adding an event to a schedule, the link for creating a new information object is labeled **Add**.
- The way that lists are used to present information is inconsistent:
  - In some lists, such as the list on the **Worksite Setup** page, check boxes are on the left-hand side, but for most other lists, such as the list on the **Group List** page, check boxes are on the right.
  - To edit some lists, the user must select a list item check box and then choose the **Revise** option in a menu bar (of links) at the top of the page and separated from the list. In other lists, each item has its own **Revise** link. For yet other lists there is a collection of links, one for each of the multiple ways the user can edit an item.

32.6.3.5 Controlling complexity of cognitive affordance content and meaning

Decompose complex instructions into simpler parts.

Cognitive affordances don't afford anything if they are too complex to understand or follow. Try decomposing long and complicated instructions into smaller, more meaningful, and more easily digested parts.

Example: Say What?

The cognitive affordance of Fig. 32-25 contains instructions that can bewilder even the most attentive user, especially someone in a wheelchair who needs to get out of there fast.

Fig. 32-25
Good luck in evacuating quickly.

Use layout and grouping of cognitive affordances to control content and meaning complexity.

Use appropriate layout and grouping by function of cognitive affordances to control content and meaning complexity.

Support user cognitive affordance content understanding through layout and spatial grouping to show relationships of task and function.

Group together objects and design elements associated with related tasks and functions.

Functions, user interface objects, and controls related to a given task or function should be grouped together spatially. The indication of relationship is strengthened by a graphical demarcation, such as a box around the group. Label the group with words that reflect the common functionality of the relationship. Grouping and labeling related data fields are especially important for data entry.

Don't group together objects and design elements that are not associated with related tasks and functions.

This guideline, the converse of the previous one, seems to be observed more often in the breach in real-world designs.

Example: Here Are Your Options

The Options dialogue box in Fig. 32-26, from an older version of Microsoft Word, illustrates a case where some controls are grouped incorrectly with some parameter settings.

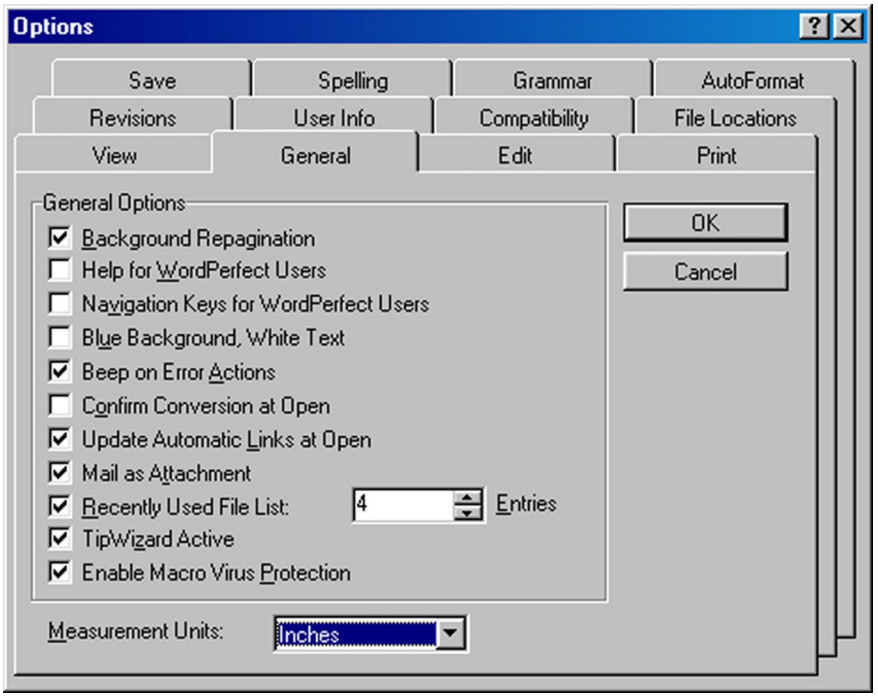


Fig. 32-26

OK *and* ***Cancel*** *controls on individual tab "card" (screen image courtesy of Tobias Frans-Jan Theebe).*

The metaphor in this design is that of a deck of tabbed index cards. The user has clicked on the **General** tab, which took the user to this **General** "card" where the user made a change in the options listed. While in the business of setting options, the user then wishes to go to another tab for more settings. The user hesitates, concerned that moving to another tab without saving the settings made in the current tab might cause them to be lost.

So, this user clicks on the **OK** to get closure for this tabbed card before moving on. To his surprise, the entire dialogue box disappears, and he must start over by selecting **Options** from the **Tools** menu at the top of the screen.

The surprise and extra work to recover were the price of the use of layout and grouping as an incorrect indication of the scope or range covered by the **OK** and **Cancel** buttons. Designers have made the buttons actually apply to the entire **Options** dialogue box, but they put the buttons on the currently open tabbed card, making them appear to apply just to the card or, in this case, just to the **General** category of options.

The **Options** dialogue box from a different version of Microsoft PowerPoint in Fig. 32-27 is a better design that places all the tabbed cards on a larger background and the **OK** and **Cancel** controls are on this background, showing clearly that the controls are grouped with the whole dialogue box and not with individual tabbed cards.

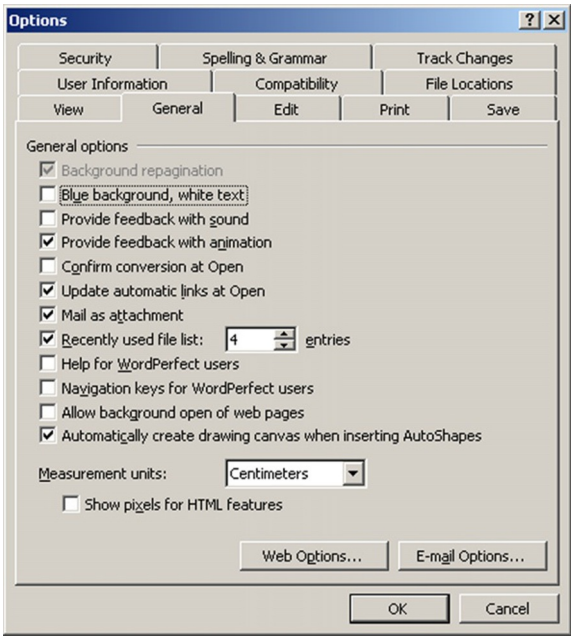


Fig. 32-27

***OK** and **Cancel** controls on background underneath all the tab "cards" (screen image courtesy of Tobias Frans-Jan Theebe).*

Example: Where to Put the Search Button?

The original design of some parameters associated with a search function in a digital library are shown in Fig. 32-28. The **Search** button is located right next to the OR radio-button choice at the bottom. Perhaps it's associated with the **Combine fields with** feature?

Fig. 32-28
Uncertain association with ***Search.***

No, it actually was intended to be associated with the entire search box, as shown in the "Suggested redesign" in Fig. 32-29.

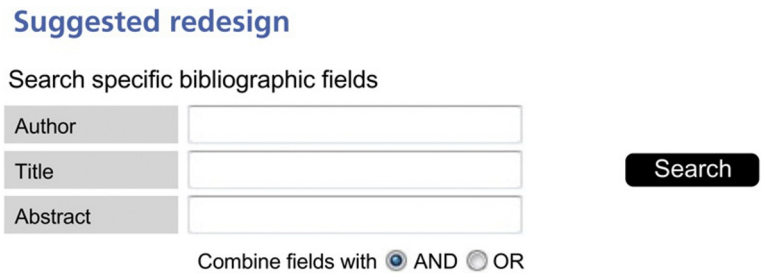


Fig. 32-29
Problem fixed with better layout and grouping.

Example: Are We Going to Eindhoven or Catalonia?

Here is a noncomputer (sort of) example from the airlines. While waiting in Milan to board a flight to Eindhoven, passengers saw the display shown in Fig. 32-30. As the display suggests, the Eindhoven flight followed a flight to Catalonia (in Spain) from the same gate.

As the flight to Catalonia began boarding, confusion started brewing in the boarding area. Many people were unsure about which flight was boarding, as both flights were displayed on the board. The main source of trouble was due to the way parts of the text were grouped in the flight announcements

Fig. 32-30
A sketch of the airline departure board in Milan.

on the overhead board. The state information **Embarco** (departing) was closer to the Eindhoven listing than to that of Catalonia, as shown in Fig. 32-30. So **Embarco** seemed to be grouped with and applied to the Eindhoven flight.

Confusion was compounded by the fact that it was 9:30 AM; the Catalonia flight was boarding late enough so that boarding could have been mistaken for the one to Eindhoven. Further conspiring against the waiting passengers was the fact that there were no oral announcements of the boardings, although there was a public-address system. Many Eindhoven passengers were getting into the Catalonia boarding line. You could see them turning Eindhoven passengers away, but still there was no announcement to clear up the problem.

Sometime later, the flight state information **Embarco** changed to **Chiuso** (closed), as seen in Fig. 32-31.

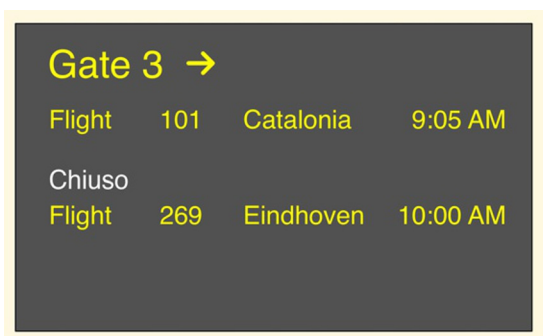


Fig. 32-31
Oh, no, Chiuso.

Many of the remaining Eindhoven passengers immediately became agitated, seeing the **Chiuso** and thinking that the Eindhoven flight was closed before they had a chance to board. In the end, everything was fine, but the poor layout of the display on the flight announcement board caused stress among passengers and extra work for the airline gate attendants. Given that this situation can occur many times a day, involving many people every day, the cost of this poor UX must have been very high, even though the airline workers seemed to be oblivious as they contemplated their cappuccino breaks.

Example: Hot Wash, Anyone?

In another simple example, in Fig. 32-32, we depict a row of push-button controls once seen on a clothes-washing machine.

The choices of **Hot wash/cold rinse**, **Warm wash/cold rinse**, and **Cold wash/cold rinse** all represent similar semantics (wash and rinse temperatures settings) and, therefore, should be grouped together. They are also expressed in similar syntax and words, so it's consistent labeling. However, because all three choices include a cold rinse, why not just say that with a separate label and not include it in all the choices?

The real problem, though, is that the fourth button, labeled **Start,** represents completely different functionality and should not be grouped with the other push buttons. Why do you think the designers made such an obvious mistake in

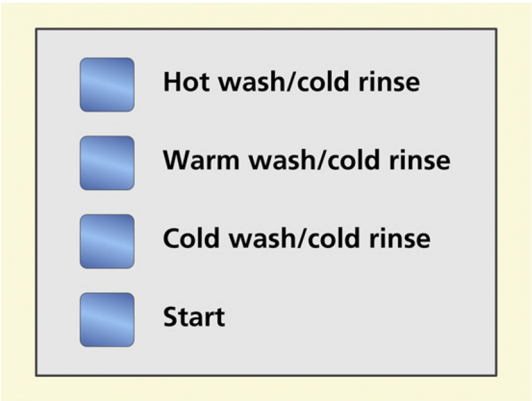


Fig. 32-32
Clothes-washing machine controls with one little inconsistency.

grouping by related functionality? We think it's because one single switch assembly is less expensive to buy and install than two separate assemblies. Here, cost won over usability.

Example: There Goes the Flight Attendant, Again

On an airplane flight once, we noticed a design flaw in the layout of the overhead controls for a pair of passengers in a two-seat configuration, a flaw that created problems for flight attendants and passengers. This control panel had push-button switches at the left and right for turning the left and right reading lights on and off.

The problem is that the flight attendant call switch was located just between the two light controls. It looked nice and symmetric, but its close proximity to the light controls made it a frequent target of unintended operation. On this flight, we saw flight attendants moving through the cabin frequently, resetting call buttons for numerous passengers.

In this design, switches for two related functions were separated by an unrelated one; the grouping of controls within the layout was not by function. Another reason calling for even further physical separation of the two kinds of switches is that light switches are used frequently, while the call switch is not.

32.6.3.6 Likely user choices and useful defaults

Sometimes it's possible to anticipate menu and button choices, choices of data values, and choices of task paths that users will most likely want or need to take. By providing direct access to those choices, and, in some cases, making them the defaults, we can help make the task more efficient for users.

Support user choices with likely and useful defaults.

Many user tasks require data entry into data fields in dialogue box and screens. Data entry is often a tedious and repetitive chore, and we should do everything we can to alleviate some of the dreary labor of this task by providing the most likely or most useful data values as defaults.

Example: What is the Date?

Many forms call for the current date in one of the fields. Using today's date as the default value for that field should be a no-brainer.

Example: Tragic Choice of Defaults

Here is a serious example of a case where the choice of default values resulted in dire consequences. This story was relayed by a participant in one of our UX short courses at a military installation. We cannot vouch for its verity but, even if it's apocryphal, it makes the point well.

A front-line spotter for missile strikes has a GPS unit on which he can calculate the exact location of an enemy facility on a map overlay. The GPS unit also serves as a radio through which he can send the enemy location back to the missile firing emplacement, which will send a missile strike with deadly accuracy.

He entered the coordinates of the enemy just before sending the message, but unfortunately, the GPS battery died before he could send the message. Because time was of the essence, he replaced the battery quickly and hit Send. The missile was fired and it hit and killed the spotter instead of the enemy.

When the old battery was removed, the system didn't retain the enemy coordinates and, when the new battery was installed, the system entered its own current GPS location as default values for the coordinates. Not having any useful alternatives, designers decided to use an easily obtained value, the local GPS coordinates of where the spotter was standing, figuring that the user would then change it.

In isolation from other important considerations, it was a bit like putting in today's date as the default for a date; it's conveniently available. But in this case, the result of that convenience was death by friendly fire. With a moment's thought, no one could imagine making the spotter's coordinates the default for aiming a missile. The problem was fixed immediately!

Provide the most likely or most useful default selections.

Among the most common violations of this guideline is the failure to select an item for a user when there is only one item from which to select, as illustrated in the next example.

Example: Only One Item to Select From

Here is a special case of applying this guideline where there is only one item from which to select. In this case, it was one item in a dialogue box list. When this user opened a directory in this dialogue box showing only one item, the **Select** button was grayed out because the user is required to select something from the list before the **Select** button becomes active. However, because there was only one item, the user assumed that the item would be selected by default.

When he clicked the grayed-out **Select** button, however, nothing happened. The user didn't realize that even though there is only one item in the list, the design requires selecting it before proceeding to click on the **Select** button. If no item is chosen, then the **Select** button doesn't give an error message nor does it prompt the user; it just sits there waiting for the user to do the "right thing." The difficulty could have been avoided by displaying the list of one item with that item already selected and highlighted, thus providing a useful default selection and allowing the **Select** button to be active from the start.

Offer most useful default cursor position.

It's a small thing in a design, but it can be so nice to have the cursor just where you need it when you arrive at a dialogue box or window in which you have to work. As a designer, you can save users the work and irritation of extra physical actions, such as an extra mouse click before typing, by providing appropriate default cursor location, for example, in a data field or text box, or within the user interface object where the user is most likely to work next.

Example: Please Set the Cursor for Me

Fig. 32-33 contains a dialogue box for planning events in a calendar system. Designers chose to highlight the frequency of occurrences of the event in terms of the number of weeks, in the **Weekly** section. This might be a little helpful to

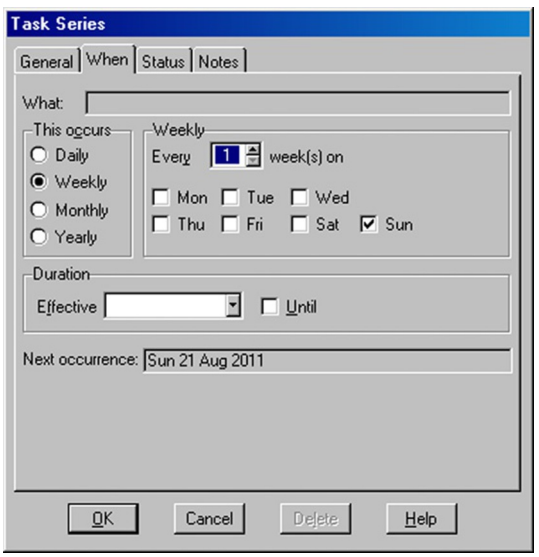


Fig. 32-33

Placement of default working location could be better (screen image courtesy of Tobias Frans-Jan Theebe).

users who will type a value into the "increment box" of this data field, but users are just as likely to use the up and down arrows of the increment box to set **Values**, in which case, the default highlighting doesn't help. Further evaluation will be necessary to confirm this, but it's possible that putting the default cursor in the **Effective Date** field at the bottom might be more useful.

32.6.3.7 Supporting human memory limitations in cognitive affordances

Earlier (Section 32.3) we elaborated on the concept of human memory limitations in UX design. In this section, we get to put this knowledge to work in specific UX design situations.

Relieve human short-term memory loads by maintaining task context visibly or audibly for the user.

Provide reminders to users of what they are doing and where they are within the task flow. Post important parts of the task context, parameters the user must keep track of within the task, so that the user doesn't have to commit them to memory.

Support human memory limits with recognition over recall.

For cases where choices, alternatives, or possible data entry values are known, designing to use recognition over recall means allowing the user to select an item from among choices rather than having to specify the choice strictly from memory. Selection among presented choices also makes for more precise communication about choices and data values, helping avoid errors from wording variations and typos.

One of the most important applications of this guideline is in the naming of files for an operation such as opening a file. This guideline says that we should allow users to select the desired file name from a directory listing rather than requiring the user to remember and type in the file name.

Example: What Do You Want, the Part Number?

To begin with, the cognitive affordance shown in Fig. 32-34 describing the desired user action is too vague and open-ended to get any kind of specific input from a user. What if the user doesn't know the exact model number and what kind of description is needed? This illustrates a case in which it would be better to use a set of hierarchical menus to narrow down the category of the product in mind and then offer a list in a pull-down menu to identify the exact item.

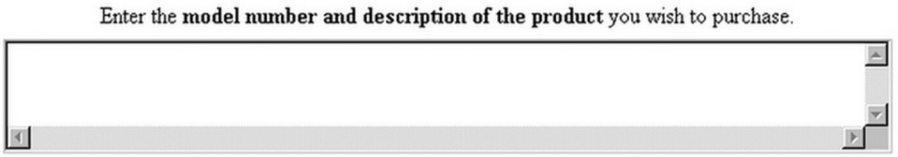


Fig. 32-34

What do you want, the part number?

Example: Help with Save As

In Fig. 32-35, we show a very early **Save As** dialogue box in a Microsoft Office application. At the top is the name of the current folder, and the user can navigate to any other folder in the usual way. But it doesn't show the names of files in the current folder.

This design precedes modern versions that show a list of existing files in this current folder, as shown in Fig. 32-36.

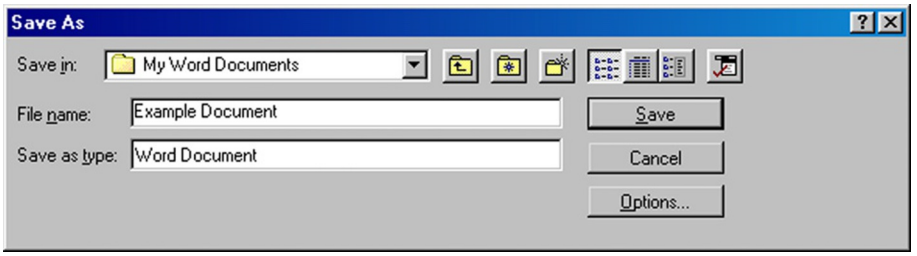


Fig. 32-35

*Early **Save As** dialogue box with no listing of files in current folder (screen image courtesy of Tobias Frans-Jan Theebe).*

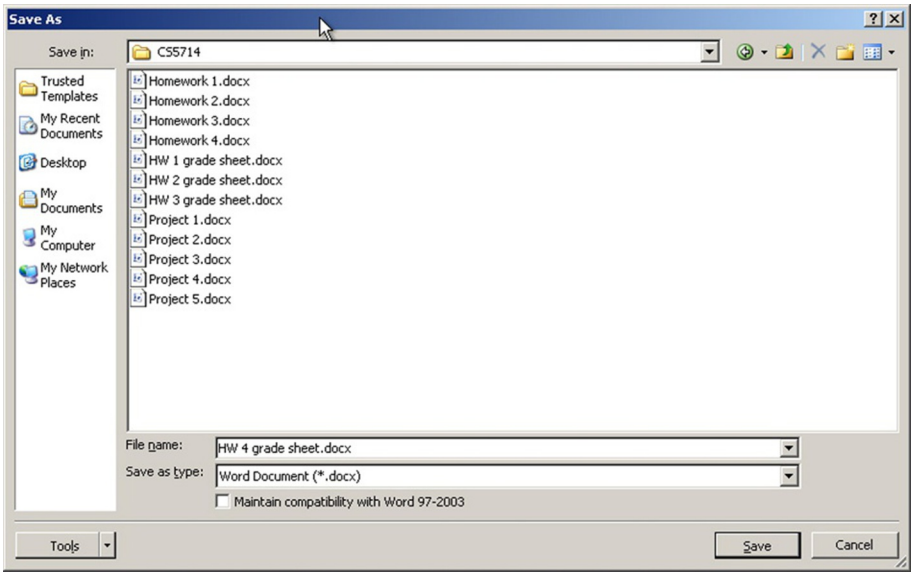


Fig. 32-36

Problem solved with listing of files in current folder.

This list supports memory by showing the names of other, possibly similar, files in the folder. If the user is employing any kind of implicit file-naming convention, it will be evident, by example, in this list.

For example, if this folder is for letters to the IRS and files are named by date, such as "letter to IRS, 3-30-2018," the list serves as an effective reminder of this naming convention. Further, if the user is saving another letter to the IRS here, dated 4-2-2018, that can be done by clicking on the 3-30-2018 letter and getting that name in the **File name:** text box, and, with a few keystrokes, editing it to be the new name.

Avoid requirement to retype or copy from one place to another.

In some applications, moving from one subtask to another requires users to remember key data or other related information and bring it to the second subtask themselves. For example, suppose a user selects an item of some kind during a task and then wishes to go to a different part of the application and apply another function to which that item is an input. We have experienced applications that required us to remember the item ourselves and reenter it as we arrived at the new functionality.

Be suspicious of usage situations that require users to write something down in order to use it somewhere else in the application; this is a sign of an opportunity to support human memory better in the design. As an example, consider a user of a Calendar Management System who needs to reschedule an appointment. If the design forces the user to delete the old one and add the new one, the user has to remember details and reenter them. Such a design doesn't follow this guideline.

Support special human memory needs in audio UX design.

Voice menus, such as telephone menus, are more difficult to remember because there is no visual reminder of the choices as there is in a screen display. Therefore, we have to organize and state menu choices in a way to reduce human memory load.

For example, we can give the most likely or most frequently used choices first because the deeper the user goes into the list, the more previous choices there are to remember. As each new choice is articulated, the user must compare it with each of the previous choices to determine the most appropriate one. If the desired item comes early, the user gets cognitive closure and doesn't need to remember the rest of the items.

32.6.3.8 Cognitive directness in cognitive affordances

Cognitive directness is about avoiding mental transformations for the user. It's about what Norman (1990, p. 23) calls "natural mapping." A good example from the world of physical actions is a lever that goes up and down on a console but is used to steer something to the left or the right. Each time it's used, the user must rethink the connection, "Let us see; lever up means steer to the left."

A classic example of cognitive directness, or the lack thereof, in product design is in the arrangement of knobs that control the burners of a cook top. If the spatial layout of the knobs is a spatial map to the burner configuration, it's easy to see which knob controls which burner. It seems easy, but many designs over the years have violated this simple idea, and users have frequently had to reconstruct their own cognitive mapping.

Avoid cognitive indirectness.

Support user cognitive affordance content understanding by presenting choices and information in cognitively direct expressions rather than in some kind of encoding that requires the user to make a mental translation. The objective of this guideline is to help the user avoid an extra step of translation, resulting in less cognitive effort and fewer errors.

Example: Rotating a Graphical Object

For a user to rotate a two-dimensional graphical object, there are two directions: clockwise and counterclockwise. While **Rotate Left** and **Rotate Right** are not technically correct, they might be better understood by many than **Rotate CW** and **Rotate CCW**. A better solution might be to show small graphical icons, circles with an arc arrow over the top pointing in clockwise and counterclockwise directions.

Example: Up and Down in Dreamweaver

Macromedia Dreamweaver is an application used to set up simple web pages. It's easy to use in many ways, but the old version we discuss here contains an interesting and definitive example of cognitive indirectness in its design. In the right-hand side pane of the site files window in Fig. 32-37 are local files as they reside on the user's PC.

The left-hand side pane of Fig. 32-37 shows a list of essentially the same files as they reside on the remote machine, the website server. As users interact with Dreamweaver to edit and test webpages locally on their PCs, they upload them

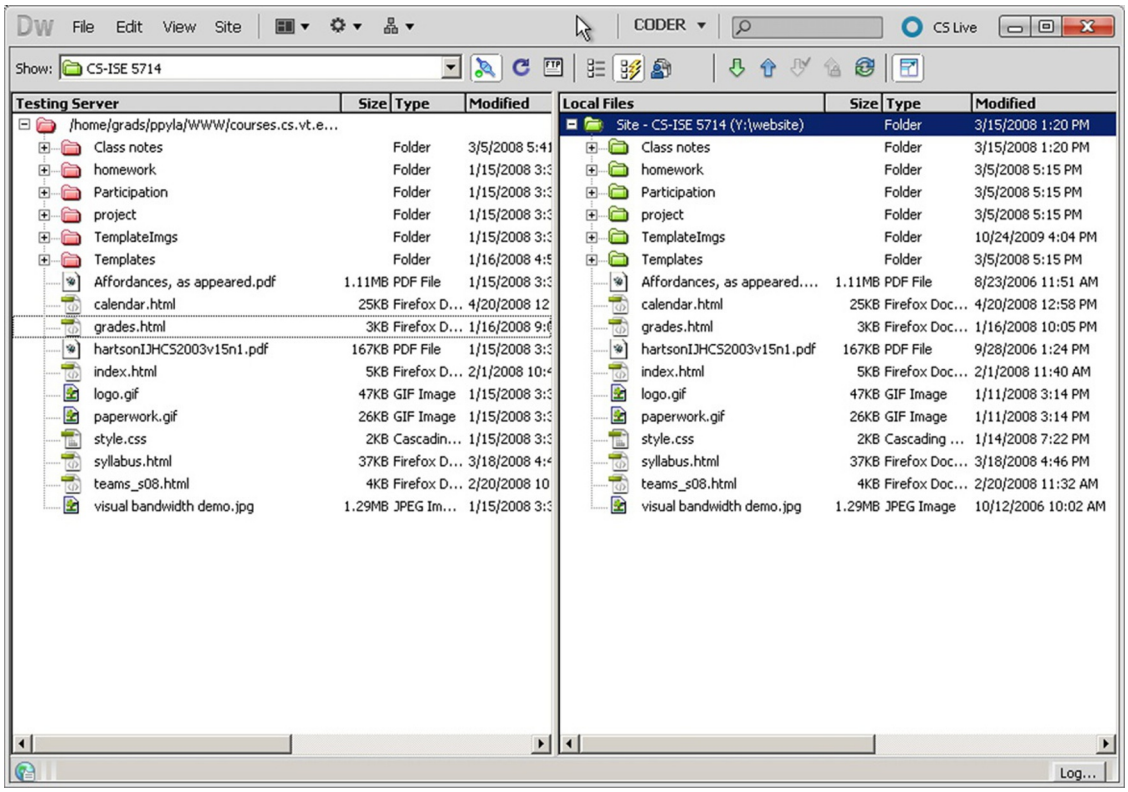


Fig. 32-37

Dreamweaver up and down arrows for uploading and downloading.

periodically to the server to make them part of the operational website. Dreamweaver has a convenient built-in "ftp" function to implement this file transfer. Uploading is accomplished by clicking on the up-arrow icon just above the **Local** site label, and downloading uses the down arrow.

The problem comes in when users, weary from editing web pages, click on the wrong arrow. The download arrow brings the remote copy of the just-edited file into the PC. Because the ftp function replaces files with the same name as new ones arriving without asking for confirmation, this feature is dangerous and can be costly. Click on the wrong icon, and you can lose a lot of work (and we have suffered from just that).

"Uploading" and "downloading" are system-centered, not usage-centered, terms and have arbitrary meaning about the direction of data flow, at least to the average nonsystems person. The up- and down-arrow icons do nothing to mitigate this poor mapping of meaning. Because the sets of files are on the left-hand side and right-hand side, not up and down, often users must stop and think about whether they want to transfer data left or right on the screen and

Physical affordance

A design feature that helps, aids, supports, facilitates, or enables user physical actions: clicking, touching, pointing, gesturing, and moving things (Section 30.3).

then translate it into "up" or "down." The icons for transfer of data should reflect this directly; a left arrow and a right arrow would do nicely. Furthermore, given that the "upload" action is the more frequent operation, making the corresponding arrow (left in this example) larger provides a better cognitive (and physical affordance in terms of click target size) affordance.

Example: The Surprise Action of a Car Heater Control

In Fig. 32-38, you can see a photo of the heater control in a car. It looks extremely simple; just turn the knob.

However, to a new user, the interaction here could be surprising. The control looks as though you grab the knob and the whole thing turns, including the numbers on its face. However, in actuality, only the outside rim turns, moving the indicator across the numbers, as seen in the sequence of Fig. 32-39.

So, if the user's mental model of the device is that rotating the knob clockwise slows down the heater fan, thinking the decreasing numbers will pass the indicator mark, the user is in for a surprise. In fact, the clockwise rotation moves the indicator to higher numbers, thus speeding up the heater fan. It can take users a long time to get used to that kind of a cognitive transformation.

Fig. 32-38

How does this car heater fan control work? (Photo courtesy of Mara Guimarães da Silva).

Fig. 32-39

Now you can see clockwise rotation where the outer rim is what turns (Photos courtesy of Mara Guimarães da Silva).

32.6.3.9 Complete information in cognitive affordances

Be complete in your design of cognitive affordances; include enough information for users to determine correct action.

For each label, menu choice, and so on, the designer should ask "Is there enough information and are there enough words used to distinguish cases?"

Support the user's understanding of cognitive affordances by providing complete and sufficient expression of meaning, to disambiguate, make more precise, and clarify. The expression of a cognitive affordance should be complete enough to allow users to predict the consequences of actions on the corresponding object.

Prevent loss of productivity due to hesitation, pondering.

Completeness helps the user distinguish alternatives without having to stop and contemplate the differences. Complete expressions of cognitive affordance meaning help avoid errors and lost productivity due to error recovery.

Use enough words for unambiguous labels.

Some people think button labels, menu choices, and verbal prompts should be terse; no one wants to read a paragraph on a button label. However, reasonably long labels are not necessarily bad, and adding words can add precision. Often, a verb plus a noun are needed to tell the whole story. For example, for the label on a button controlling a step in a task to add a record in an application, consider using **Add Record** instead of just **Add**.

As another example of completeness in labeling, for the label on a knob controlling the speed of a machine, rather than **Adjust** or **Speed**, consider using **Adjust Speed** or maybe even **Clockwise to Increase Speed**, which includes information about how to make the adjustment.

Add supplementary information, if necessary.

If you cannot reasonably get all the necessary information in the label of a button or tab or link, for example, consider using a fly-over pop-up to supplement the label with more information.

Give enough information for users to make confident decisions.

Example: What Do You Mean "Revert?"

In Fig. 32-40 is a message from an old version of Microsoft Word that has given us pause more than once. We think we know what button we should click, but we are not entirely confident, and it seems as though it could have a significant effect on our file.

Fig. 32-40
What are the consequences of "reverting?"

Give enough alternatives for user needs.

Few things are as frustrating to a user as a dialogue box or other UX object presenting choices that don't include the one alternative that user really needs.

32.6.3.10 User/usage centeredness in cognitive affordances

Employ usage-centered wording, the language of the user and the work context, in cognitive affordances.

We find that many of our students don't understand what it means to be user centered or usage centered in UX design. Mainly, it means to use the vocabulary and concepts of the user's work context rather than the vocabulary and context of the system. This difference between the language of the user's work domain and the language of the system is the essence of "translation" in the translation part of the Interaction Cycle.

As designers, we have to help users make that translation so they don't have to encode or convert their work domain vocabulary into the corresponding concepts in the system domain.

Example: What Is Your Phone?

As a modest but compelling example of user-centeredness in its simplest meaning, consider the difference between the terms "cell phone" and "mobile phone." The first term is system- and technology-centered because it refers to implementation technology (cellular networking). The second term is user-centered because it refers to how it is used (a usage capability).

Example: User-Centeredness versus Branding

In a town in South Carolina, we have seen a jewelry shop called *The Jeweler's Bench*. Not far from there is another called *Bling for You*. The first name in inwardly directed and jeweler-centered. The second is outwardly directed and customer-centered. How do these names fare with respect to branding? It depends on the image you wish to project. The first image is process-oriented and implies technical prowess and precision. The second is product-oriented and suggests beautiful things you can buy for yourself.

Example: A Mysterious Toaster

This example is about a toaster that we observed at a hotel brunch buffet, and it's right on target to illustrate user/usage centeredness. Users put bread on the input side of a conveyor belt going into the toaster system. Inside were overhead heating coils, and the bread came out the other end as toast.

The machine had a single control, a knob labeled **Speed** with additional labels for **Faster** (clockwise rotation of the knob) and **Slower** (counterclockwise rotation). A slower moving belt makes darker toast because the bread is under the heating coils longer; faster movement means lighter toast. Even though this concept is simple, there was a distinct language gulf and it led to a bit of confusion and discussion on the part of users we observed. The concept of speed somehow just didn't match their mental model of toast making. We even heard one person ask, "Why do we have a knob to control toaster speed? Why would anyone want to wait to make toast slowly when they could get it faster?" The knob had been labeled with the language of the system's physical control domain.

This is a good example of a design that fails to help the user with the translation from task or work domain language to system control language. Indeed, the knob did make the belt move faster or slower. But the user doesn't really care about the physics of the system domain; the user is living in the work domain of making toast. In that domain, the system control terms translate to **Lighter** and **Darker**, which would have been more effective as knob labels by helping bridge the gulf of execution from the system toward the user's task at hand.

32.6.3.11 Avoiding errors with cognitive affordances

The Japanese have a term, "poka-yoke," that means error proofing. It refers to a manufacturing technique to prevent parts of products from being made, assembled, or used incorrectly. Most physical safety interlocks are examples. For instance, interlocks in most automatic transmissions enforce a bit of safety by not allowing the driver to remove the key until the transmission is in park and not allowing shifting out of park unless the brake is depressed.

Find ways to anticipate and avoid user errors in your design.

Anticipating user errors in the workflow, of course, stems back to usage research, and concern for avoiding errors continues throughout requirements, design, and UX evaluation.

Help users avoid inappropriate and erroneous choices.

This guideline has three parts: one to disable the choices, the second to show the user that they are disabled, and the third to explain why they are disabled.

Disable buttons, menu choices to make inappropriate choices unavailable.

Help users avoid errors within the task flow by disabling choices in buttons, menus, and icons that are inappropriate at a given point in the interaction.

Gray out to make inappropriate choices appear unavailable.

As a corollary to the previous guideline, support user awareness of unavailable choices by making cognitive affordances for those choices *appear* unavailable, in addition to *being* unavailable. This is done by making some adjustment to the presentation of the corresponding cognitive affordance.

One way is to remove the presentation of that cognitive affordance, but this leads to an inconsistent overall display and leaves the user wondering where that cognitive affordance went. The conventional approach is to "gray out" the cognitive affordance in question, which is universally taken to mean the function denoted by the cognitive affordance still exists as part of the system but is currently unavailable or inappropriate.

But help users understand why a choice is unavailable.

If a system operation or function is not available or not appropriate, it's usually because the conditions for its use are not met. One of the most frustrating things for users, however, is to have a button or menu choice grayed out but no indication about *why* the corresponding function is unavailable, and what has to be done to get this button un-grayed and make the function available.

We suggest an approach that would be a break with traditional GUI object behavior but that could help avoid that source of user frustration. Clicking on or hovering over a grayed-out object could yield a pop up (e.g., a "tool tip") with this crucial explanation of why it's grayed out and what you must do to create the conditions to activate the function of that user interface object.

Example: When Am I Supposed to Click the Button?

In a document retrieval system, one of the user tasks is adding new keywords to existing documents, documents already entered into the system. Associated with this task is a text box for typing in a new keyword and a button labeled **Add Keyword**. The user was not sure whether to click on the **Add Keyword** button first to initiate that task or to type the new keyword and then click on **Add Keyword** to "put it away."

A user tried the former and nothing happened, no observable action and no feedback, so the user deduced that the proper sequence was to first type the keyword and then click the button. No harm was done except a little confusion and lost time. However, the same glitch is likely to happen again with other users and with this user at a later time.

The solution is to gray out the **Add Keyword** button to show when it doesn't apply, making it obvious that it's not active until a keyword is entered. Per our suggestion earlier, we could add an informative pop-up message that appears when someone clicks on the grayed-out button to the effect that the user must first type something into the new keyword text box before this button becomes active and allows the user to commit to adding that keyword.

32.6.3.12 Cognitive affordances for error recovery

Provide a clear way to undo and reverse actions.

As much as possible, provide ways for users to back out of error situations by "undo" actions. Although they are more difficult to implement, multiple levels of undo and selective undo among steps are more powerful for the user.

Offer constructive help for error recovery.

Users learn about errors through error messages as feedback, which is considered in the assessment part of the Interaction Cycle. Feedback occurs as part of the system response (Section 32.9.1). A system response designed to support error recovery will usually supplement the feedback with feed-forward, a cognitive affordance here in the translation part of the Interaction Cycle to help users know what actions or task steps to take for error recovery.

32.6.3.13 Cognitive affordances for modes

Modes are states in which actions have different meanings than the same actions in different states. The simplest example is a hypothetical email system. When in the mode of managing email files and directories, the command **Ctrl-S** means **Save**. However, when you are in the mode of composing an email message, **Ctrl-S** means **Send**. This design, which we have seen in the "old days," is an invitation to errors. Many a message has been sent prematurely out of the habit of doing a **Ctrl-S** periodically out of habit to be sure everything is saved.

The problem with most modes in UX design is the abrupt change of the meanings of user actions. It's often difficult for users to shift focus between modes, and when they forget to shift as they cross modal boundaries, the outcomes can be confusing and even damaging. It's a kind of bait and switch; you just get your users comfortable in doing something one way and then change the meaning of the actions they are using.

Modes within UX designs can also work strongly against experienced users, who move fast and habitually without thinking much about their actions. In a kind of "UX karate," they start leaning one way in one mode, and then their own usage momentum is used against them in the other mode.

Avoid confusing modalities.

If it's possible to avoid modes altogether, the best advice is to do so.

Example: Don't Be in a Bad Mode

Think about old school digital watches. Enough said.

Distinguish modes clearly.

If modes become necessary in your UX design, the next-best advice is to be sure that users are aware of each mode and avoid confusion across modes.

Use "good modes" where they help natural interaction without confusion

Not all modes are bad. The use of modes in design can represent a case for interpreting design guidelines, not just applying them blindly. The guideline to avoid modes is often good advice because modes tend to create confusion. But modes can also be used in designs in ways that are helpful and not at all confusing.

Example: Are You in a Good Mode?

An example of a good mode needed in a design comes from audio equalizer controls on the stereo in a particular car. As with most radio equalizers, there are choices of fixed equalizer settings, often called "presets," including audio styles such as voice, jazz, rock, classical, new age, and so on.

However, because there is no indication in the radio display of the current equalizer setting, you have to guess or have faith. If you push the Equalizer button to check the current setting, it changes the setting, and then you have to toggle back through all the values to recover the original setting. This is a nonmodal design because the Equalizer button means the same thing every time you push it. It's consistent; every button push yields the same result: toggling the setting.

It would be better to have a slightly moded design so that it starts in a "display mode," which means an initial button push causes it to display the current setting without changing the setting so that you can check the equalizer setting without disturbing the setting itself. If you do wish to change the setting, you push the same Equalizer button again within a certain short time period to change it to the "setting mode" in which button pushes will toggle the setting. Most such buttons behave in this good, moded way, except in this particular car.

32.6.4 Task Structure

In Fig. 32-41, we highlight the "task structure" portion of the translation part of the Interaction Cycle.

Support of task structure in this part of the Interaction Cycle means supporting user needs with the logical flow of tasks and task steps, including human memory support in the task structure; task design simplicity, flexibility, and efficiency; maintaining the locus of control with the user within a task; and offering natural direct manipulation interaction.

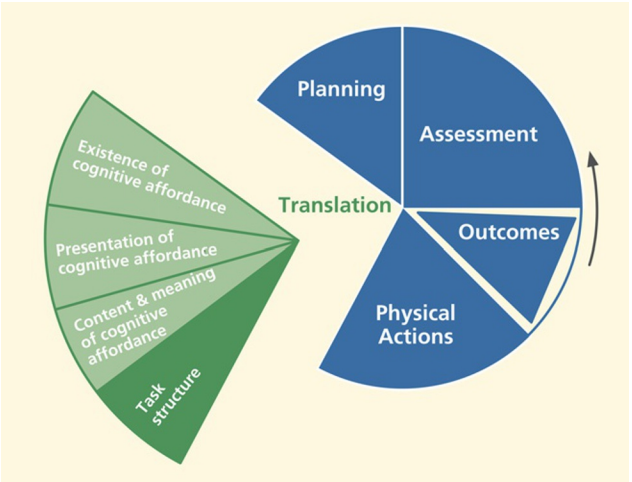


Fig. 32-41
The task structure part of translation.

32.6.4.1 Human working memory loads in task structure

Support human memory limitations in the design of task structure.

The most important way to support human memory limitations within the design of task structure is to provide task closure as soon and as often as possible, avoiding interruption and stacking of subtasks. This means "chunking" tasks into small sequences with closure after each part.

While it may seem tidy from the computer science point of view to use a "preorder" traversal of the hierarchical task structure, it can overload the user's working memory, requiring stacking of context each time the user goes to a deeper level and "popping" the stack, or remembering the stacked context, each time the user emerges up a level in the structure.

Interruption and stacking occur when the user must consider other tasks before completing the current one. Having to juggle several "balls" in the air, several tasks in a partial state of completion, adds an often-unnecessary load to human memory.

32.6.4.2 Design task structure for flexibility and efficiency

Support user with effective task structure and interaction control.

Support user needs for flexibility within the logical task flow by providing alternative ways to do tasks. Meet user needs for efficiency with shortcuts for frequently performed tasks and provide support for task thread continuity, supporting the most likely next step.

Provide alternative ways to perform tasks.

One of the most striking observations during task-based UX evaluation is the amazing variety of ways users approach task structure. Users take paths never imagined by the designers.

There are two ways designers can become attuned to this diversity of task paths. One is through careful attention to multiple ways of doing things in user research, and the other is to leverage observations of such user behavior in UX evaluation. Don't just discount observations of users gone "astray" as "incorrect" task performance; try to learn about valuable alternative paths.

Provide shortcuts.

No one wants to have to make too many mouse clicks or other user actions to complete a task, especially in complex task sequences (Wright, Lickorish, & Milroy, 1994). For efficiency within frequently performed tasks, experienced users especially need shortcuts, such as "hot key" (or "accelerator key") alternatives for other more complicated action combinations, such as selecting choices from pull-down menus.

Keyboard alternatives are particularly useful in tasks that otherwise require keyboard actions such as form filling or word processing; staying within the keyboard for these "commands" avoids having the physical "switching" actions required for moving between (for example) the keyboard and mouse, two physically different devices.

32.6.4.3 Grouping for task efficiency

Provide logical grouping in layout of objects.
Group together objects and functions related by task or user work activity.

Under the topic of layout and grouping to control content and meaning complexity, we grouped related things to make their meanings clear. Here, we advocate grouping objects and other things related to the same task or user work activity as a means of conveniently having the needed components for a task at hand. This kind of grouping can be accomplished spatially with screen or other device layout, or it can be manifest sequentially, as in a sequence of menu choices.

As Norman (2006) illustrates, in a taxonomic "hardware store" organization, hammers of all different kinds are all hanging together, and all different kinds of

nails are organized in bins somewhere else. But a carpenter organizes his or her tools so that the hammer and nails are in proximity because the two are used together in work activities.

But avoid grouping of objects and functions if they need to be dealt with separately.

Grouping user interface objects such as buttons, menus, value settings, and so on, creates the impression that the group comprises a single focus for user action. If more is needed for that task goal, each requiring separate actions, don't group the objects tightly together, but make clear the separate objectives and the requirement for separate actions.

Example: Oops, I Forgot to Do the Rest of It

A dialogue box sketch from a paper prototype of a Ticket Kiosk System is shown in Fig. 32-42.

It contains two objectives and two corresponding objects for value settings by the user—proximity of the starting time of a movie and the distance of the movie theater from the kiosk. Most of the participants who used this dialogue box as part of a ticket-buying task made the first setting and clicked on Continue, not noticing the second component. The solution that worked was to separate the two value-setting operations into two dialogue boxes, forcing a separation of the focus and linearizing the two actions.

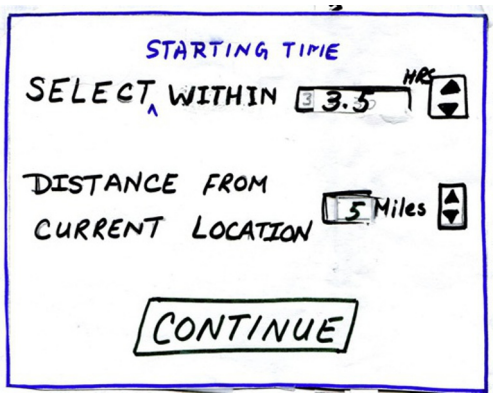


Fig. 32-42
An overloaded dialogue box in a paper prototype.

32.6.4.4 Task thread continuity: Anticipating the most likely next step or task path

Support task thread continuity by anticipating the most likely next task, step, or action.

Task thread continuity is a design goal relating to task flow in which the user can pursue a task thread of possibly many steps without an interruption or "discontinuity." It's accomplished in design by anticipating most likely and other possible next steps at any point in the task flow and providing, at hand, the necessary cognitive, physical, and functional affordances to continue the thread.

The likely next steps to support can include tasks or steps the user may wish to take but which are not necessarily part of what designers envisioned as the "main" task thread. Therefore, these various task directions might not be identified by pure task analysis, but are steps that a practitioner or designer might see in usage research while watching users perform the tasks in a real work activity context. Effective observation in UX evaluation also can reveal diversions, branching, and alternative task paths that users associate with the main thread.

Attention to task thread continuity is especially important when designing the contents of context menus, right-click menus associated with objects or steps in tasks. It's also important when designing message dialogue boxes that offer branching in the task path, which is when users need at hand other possibilities associated with the current task.

Example: If You Tell Them What They Should Do, Help Them Get There

Probably the most defining example of task thread continuity is seen in a message dialogue box that describes a problematic system state and suggests one or more possible courses of action as a remedy. But then the user is frustrated by a lack of any help in getting to these suggested new task paths.

Task thread continuity is easily supported by adding buttons that offer a direct way to follow each of the suggested actions. Suppose a dialogue box message tells a user that the page margins are too wide to fit on a printed page and suggests resetting page margins so that the document can be printed. It's enormously helpful if this guideline is followed by including a button that will take the user directly to the page margin setup screen.

Example: Seeing the Query With the Results

Designers of information retrieval systems sometimes see the task sequence of formulating a query, submitting it, and getting the results as closure on the

task structure, and it often is. So, in some designs, the query screen is replaced with the results screen. However, for many users, this is not the end of the task thread.

Once the results are displayed, the next step is to assess the success of the retrieval. If the query is complex or much has happened since the query was submitted, the user will need to review the original query to determine whether the results were what was expected. The next step may be to modify the query and try again. So, this often-simple linear task can have a thread with larger scope. The design should support these likely alternative task paths by not *replacing* the query display with that of the results and by providing an option to modify the query from the results page.

Example: May We Help You Spend More Money?

Designers of successful online shopping sites such as Amazon.com have figured out how to make it convenient for shoppers by providing for likely next steps (seeing and then buying) in their shopping tasks. They support convenience in ordering with the ubiquitous **Buy It Now** or **Add to Cart** buttons. They also support product research. If a potential customer shows interest in a product, the site quickly displays links to reviews, other products, accessories that go with it, and alternative similar products that other customers have bought.

Example: What If I Want to Save It In a New Folder?

In early Microsoft Office applications, the **Save As** dialogue box didn't contain the icon for creating a new folder (the second icon to the left of **Tools** in the dialogue box in Fig. 32-43). Eventually, designers realized that, as part of the **Save As** task, users had to think about where to put the file, and, as part of that planning for organizing their file structures, they often needed to create new folders to modify or expand the current file structure.

Early users had to back out of the **Save As** task and go to Windows Explorer, navigate to the proper context, create the folder, and then return to the Office application and do the **Save As** all over again. By including the ability to create a new folder within the **Save As** dialogue box, this likely next step was accommodated directly.

In some cases, the most likely next step is so likely that task thread continuity is supported by adding a slight amount of automation and doing the step for the user. The following example is about a case in which this kind of help was needed.

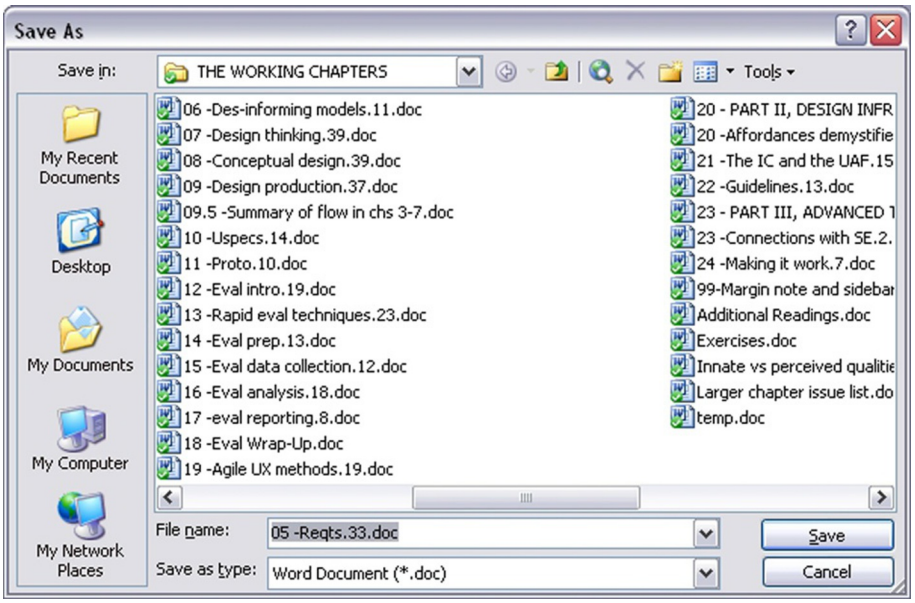


Fig. 32-43
Addition of an icon to create a new file in the ***Save As*** *dialogue box.*

Example: Resetting Over and Over

For frequent users of Word, the Outline view helps organize material within a document. You can use the Outline view to move quickly from where you are in the document to another specific location. In the Outline view, you get a choice of the number of levels of outline to be shown. It's common for users to want to keep this Outline view level setting at a high level, affording a view of the whole outline. So, for many users, the most likely-used setting would be that high setting. Many users rarely even choose anything else.

Regardless of any user's level preferences, if a user goes to the Outline view, it's because he or she wants to see and use the outline. But the default level setting in Word's Outline view is the only setting that really is not an outline. The default Word Outline view, **Show all levels**, is a useless mash-up of outline parts and nonoutline text. As a default, this setting is the least useful for anyone.

Every time users launch a Word document, they face that annoying Outline view setting. Even once you have shown your preference by setting the level, the system inevitably strays away from that setting during editing, forcing you to reset it frequently. Frequent users of Word will have made this setting thousands of times over the years. Why can't Word help users by saving the settings they use so consistently and have set so often?

Designers might argue that they cannot assume they know what level a user needs, so they follow the guideline to "give the user control." But why design

a default that guarantees the user will have to change it instead of something that might be useful to at least some users some of the time? Why not detect the highest level present in the document and use that as a default? After all, the user did request to see the outline.

Example: Why Make Me Choose from Just One Thing?

Earlier, we described an example in which the user had to select an item from a dialogue box list of choices, even though there was only one item in the list. Our point there was that preselecting the item for the user made for a useful default.

The same idea applies here: When there is only one choice, the designer can support user efficiency through task thread continuity by assuming the most likely next action to be selecting that only choice and making the selection in advance for the user. An example of this comes from Microsoft Outlook.

When an Outlook user selects **Rules and Alerts** from the **Tools** menu and clicks on the **Run Rules Now** button, the **Run Rules Now** dialogue box appears. In cases where there is only one rule, it's highlighted in the display, making it *look* selected. However, be careful, that rule is not selected; the highlighting is a false affordance.

Look closely and you see that the checkbox to its left is unchecked, and *that* is the indication of what is selected. The result is the **Run Now** button is grayed out, causing some users to pause in confusion about why the rule cannot now be run. Most such users figure it out eventually, but lose time and patience in the confusion. This UX glitch can be avoided by preselecting this only choice as the default.

32.6.4.5 Not undoing user work

Make the most of user's work.
Don't make the user redo any work.
Don't require users to reenter data.

Don't you hate it when you fill out part of a form and go away to get more information or to attend temporarily to something else, and, when you come back, you return to an empty form? Or, when you encounter an error with the data in one field, the form comes back empty? This usually comes from lazy programming because it takes some buffering to retain your partial information. Don't do this disservice to your users.

Retain user state information.

Retention helps users keep track of state information, such as user preferences, that they set in the course of usage. It's exasperating to have to reset preferences and other state settings that don't persist across different work sessions.

Example: Hey, Remember What I Was Doing?

It would help users if Windows could be a little bit more helpful in keeping track of the focus of their work, especially keeping track of where they have been working within the directory structure. Too often, you have to reestablish your work context by searching through the whole file directory in a dialogue box to, say, open a file.

Then, later, if you wish to do a **Save As** with the file, you may have to search that whole file directory again from the top to place the new file near the original one. We are not asking for built-in artificial intelligence, but it would seem that if you are working on a file in a certain part of the directory structure and want to do a **Save As**, it's very likely that the file is related to the original, and, therefore, needs to be saved close to it in the file structure.

Fortunately, in Windows 10, the **Save As** function now does offer choices of recent files and folders to help solve this problem of keeping your place in the directory structure (Fig. 32-44).

32.6.4.6 Keeping users in control

Avoid the feeling of loss of control.

Sometimes, although users are still actually in control, interaction dialogue can make users feel as though the computer is taking control. Although designers may not give a second thought to language such as, "You need to answer your email," these words can project a bossy attitude to users. Something such as, "You have new email," or, "New email is ready for reading," conveys the same meaning but does so in a way that helps users feel that they are not being commanded to do something; they can respond whenever they find it convenient.

Avoid real loss of control.

More bothersome to users and more detrimental to productivity is a real loss of user control. You, the designer, may think you know what is best for the user, but

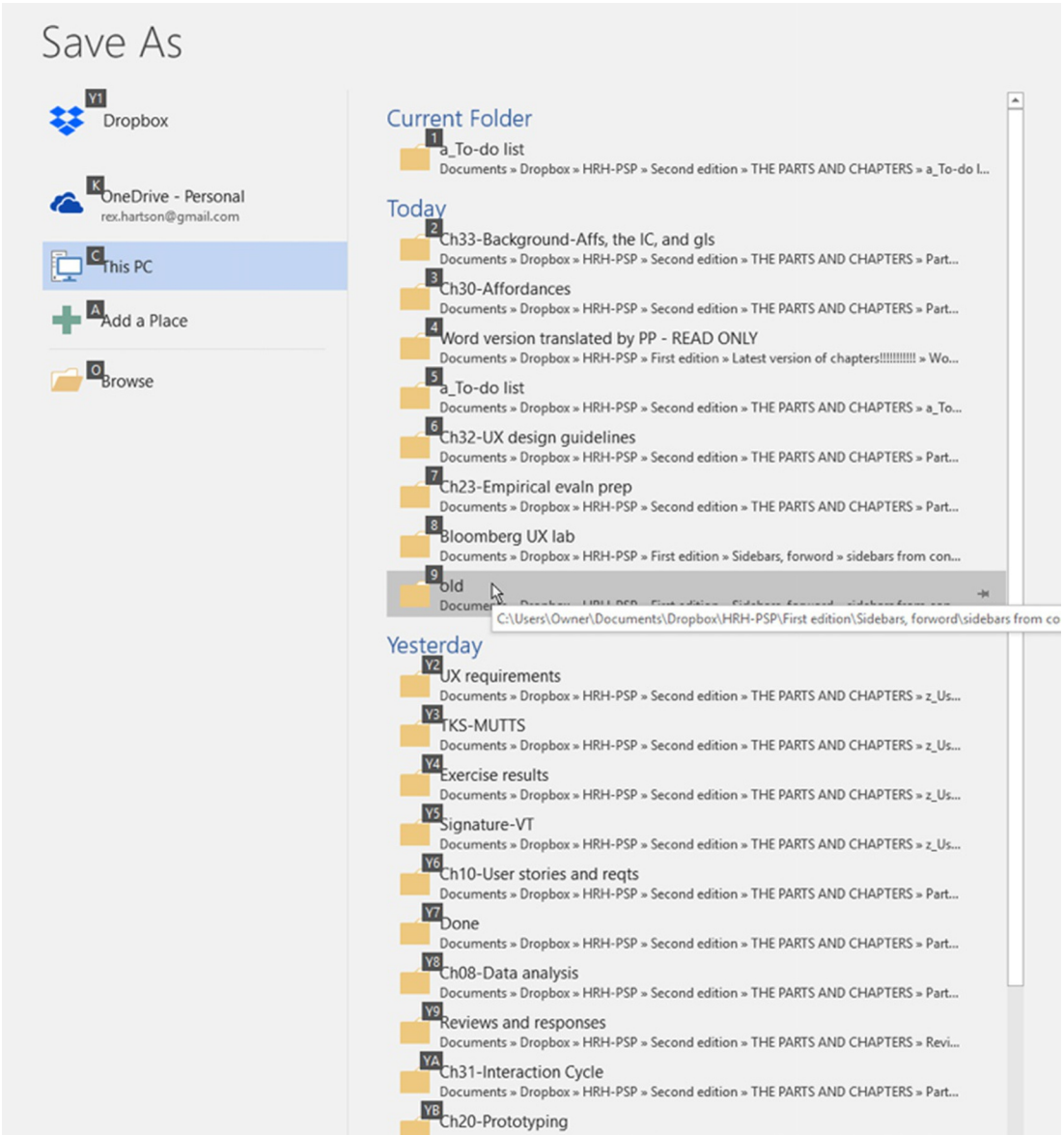


Fig. 32-44

Windows 10 ***Save As*** *help with recent working locations in the directory.*

you would do best to avoid the temptation of being high-handed in matters of control within interaction. Few kinds of user experience more readily give rise to anger in users than a loss of control. It doesn't make them behave the way you think they should; it only forces them to take extra effort to work around your design.

One of the most maddening examples of loss of user control we have experienced comes from EndNote, an otherwise powerful and effective bibliographic support application for word processing. This problem has existed in every version of EndNote we have ever used. When EndNote is used as a plug-in to Microsoft Word, it can be scanning your document invisibly for actions to take with regard to your bibliographic citations.

If an action is deemed necessary, for example, to format an in-line citation, EndNote often arbitrarily takes control away from a user doing editing and moves the cursor to the location where the action is needed, often many pages away from the focus of attention of the user and usually without any indication of what happened. At that point, users are probably not interested in thinking about bibliographic citations, but are more concerned with their task at hand, such as editing. All of a sudden, control is jerked away, and the working context disappears. It takes extra cognitive energy and extra physical actions to get back to where the user was working. The worst part is that it can happen repeatedly, each time with an increasingly negative emotional user reaction. I have actually uninstalled EndNote for a time while working on the chapters of this book!

Always provide a way for the user to "bail out" of an ongoing operation.

Don't trap a user in an interaction. Always allow a way for users to escape if they decide not to proceed after getting part way into a task sequence. The usual way to design for this guideline is to include a **Cancel**, usually as a dialogue box button.

32.7 PHYSICAL ACTIONS

Physical actions guidelines support users in doing physical actions, including typing, clicking, dragging in a GUI, scrolling on a web page, speaking with a voice interface, walking in a virtual environment, moving one's hands in gestural interaction, and gazing with the eyes. This is the one part of the user's Interaction Cycle where there is essentially no cognitive component; the user already knows what to do and how to do it.

Issues here are limited to how well the design supports the physical actions of doing it, acting upon user interface objects to access all features, and functionality within the system. The two primary areas of design considerations are how well the design supports users in sensing the object(s) to be

Fitts' law

An empirically-based theory expressed as a set of mathematical formulae that predict the time to make a cursor or other movement is proportional to log2 of distance moved and inversely proportional to log2 of target cross-section normal to the direction of movement (Section 30.3.2.5).

manipulated and how well the design supports users in doing the physical manipulation. As a simple example, it's about seeing a button and clicking on it.

Physical actions are the one place in the Interaction Cycle where physical affordances are relevant, where you will find issues about Fitts' law, manual dexterity, physical disabilities, awkwardness, and physical fatigue.

32.7.1 Sensing Objects of Physical Actions

In Fig. 32-45, we highlight the "sensing user interface object" part within the breakdown of the physical actions part of the Interaction Cycle.

32.7.1.1 Sensing objects to manipulate

The "Sensing user interface object" portion of the physical actions part is about designing to support user sensory (for example, visual, auditory, or tactile) needs in locating the appropriate physical affordance quickly in order to manipulate it. Sensing for physical actions is about presentation of physical affordances, and the associated design issues are similar to those of the presentation of cognitive affordances in other parts of the Interaction Cycle,

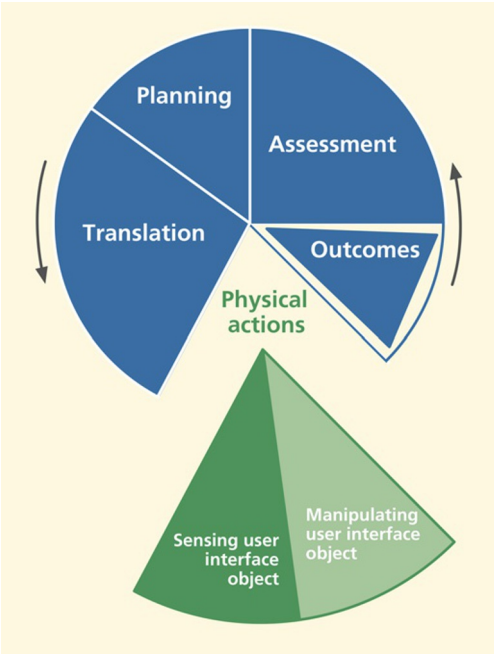


Fig. 32-45

Sensing the user interface (UI) object, within physical actions.

including visibility, noticeability, findability, distinguishability, discernibility, sensory disabilities, and presentation medium.

Support users making physical actions with effective sensory affordances for sensing physical affordances.

Make objects to be manipulated visible, discernable, legible, noticeable, and distinguishable. When possible, locate the focus of attention (the cursor, for example) near the objects to be manipulated.

32.7.1.2 Sensing objects during manipulation

Not only is it important to be able to sense objects statically to initiate physical actions, but users also need to be able to sense the cursor and the physical affordance object dynamically to keep track of them during manipulation. As an example, in dragging a graphical object, the user's dynamic sensory needs are supported by showing an outline of the graphical object, aiding its placement in a drawing application.

As another very simple example, if the cursor is the same color as the background, the cursor can disappear into the background while moving it, making it difficult to judge how far to move the mouse back to get it visible again.

32.7.2 Help User in Doing Physical Actions

In Fig. 32-46, we highlight the "manipulating user interface object" part within the breakdown of the physical actions part of the Interaction Cycle.

This part of the Interaction Cycle is about supporting user physical needs at the time of making physical actions; it's about making user interface object manipulation physically easy. It's especially about designing to make physical actions efficient for expert users.

Physicality

Referring to real direct physical interaction with real physical (hardware) devices like in the grasping and moving of knobs and levers (Section 30.3.2.4).

Support user with effective physical affordances for manipulating objects, help in doing actions.

Issues relevant to supporting physical actions include awkwardness and physical disabilities, manual dexterity and Fitts' law, plus haptics and physicality.

32.7.2.1 Awkwardness and physical disabilities

One of the easiest aspects of designing for physical actions is avoiding awkwardness. It's also one of the easiest areas in which to find existing problems in UX evaluation.

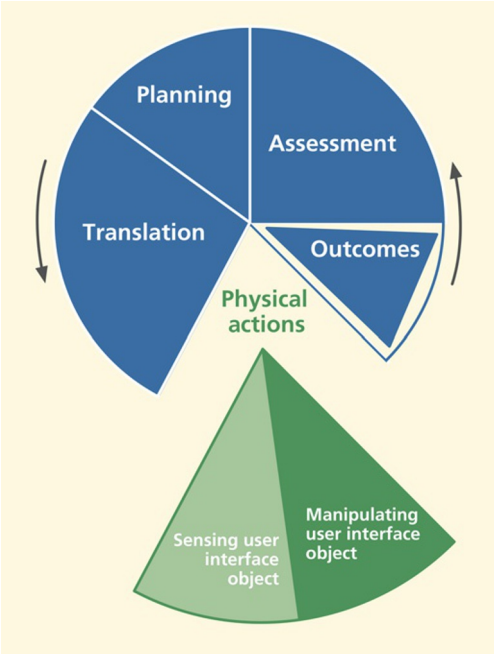


Fig. 32-46
Manipulating the user interface (UI) object within physical actions.

Avoid physical awkwardness.

Issues of physical awkwardness are often about time and energy expended in physical motions. The classic example of this issue is a user having to alternate constantly among multiple input devices such as between a keyboard and a mouse or between either device and a touchscreen.

This device switching involves constant "homing" actions that require time-consuming and effortful distraction of cognitive focus and visual attention. Keyboard combinations requiring multiple fingers on multiple keys can also be awkward user actions that hinder smooth and fast interaction.

Accommodate physical disabilities.

Not all human users have the same physical abilities—range of motion, fine motor control, vision, or hearing. Some users are innately limited; some have disabilities due to accidents. Although in-depth coverage of accessibility issues is beyond our scope, accommodation of user disabilities is an extremely important part of designing for the physical actions part of the Interaction Cycle.

32.7.2.2 Manual dexterity and Fitts' law

Design issues related to Fitts' law are about movement distances, mutual object proximities, and target object size. Performance is reckoned in terms of both time and errors. In a strict interpretation, an error would be clicking anywhere except on the correct object. A more practical interpretation would limit errors to clicking on incorrect objects that are near the correct object; this is the kind of error that can have a more negative effect on the interaction. This discussion leads to the following guidelines.

Design layout to support manual dexterity and Fitts' law.

Support targeted cursor movement by making selectable objects large enough.

The bottom line about sizes and cursor movement among targets is simple: small objects are harder to click on than large ones. Give your interaction objects enough size, both in cross-section for accuracy in the cursor movement direction and in their depth to support accurate termination of movement within the target object.

Group clickable objects related by task flow close together.

Avoid fatigue and slow movement times. Large movement distances require more time and can lead to more targeting errors. Short distances between related objects will result in shorter movement times and fewer errors.

But don't group objects too close, and don't include unrelated objects in the grouping.

Avoid erroneous selection that can be caused by close proximity of target objects to nontarget objects.

Example: Oops, I Missed the Icon

A software drawing application has a very large number of functions, most of which are accessible via small icons in a tool bar. Each function can also be invoked by another way (e.g., a menu choice), but our observations tell us that expert users like to use the tool bar icons.

But there are problems in using the icons because there are so many icons, they are small, and they are crowded together. This, combined with the fast actions of experienced users, leads to clicking on the wrong icon more often than users would like.

Physical overshoot

Moving an object, a cursor, a slider, a lever, a switch too far in making the physical action, beyond where it was intended to be (Section 30.3.2.6).

32.7.2.3 Constraining physical actions to avoid physical overshoot errors

Design physical movement to avoid physical overshoot.

Just as in the case of cursor movement, other kinds of physical actions can be at risk for overshoot, extending the movement beyond what was intended. This concept is best illustrated by the hair dryer switch example that follows.

Example: Blow Dry, Anyone?

Suppose you are using a hair dryer on the low setting, and you are ready to switch it off. To move the hair dryer switch takes a certain threshold pressure to overcome initial resistance. Once in motion, however, unless the user is adept at reducing this pressure instantly, the switch can move beyond the intended setting.

A strong detent at each switch position can help prevent the movement from overshooting, but it's still easy to push the switch too far, as the photo of a hair dryer switch in Fig. 32-47 illustrates. Starting in the **LOW** position and pushing the switch toward **OFF**, the switch configuration makes it easy to move accidentally beyond **OFF** over to the **HIGH** setting.

This physical overshoot is preventable with a switch design that goes directly from **HIGH** to **LOW** and then to **OFF** in a logical progression. Having the

Fig. 32-47

A hair dryer control switch inviting physical overshoot.

OFF position at one end of the physical movement is a kind of physical constraint or boundary condition that allows you to push the switch to **OFF** firmly and quickly without a careful touch or worrying about overshooting.

Why do essentially all hair dryers have this UX flaw? It's probably easier to manufacture a switch with the neutral **OFF** position in the middle.

32.7.2.4 Haptics and physicality

Haptics is about the sense of touch and physical grasping, and physicality is about real physical interaction using real physical devices, such as real knobs and levels, instead of "virtual" interaction via "soft" devices.

Include physicality in your design when the alternatives are not as satisfying to the user.

Example: Beamer Without Knobs

The BMW iDrive idea seemed so good on paper. It was simplicity in itself. No panels cluttered with knobs and buttons. How cool and forward looking. Designers realized that drivers could do *anything* via a set of menus. But drivers soon realized that the controls for everything were buried in a maze of hierarchical menus. No longer could you reach out and tweak the heater fan speed without looking. Fortunately, physical knobs are now coming back in BMWs.

Example: Roger's New Microwave

Here is an old email from our friend, Roger Ehrich, only slightly edited:

> Hey Rex, since our microwave was about 25 years old, we worried about radiation leakage, so we reluctantly got a new one. The old one had a knob that you twisted to set the time, and a START button that, unlike in Windows, actually started the thing. The new one had a digital interface and Marion and I spent over 10 minutes trying to get it to even turn on, but we got nothing but an error message. I feel you should never get an error message from an appliance! Eventually we got it to turn on. The sequence was not complicated, but it will not tolerate any variation in user behavior. The problem is that the design is modal, some buttons being multi-functional and sequential. A casual user like me will forget and get it wrong again. Better for me to take my popcorn over to a neighbor who remembers what to do. Anyway, here's to the good old days and the timer knob.
>
> – Regards, Roger

Example: Great Physicality in Truck Radio and Heater Knobs

Fig. 32-48 is a photo of the radio and heater controls of a pickup truck.

Fig. 32-48

Great physicality in the radio volume control and heater control knobs.

The large and easily grasped outside ring of the volume control knob is a joy to use, and it's not doubled up with any other mode. Also note the heater control knobs below the radio. Again, the physicality of grabbing and adjusting these knobs gives great pleasure on a cold winter morning.

The only downside: no tuning knob.

32.8 OUTCOMES

In Fig. 32-49, we highlight the outcomes part of the Interaction Cycle.

The outcomes part of the Interaction Cycle is about usefulness for supporting users through complete and correct "backend" functionality, effective functional affordances. There are no other UX issues in outcomes, because outcomes are computations and state changes that are internal to the system, invisible to users.

Usefulness

A component of user experience based on utility, system functionality that gives users the ability to accomplish the goals of work (or play) through using the system or product (Section 1.4.3).

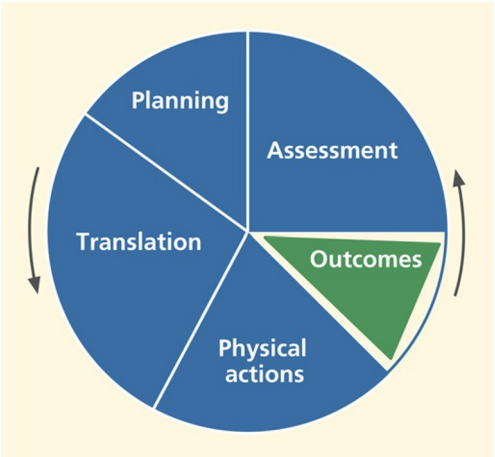


Fig. 32-49
The outcomes part of the Interaction Cycle.

32.8.1 System Functionality

The outcomes part of the Interaction Cycle is mainly about nonuser-interface system functionality, which includes all issues about software bugs on the software engineering side and issues about completeness and correctness of the backend functional software.

Check your functionality for missing features.
Don't let your functionality grow into a Jack-of-all-trades, but master of none.

If you try to do too many things in the functionality of your system, you may end up not doing anything well. Norman has warned us against general-purpose machines intended to do many different functions. He suggests, rather, "information appliances" (Norman, 1998), each intended for more specialized functions.

Check your functionality for nonuser-interface software bugs.

32.8.2 System Response Time

Slow system response can impact their perceived usage experience. Computer hardware performance, networking, and communications are usually to blame. The only thing we can do in the UX design to help is to communicate the status of user actions via effective feedback (Section 32.9.2).

32.8.3 Automation Issues

Automation, in the sense we are using the term here, means moving functions and control from the user to the internal system functionality. This can result in users not having what they need. When designers try to guess what users will need, they almost always end up getting it wrong. Design questions about automation and user control can be tricky.

Avoid loss of user control from too much automation.

The following examples show very small-scale cases of automation, taking control from the user. Small though they may be, they can still be frustrating to users who encounter them.

Example: Does the IRS Know About This?

The problem in this example no longer exists in Windows Explorer, but an early version of Windows Explorer would not let you name a new folder with all uppercase letters. In particular, suppose you needed a folder for tax documents and tried to name it "IRS." With that version of Windows, after you pressed **Enter**, the name would be changed to "Irs."

So, in slight confusion, you try again, but no deal. This had to be a deliberate "feature," probably made by a software person to protect users from what appeared to be a typographic error, but that ended up being a high-handed grasping of user control.

Example: The John Hancock Problem

Suppose a user named H. John Hancock is typing a business letter in an early version of Microsoft Word, intending to sign it at the end as:

H. John Hancock
Sr. Vice President

Instead he got (Fig. 32-50):

H. John Hancock
I. Sr. Vice President

Mr. Hancock, not being used to the technology of the 21st century, was confused about the "I", so he backed up and typed the name again, but, when he pressed **Enter** again, he got the same result. At first, he didn't know what was happening, why the "I" appeared, or how to finish the letter without getting the "I" there. At least for a few moments, the task was blocked, and Mr. Hancock was frustrated.

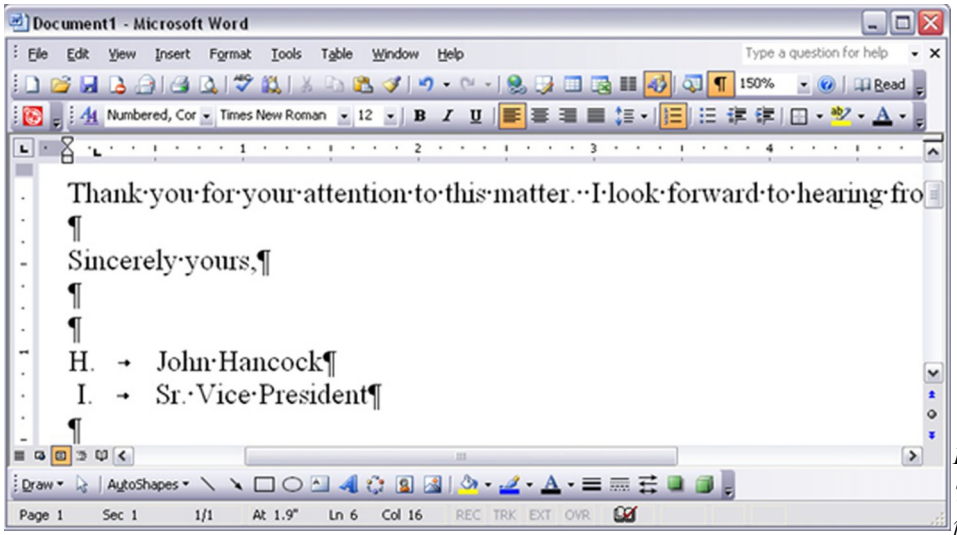


Fig. 32-50
The H. John Hancock problem.

Being a somewhat experienced user of Word, his composition of text going back to some famous early American documents, he eventually determined that the cause of the problem was that the **Automatic Numbered List** option was turned on as a kind of mode. At least for this occasion and this user, the **Automatic Numbered List** option imposed too much automation and not enough user control, and the user had difficulty understanding what was happening.

There was, in fact, useful feedback indicating the user was in the Automatic Numbered List mode, but it was presented as a "status" message displayed at the top of the window (Fig. 32-51).

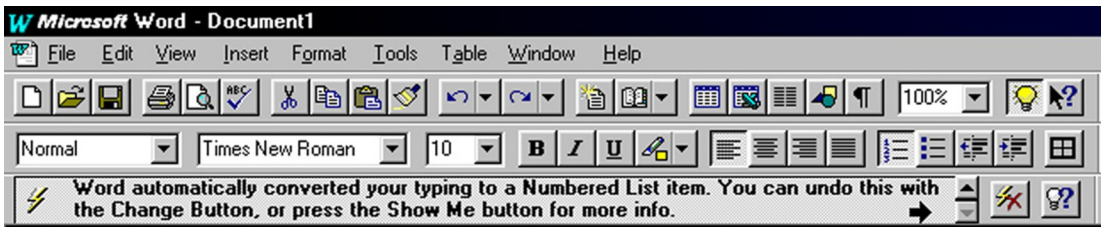


Fig. 32-51
If only Mr. Hancock had seen this (screen image courtesy of Tobias Frans-Jan Theebe).

However, Mr. Hancock didn't notice this feedback message because it violated the assessment guideline to "Locate feedback within the user's focus of attention, perhaps in a pop-up dialogue box but not in a message or status line at the top or bottom of the screen."

Help the user by automating where there is an obvious need.

In some cases, automation can be helpful. The following example is about one such case.

Example: Sorry, Off Route; You Lose!

No matter how good your GPS system is, as a human driver you can still make mistakes and drive off course, deviating from the route planned by the system. Today's GPS units are very good at helping the driver recover and get back on route. It recalculates a new route from the current position immediately and automatically, without missing a beat. Recovery is so smooth and easy that it hardly seems like an error.

Before this kind of GPS, in the early days of GPS map systems for travel navigation, there was another system developed by Microsoft, called Streets and Trips. It used a GPS antenna and receiver plugged into a USB port in a laptop. But, when the driver got off track, the screen displayed the error message **Off Route!** in a large bright red font.

Somehow you just had to know that you had to press one of the **F**, or function, keys to request recalculation of the route in order to recover. When you are busy contending with traffic and road signs, that is the time you would gladly have the system take control and share more of the responsibility. To be fair, this option probably was available in one of the preference settings or other menu choices, but the default behavior was not very usable, and this option was not easily discovered.

Designers of the Microsoft system may have decided to follow the design guideline to "keep the locus of control with the user." While user control is often the best thing, there are times when it's critical for the system to take charge and do what is needed. The work context of this UX problem includes:

- The user is busy with other tasks that cannot be automated.
- It's dangerous to distract the user/driver with additional workload.
- Getting off track can be stressful, detracting further from the focus.
- Having to intervene and tell the system to recalculate the route interferes with the user's most important task, that of driving.

This interpretation of the guideline means that, on one hand, the system doesn't insist on staying on the current route regardless of driver actions, but quietly allows the driver to make impromptu detours while the system continues to recalculate the route to help the driver eventually reach the destination.

32.9 ASSESSMENT

Assessment guidelines are to support users in understanding information displays of the results of outcomes and other feedback about outcomes such as error indications. Assessment, along with translation, is one of the places in the Interaction Cycle where cognitive affordances play a primary role.

32.9.1 System Response

A system response can contain:

- Feedback, information about course of interaction so far
- Information display, results of outcome computation
- Feed-forward, information about what to do next.

As an example, consider this message: "The value you entered for your phone number was not accepted by the system. Please try again using only numeric characters."

- The first sentence, "The value you entered for your phone number was not accepted by the system," is feedback (in the assessment part of the Interaction Cycle) about a slight problem in the course of interaction.
- The second sentence, "Please try again using only numeric characters," is feed-forward, a cognitive affordance for the translation part of the next iteration within the Interaction Cycle.

32.9.2 Assessment of System Feedback

Fig. 32-52 highlights the assessment part of the Interaction Cycle.

Feedback about errors and interaction problems is essential in supporting users in understanding the course of their interactions. Feedback is the only way users will know if an error has occurred and why. There is a strong parallel between assessment issues about cognitive affordances as feedback and translation issues about cognitive affordances as feed-forward, including existence of feedback when it's needed, sensing feedback through effective

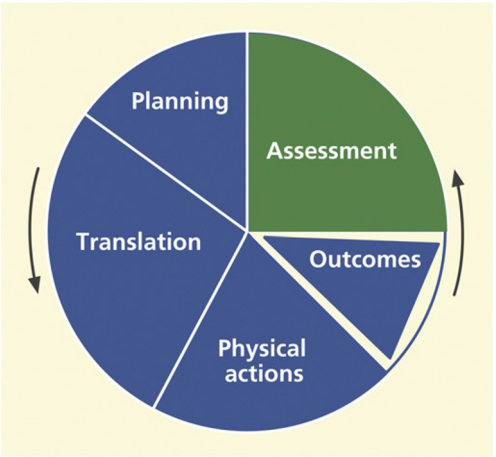


Fig. 32-52
The assessment part of the Interaction Cycle.

presentation, and understanding feedback through effective representation of content and meaning.

32.9.3 Existence of Feedback

In Fig. 32-53 we highlight the "existence of feedback" portion of the assessment part of the Interaction Cycle.

The "existence of feedback" portion of the assessment part of the Interaction Cycle is about providing necessary feedback to support users' need to know whether the course of interaction is proceeding toward meeting their planning goals.

Provide feedback for all user actions.

For most systems and applications, the existence of feedback is essential for users; feedback keeps users on track. One notable exception is the Unix operating system, in which no news is good news. No feedback in Unix means no errors. For expert users, this tacit positive feedback is efficient and keeps out of the way of high-powered interaction. For most users of most other systems, however, no news is just no news.

Provide progress feedback on long operations.

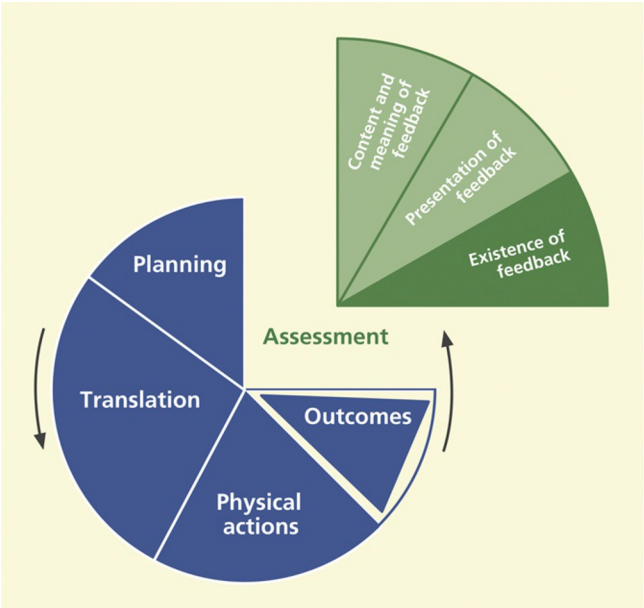


Fig. 32-53
Existence of feedback, within assessment.

For a system operation requiring significant processing time, it's essential to inform the user when the system is still computing. Keep users aware of function or operation progress with some kind of feedback as a progress report, such as a percent-done indicator.

Example: Database System Not Helpful about Progress in Pack Operation

Consider the case of a user of a dbase-family database application who had been deleting lots of records in a large database. He knew that, in dbase applications, "deleted" records are really only marked for deletion and can still be undeleted until a **Pack** operation is performed, permanently removing all records marked for deletion.

At some point, he did the **Pack** operation, but it didn't seem to work. After waiting what seemed like a long time (several seconds), he pushed the **Escape** key to get back control of the computer, and things just got more confusing about the state of the system.

It turns out that the **Pack** operation was working, but there was no indication to the user of its progress. By pushing the **Escape** key while the system was still performing the **Pack** function, the user may have left things in an indeterminate state. If the system had let him know it was, in fact, still doing the requested **Pack** operation, he would have waited for it to complete.

Request confirmation as a kind of intervening feedback.

To prevent costly errors, it's wise to solicit user confirmation before proceeding with potentially destructive actions.

But don't overuse confirmation requests and annoy.

When the upcoming action is reversible or not potentially destructive, the annoyance of having to deal with a confirmation may outweigh any possible protection for the user.

32.9.4 Presentation of Feedback

Fig. 32-54 highlights the "presentation of feedback" portion of the assessment part of the Interaction Cycle.

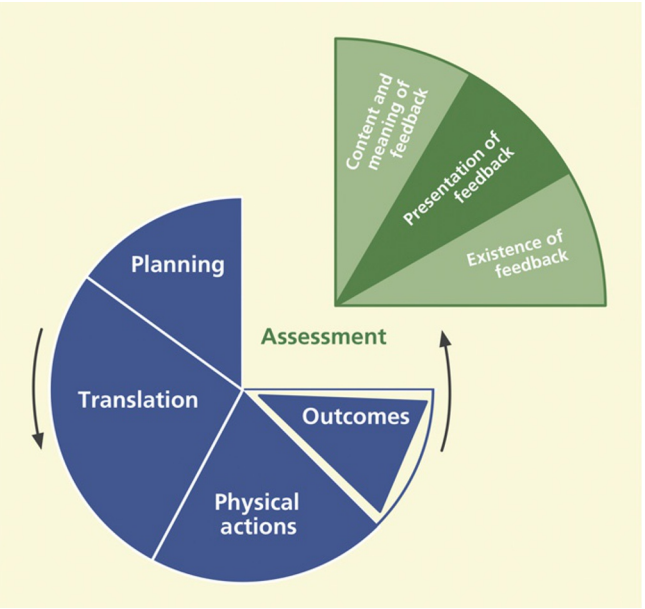


Fig. 32-54
Presentation of feedback, within assessment.

This portion of the assessment part of the Interaction Cycle is about supporting user sensing, such as seeing, hearing, or feeling, of feedback with effective design of feedback presentation and appearance. Presentation of feedback is about how feedback *appears* to users, not how it conveys meaning. Users must be able to sense (e.g., see or hear) feedback before it can be useful to them in usage.

Support user with effective sensory affordances in presentation of feedback.

32.9.4.1 Feedback visibility

Obviously, feedback cannot be effective if it cannot be seen or heard when it's needed.

Make feedback visible.

It's the designer's job to be sure each instance of feedback is visible when it's needed in the interaction.

32.9.4.2 Feedback noticeability

Make feedback noticeable.

When needed feedback exists and is visible, the next consideration is its noticeability or likelihood of being noticed or sensed. Just putting feedback on the screen is not enough, especially if the user doesn't necessarily know it exists or is not necessarily looking for it.

These design issues are largely about supporting awareness. Relevant feedback should come to users' attention without users seeking it. The primary design factor in this regard is location, putting feedback within the users' focus of attention. It's also about contrast, size, and layout complexity and their effect on separation of feedback from the background and from the clutter of other user interface objects.

Locate feedback within the user's focus of attention.

A pop-up dialogue box that appears directly within the user's focus of attention in the middle of the screen is much more noticeable than a message or status line at the top or bottom of the screen.

Make feedback large enough to notice.

32.9.4.3 Feedback legibility

Make text legible, readable.

Text legibility is about being discernable, not about its content being understandable. Font size, font type, color, and contrast are the primary relevant design factors.

32.9.4.4 Feedback presentation complexity

Control feedback presentation complexity with effective layout, organization, and grouping.

Support user needs to locate and be aware of feedback by controlling layout complexity of user interface objects. Screen clutter can obscure needed feedback.

32.9.4.5 Feedback timing

Support user needs to notice feedback with appropriate timing of appearance or display of feedback. Present feedback promptly and with adequate persistence, that is, avoid flashing the feedback briefly and removing it.

Help users detect error situations early.

Example: Don't Let Them Get Into Too Much Trouble

A local software company asked us to inspect one of their software tools. In this tool, users are restricted to certain subsets of functionality based on privileges, which, in turn, are based on various key work roles. A UX problem with a large impact on users arose when users were not aware of which parts of the functionality they were allowed to use.

As the result of a designer assumption that each user would know their privilege-based limitations, users were allowed to navigate deeply into the structure of tasks that they were not supposed to be performing. They could carry out all the steps of the corresponding transactions, but when they tried to "commit" the transaction at the end, they were told they didn't have the privileges to do that task and were blocked, and their time and effort were wasted. The moment when users try to save their work might be a convenient time for a program to check permissions, but it is better for users to realize much earlier when they are on a path to an error, thereby saving lost productivity.

32.9.4.6 Feedback presentation consistency

Maintain a consistent appearance across similar kinds of feedback.

Maintain a consistent location of feedback presentation on the screen to help users notice it quickly.

32.9.4.7 Feedback presentation medium

Consider appropriate alternatives for presenting feedback.

Use the most effective feedback presentation medium.
Consider audio as an alternative channel.

Audio can be more effective than visual media to get users' attention in cases of a heavy task load or heavy sensory work load. Audio is also an excellent alternative for vision-impaired users.

32.9.5 Content and Meaning of Feedback

In Fig. 32-55, we highlight the "content and meaning of feedback" portion of the assessment part of the Interaction Cycle.

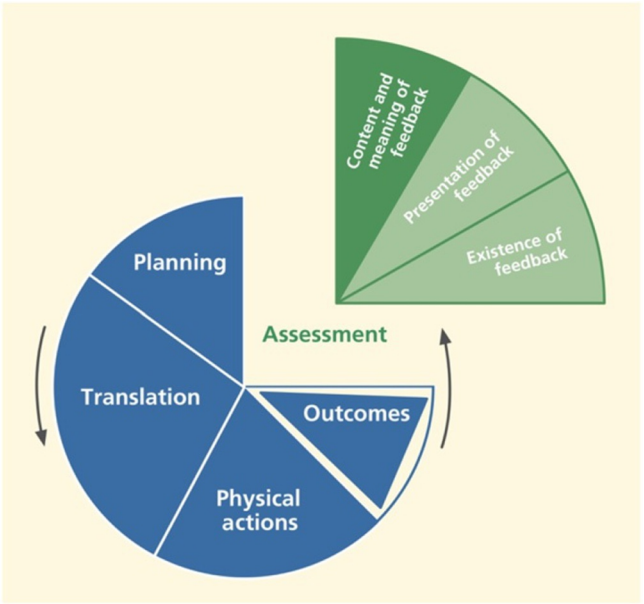


Fig. 32-55
Content/meaning of feedback, within assessment.

The content and meaning of feedback represent the knowledge that must be conveyed to users to be effective in helping them understand action outcomes and interaction progress. This understanding is conveyed through effective content and meaning in feedback, which is dependent on clarity, completeness, proper tone, usage centeredness, and consistency of feedback content.

Help users understand outcomes with effective content/meaning in feedback.

Support user ability to determine the outcomes of their actions through understanding and comprehension of feedback content and meaning.

32.9.5.1 Clarity of feedback

Design feedback for clarity.

Use precise wording and carefully chosen vocabulary to compose correct, complete, and sufficient expressions of content and meaning of feedback.

Support clear understanding of outcome (system state change) so users can assess effect of actions.
Give clear indication of error conditions.

Example: Unavailable?

Fig. 32-56 contains an error message that occurred during a **Save As** file operation in an early version of Microsoft Word. This is a classic example that has generated considerable discussion among our students. The UX problems and design issues extend well beyond just the content of the error message.

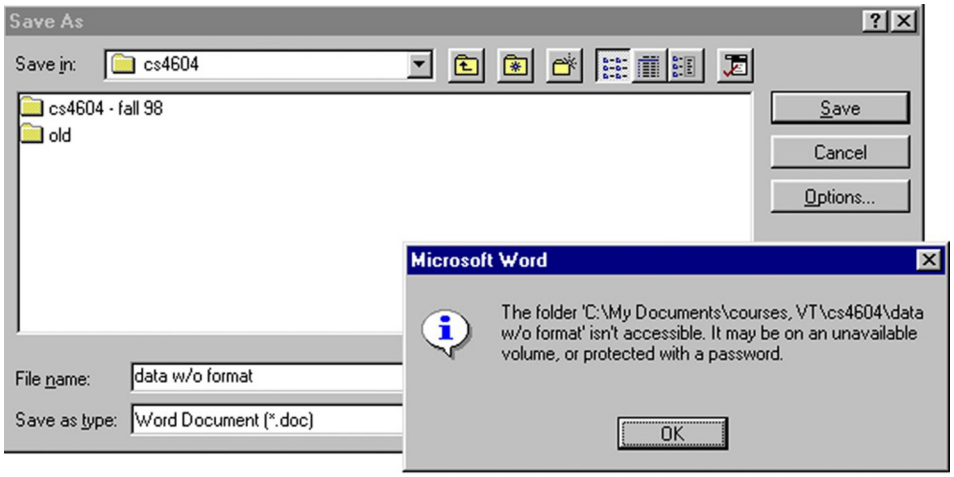


Fig. 32-56
A confusing and seemingly irrelevant error message.

In this **Save As** operation the user was attempting to save a file of unformatted data, calling it "data w/o format" for short. The resulting error message is confusing because it's about a folder not being accessible because of things like

unavailable volumes or password protection. This seems about as unclear and unrelated to the task as it could be.

In fact, the only way to understand this message is to understand something more fundamental about the **Save As** dialogue box. The design of the **File Name:** text field is *overloaded*. The usual input entered here is a file name, which is by default associated with the folder name in the **Save in:** field at the top.

But some designer must have said, "That is fine for all the GUI cowards, but what about our legions of former DOS users, our heroic power users who want to enter the full command-style directory path name for the file?" So, the design was overloaded to accept full path names of files as well, but no clue was added to the labeling to reveal this option. Because path names contain the slash (/) as a dedicated delimiter, a slash within a file name cannot be parsed unambiguously, so it's not allowed.

In our class discussions of this example, it usually takes students a long time to realize that the design solution is to unload the overloading by the simple addition of a third text field at the bottom for **Full File Directory Path Name**. Slashes in file names still cannot be allowed because any file name can also appear in a path name, but at least now, when a slash does appear in a file name in the **File Name:** field, a simple message, "Slash is not allowed in file names," can be used to give a clear indication of the real error.

A more recent version of Word half-way solves the problem by adding to the original error message: "or the file name contains a \ or /".

32.9.5.2 Precise wording

Support user understanding of feedback content by precise expression of meaning through precise word choices. Don't allow wording of feedback to be treated as an unimportant part of UX design.

32.9.5.3 Completeness of feedback

Support user understanding of feedback by providing complete information through sufficient expression of meaning, to disambiguate, make more precise, and clarify. For each feedback message, the designer should ask "Is there enough information?" "Are there enough words used to distinguish cases?"

Be complete in your design of feedback; include enough information for users to fully understand outcomes and be either confident that their command worked or certain about why it didn't.

The expression of a cognitive affordance should be complete enough to allow users to fully understand the outcomes of their actions and the status of their course of interaction.

Prevent loss of productivity due to hesitation, pondering.

Having to ponder over the meaning of feedback can lead to lost productivity. Help your users move on to the next step quickly, even if it's error recovery.

Add supplementary information, if necessary.

Short feedback is not necessarily the most effective. If necessary, add additional information to make sure that your feedback information is complete and sufficient.

Give enough information for users to make confident decisions about the status of their course of interaction.
Help users understand what the real error is.
Give enough information about the possibilities or alternatives so users can make an informed response to a confirmation request.

Example: Quick, What to Do?

The exit message from the Microsoft Outlook email system in Fig. 32-57 is an example of not giving enough information for users to make confident decisions. This message is displayed when a user tries to exit the Outlook email system before all queued messages are sent.

When users first encountered this message, they were often unsure about how to respond because it didn't inform them of the consequences of either choice. What are the consequences of "exiting anyway?" If the user exits anyway, does it

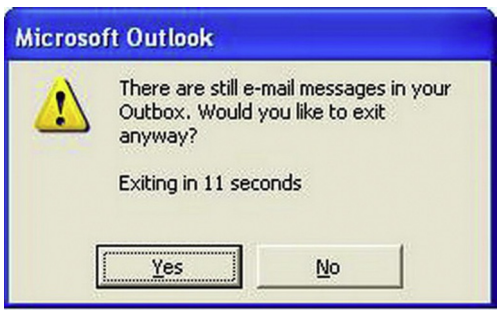


Fig. 32-57
Not enough information in this feedback (and feed-forward) message.

still send the outstanding messages, or do they get lost? One would hope that the system could go ahead and send the messages, regardless, but why, then, did it give this message? So, maybe the user will lose those messages. What made it worse was the fact that control would be snatched away in 11 seconds, and counting. How imperious!

Fig. 32-58 is an updated version of this same message, only this time, it gives a bit more information about the repercussions of exiting prematurely, but it still doesn't say if exiting will cause messages to be lost or just queued for later.

Fig. 32-58
This is better, but still could be more helpful.

32.9.5.4 Tone of feedback expression

When writing the content of a feedback message, it can be tempting to castigate the user for making a "stupid" mistake. As a professional UX designer, you must separate yourself from those feelings and put yourselves in the shoes of the user. You cannot know the conditions under which your error messages are received, but the occurrence of errors could well mean that the user is already in a stressful situation, so don't be guilty of adding to the user's distress with a caustic, sarcastic, or scornful tone.

Design feedback wording, especially error messages, for positive psychological impact.

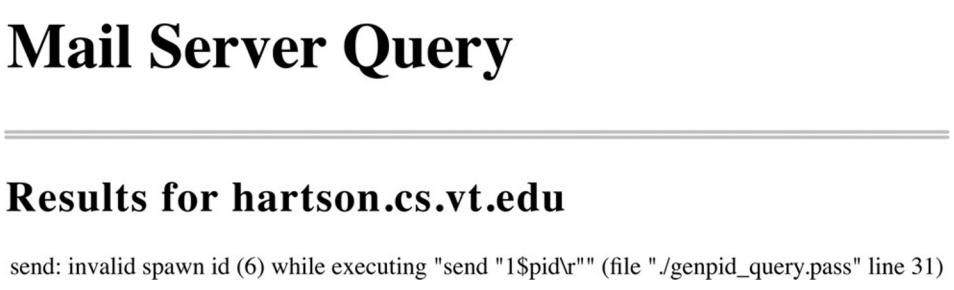
Make the system take blame for errors.
Be positive, to encourage.
Provide helpful, informative error messages, not "cute" unhelpful messages.

32.9.5.5 Usage centeredness of feedback

Employ usage-centered wording, the language of the user and the work context, in displays, messages, and other feedback.

We mentioned that user centeredness is a design concept that often seems unclear to students and some practitioners. Because it's mainly about using the vocabulary and concepts of the user's work context rather than the technical vocabulary and context of the system, we should probably call it "work-context-centered" design.

In Fig. 32-59, we see a real email system feedback message received by one of us many years ago. It's clearly system centered, if anything, and not user or work context centered. Systems people will argue correctly that the technical information in this message is valuable to them in tracing the source of the problem.


Mail Server Query

Results for hartson.cs.vt.edu

send: invalid spawn id (6) while executing "send "1$pid\r"" (file "./genpid_query.pass" line 31)

Fig. 32-59
Gobbledygook email message.

That is not the issue here; rather, it's a question of the message audience. This message is sent to users, not the systems people, and it's clearly an unacceptable message to users. Designers must seek ways to get the right message to the right audience. One solution is to give a nontechnical explanation here and add a button that says **Click here for a technical description of the problem for your systems representative.** Then, save this jargon for the systems people.

This message in the next example is similar in some ways, but is more interesting in other ways.

Example: Out of Paper, Again?

As an in-class exercise, we used to display the computer message in Fig. 32-60 and ask the students to comment on it.

Some students, usually ones who were not engineering majors, would react negatively from the start. After the usual comments pro and con, we would ask the class whether they thought it was usage centered. This usually caused some confusion and much disagreement. Then we ask a very specific question: Do you think this message is really about an *error*? In truth, the correct answer to this depends on your viewpoint, a reply we *never* got from a student.

Fig. 32-60
Classic system-centered "error" message.

The system-centered answer is yes; technically an "error condition" arose in the operating system error-handling component when it got an interrupt from the printer, flagging a situation in which there is a need for action to fix a problem. The process used inside the operating system is carried out by what the software systems people call an error-handling routine. This answer is correct, but not absolute.

From a user-, usage-, or work-context-centered view, it's definitely and 100% *not an error*. If you use the printer enough, it will run out of paper, and you will have to replace the supply. So, running out of paper is part of the normal workflow, a natural occurrence that signals a point where the human has a responsibility within the overall collaborative human-system task flow. From this perspective, we told our students we had to conclude that this was not an acceptable message to send to a user; it was not usage centered.

We decided that this exercise was a definitive litmus test for determining whether students could think user centrically. Some of our undergraduate CS students never got it. They stubbornly stuck to their judgment that there was an error and that it was perfectly appropriate to send this message to a user.

32.9.5.6 Consistency of feedback

Be consistent with feedback.

In the context of feedback, the requirement for consistency is essentially the same as it was for the expression of cognitive affordances: choose one term for each concept and use it throughout the application.

Label outcome or destination screen or object consistently with starting point, or departure, and action.

This guideline is a special case of consistency that applies to a situation where a button or menu selection leads the user to a new screen or dialogue box, a common occurrence in interaction flow. This guideline requires that the name of the destination given in the departure button label or menu choice be the same as its name when you arrive at the new screen or dialogue box. The next example is typical of a common violation of this guideline.

Example: Am I in the Right Place?

In Fig. 32-61, we see an overlay of two partial windows within an old personal document system. In the bottom layer is a menu listing some possible operations within this document system. When you click on **Add New Entry**, you go to the window in the top layer, but the title of that window is not **Add New Entry**, it's

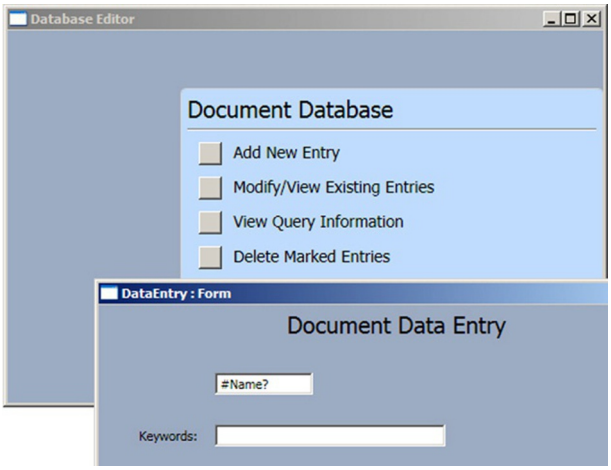


Fig. 32-61

Arrival label doesn't match departure label (screen image courtesy of Raphael Summers).

Document Data Entry. To a user, this *could* mean the same thing, but the words used at the point of departure were **Add New Entry**.

Finding different words, **Document Data Entry**, at the destination can be confusing and can raise doubts about the success of the user action. The explanation given us by the designer was that the destination window in the top layer is a destination shared by both the **Add New Entry** menu choice and the **Modify/View Existing Entries** menu choice. Because state variables are passed in

the transition, the corresponding functionality is applied correctly, but the same window was used to do the processing.

Therefore, the designer had picked a name that sort of represented both menu choices. Our opinion was that the destination window name ended up representing neither choice well, and it takes only a little more effort to use two separate windows.

Example: Title of Destination Doesn't Match Simple Search Tab Label

In this example, consider the **Simple Search** tab, displayed at the top of most screens in this digital library application and shown in Fig. 32-62.

Fig. 32-62
*The **Simple Search** tab at the top of a digital library application screen.*

That tab leads to a screen that is labeled **Search all bibliographic fields**, as shown in Fig. 32-63.

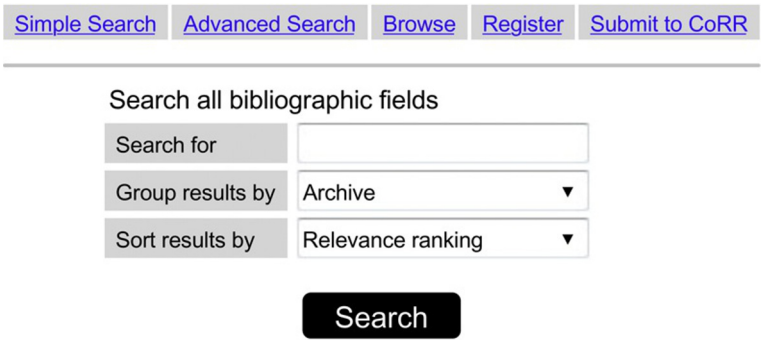


Fig. 32-63
*However, it leads to **Search all bibliographic fields**, not a match.*

We had to conclude that the departure label on the **Simple Search** tab and the destination label, Search all bibliographic fields, don't match well enough because we observed users showing surprise upon arrival and not being sure about whether they had arrived at the right place. We suggested a slight change in the wording of the destination label for the **Simple Search** function to include the same name, **Simple Search**, used in the tab and not to sacrifice the additional information in the destination label, **Search all bibliographic fields**, as shown in Fig. 32-64.

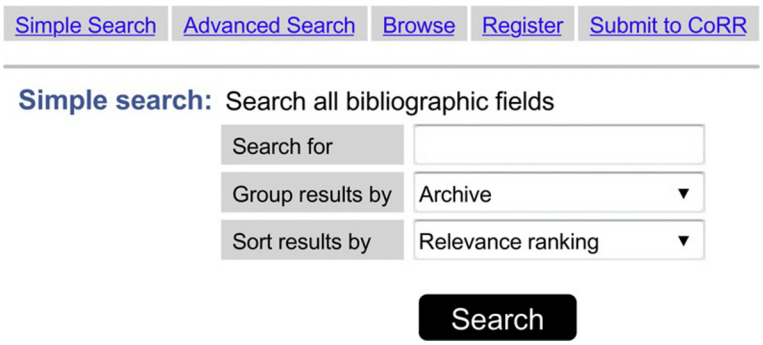


Fig. 32-64
Problem fixed by adding ***Simple Search:*** *to the destination label.*

Fig. 32-65 shows an example of departure and arrival label matching in a network services tool I helped design for Network Infrastructure & Services at Virginia Tech.

Notice how the wording of the button label, **Add Announcement**, in the first screen matches the same wording in the title of the subsequent screen.

32.9.5.7 User control over feedback detail

Organize feedback for ease of understanding.

When a significant volume of feedback detail is available, it's best not to overwhelm the user by giving all the information at once. Rather, give the most important information upfront, establishing the nature of the situation, and provide controls affording the user a way to ask for more details, as needed.

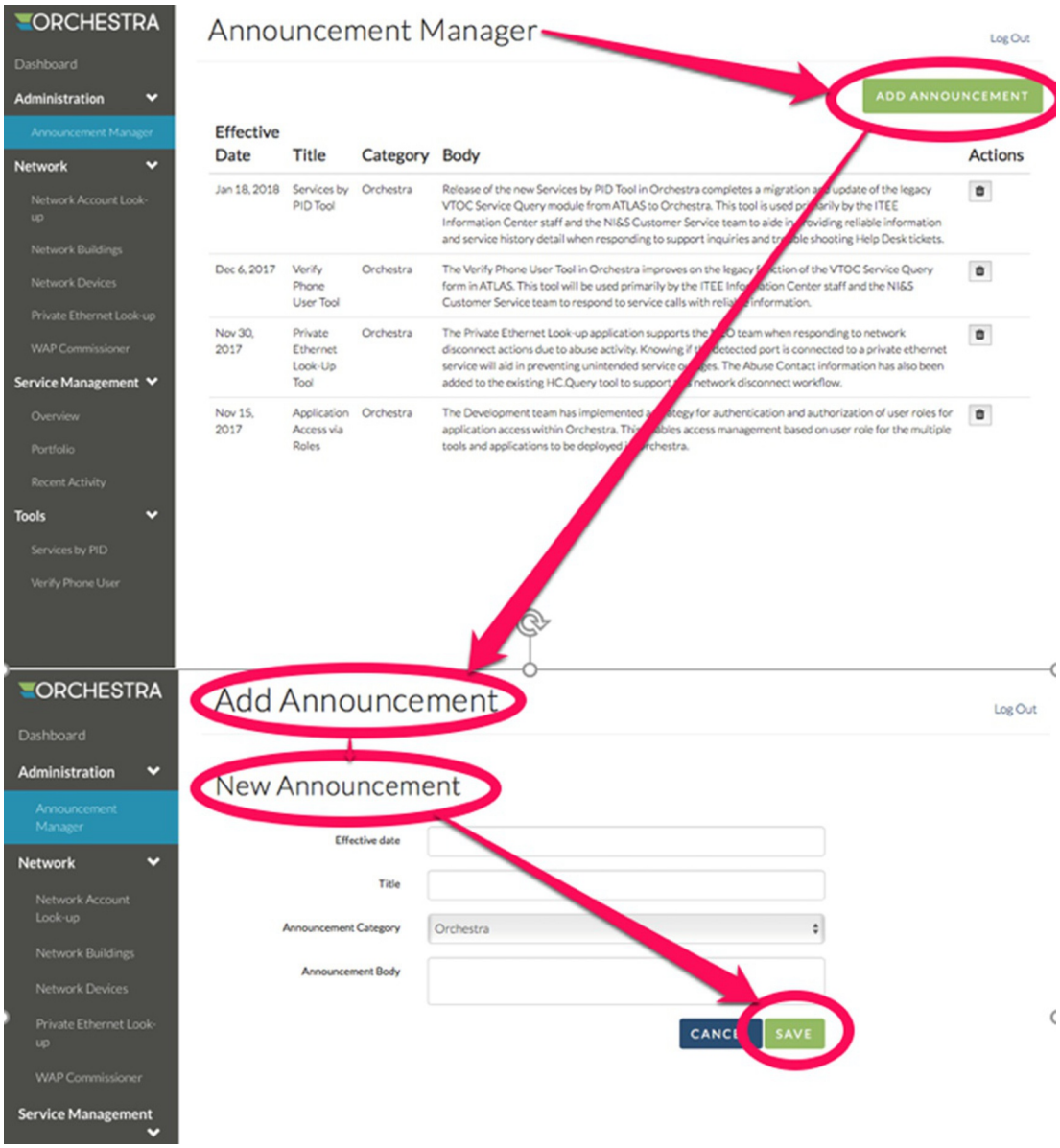
Provide user control over amount and detail of feedback.
Give only most important information at first; more on demand.

32.9.6 Assessment of Information Displays

32.9.6.1 Information organization for presentation

Organize information displays for ease of understanding.

There are entire books available on the topics of information visualization and information display design, among which the work of Tufte (1983, 1990, 1997) is perhaps the most well-known. We don't attempt to duplicate that material here, but rather reference the interested reader to pursue these topics in detail from those sources. We can, however, offer a few simple



guidelines to help with the routine presentation of information in your displays of results.

Eliminate unnecessary words.

Group related information.

Control density of displays; use white space to set off.

Fig. 32-65

Matching departure and arrival labels in a network services tool. Thanks for permission to use this example to Joe Hutson and Mathew Mathai, Network Infrastructure and Services, Virginia Tech.

Columns are easier to read than wide rows.

This guideline is the reason that newspapers are printed in columns.

Use abstraction per Shneiderman's "mantra": Overview first; zoom and filter; details on demand.

Ben Shneiderman has a "mantra" for controlling complexity in information display design (Shneiderman & Plaisant, 2005, p. 583):

- Overview first.
- Zoom and filter.
- Details on demand.

Example: The Great Train Mystery

Train passengers in Europe will notice entering passengers competing for seats that face the direction of travel. At first, it might seem that this is simply about what people were used to in cars and busses. But some people we interviewed had stronger feelings about it, saying they really were uncomfortable traveling backward and could not enjoy the scenery nearly as much that way.

Believing people in both seats see the same things out the window, we wondered if it really mattered, so we did a little psychological experiment and compared our own user experiences from both sides. We began to think about the view in the train window as an information display.

In terms of bandwidth, though, it didn't seem to matter; the total amount of viewable information was the same. All passengers see the same things and they see each thing for the same amount of time. Then we recalled Ben Shneiderman's rules for controlling complexity in information display design (see earlier discussion).

Applying this guideline to the view from a train window, we realized that a passenger traveling forward is moving *toward* what is in the view. This traveler sees the overview in the distance first, selects aspects of interest, and, as the trains goes by, zooms in on those aspects for details.

In contrast, a passenger traveling backward sees the close-up details first, which then zoom out and fade into an overview in the distance. But this close-up view is not very useful because it arrives too soon without a point of focus. By the time the passenger identifies something of interest, the chance to zoom in on it has passed; it's already getting further away. The result can be an unsatisfying user experience.

32.9.6.2 Visual bandwidth for information display

One of the factors that limit the ability of users to perceive and process displayed information is visual bandwidth of the display medium. If we are talking about the usual computer display, we must use a display monitor with a very small space for all our information presentation. This is tiny in comparison to, say, a newspaper.

When folded open, a newspaper has many times the area, and many times the capacity to display information, of the average computer screen. And a reader/user can scan or browse a newspaper much more rapidly. Reading devices such as Amazon's Kindle and Apple's iPad are pretty good for reading and thumbing through single book pages, but they lack the visual bandwidth afforded for "fanning" through pages for perusal or scanning provided by a real paper book.

Designs that speed up scrolling and paging do help, but it's difficult to beat the browsing bandwidth of paper. A reader can put a finger in one page of a newspaper, scan the major stories on another page, and flash back effortlessly to the "book-marked" page for detailed reading.

Example: Visual Bandwidth

In our UX classes, we used to have an in-class demonstration to illustrate this concept. We started with sheets of paper covered with printed text. Then we gave students a set of cardboard pieces the same size as the paper but each with a smaller cutout through which they must read the text.

One had a narrow vertical cutout that the reader had to scan, or scroll, horizontally across the page. Another had a low horizontal cutout that the reader had to scan vertically up and down the page. A third one had a small square in the middle that limited visual bandwidth in both vertical and horizontal directions and required the user to scroll in both directions.

You can achieve the same effect on a computer screen by resizing the window and adjusting the width and height accordingly. For example, in Fig. 32-66, you can see limited horizontal visual bandwidth, requiring excessive horizontal scrolling to read. In Fig. 32-67, you can see limited vertical visual bandwidth, requiring excessive vertical scrolling to read. And in Fig. 32-68, you can see limited horizontal and vertical visual bandwidth, requiring excessive scrolling in both directions. It was easy for students to conclude that any visual bandwidth limitation, plus the necessary scrolling, was a palpable barrier to the task of reading information displays.

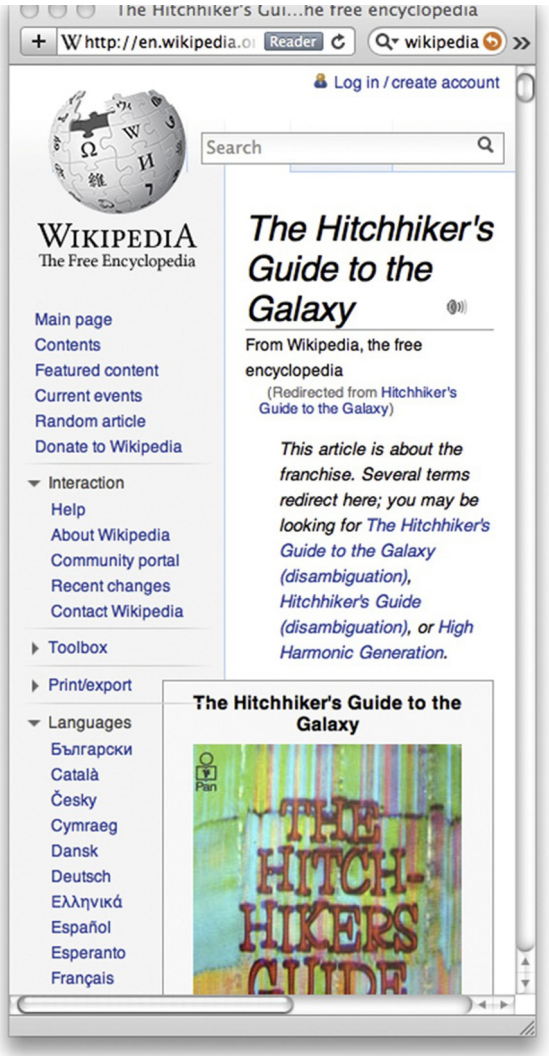


Fig. 32-66
Limited horizontal visual bandwidth.

32.10 OVERALL

This section concludes the litany of guidelines with a set of guidelines that apply globally and generally to an overall UX design rather than being associated with a specific part of the Interaction Cycle.

32.10.1 Overall Simplicity

As Norman (2007b) points out, most people think of simplicity in terms of a product that has all the features but operates with a single button. His point is that people genuinely want features and only say they want simplicity. At least for

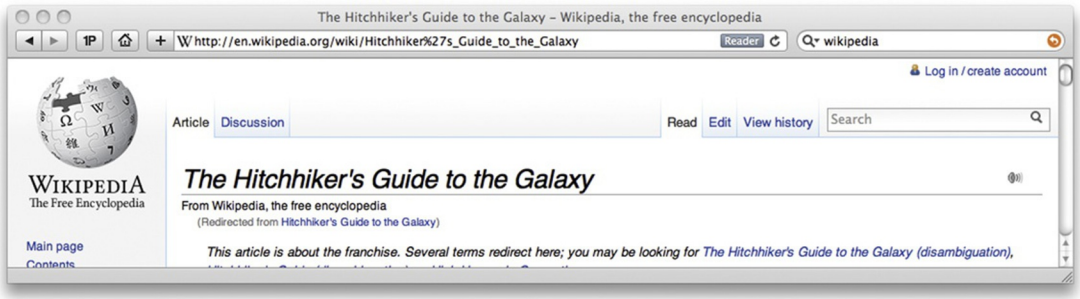


Fig. 32-67

Limited vertical visual bandwidth.

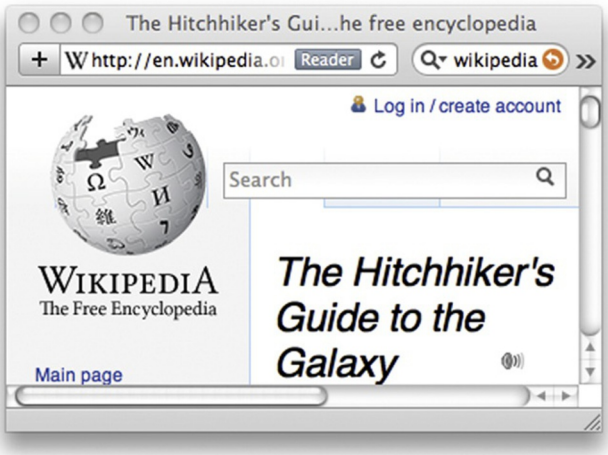


Fig. 32-68

Limited horizontal and vertical visual bandwidth.

consumer appliances, it's all about marketing, and marketing people know that features sell. And more features imply more controls.

Norman (2007b) says that even if a product design automates some features well enough so that fewer controls are necessary, people are willing to pay more for machines with more controls. Users don't want to give up control. Also, more controls give the *appearance* of more power, more functionality, and more features.

But in the computer you use to get things done at work, complexity can be a barrier to productivity. The desire is for full functionality without sacrificing UX.

Don't try to achieve the appearance of simplicity by just reducing usefulness.

A well-known web-search service provider seeking improved ease of use "simplified" their search page. Unfortunately, they did it without a real

understanding of what simplicity means. They just reduced their functionality but did nothing to improve the usability of the remaining functionality. The result was a less useful search function, and the user was still left to figure out how to use it.

Organize complex systems to make the most frequent operations simple.

Some systems cannot be entirely simple, but you can still design to make some of the most frequently used operations or tasks as simple as possible. See Isaacson (2012) for an in-depth account of the role of simplicity in Steve Jobs' vision of design. Simplicity for the user often came from attention to the tiniest of details.

Honan (2013) tells us the simple way to simplicity in design: Have the courage to remove layers and features rather than adding them. "Simple doesn't just sell, it sticks. Simple made hits of the Nest thermostat, Fitbit, and TiVo. Simple brought Apple back from the dead. It's why you have Netflix."

Example: Oh, No, They Changed the Phone System!

Years ago, our university began using a special digital phone system. It had, and still has, an enormous amount of functionality. Everyone in the university was asked to attend a one-day workshop on how to use the new phone system. Most employees rebelled and refused to attend a workshop to learn how to use a telephone—something they had been using all their lives.

They were issued a 50-page user's guide entitled "Excerpts from the PhoneMail System User Guide." Fifty pages and still an excerpt; who is going to read that? The answer is that almost everyone had to read at least parts of it because the designer's approach was to make all functions equally difficult to do. The 10% of the functionality that people had to use every day was just as mysterious as the other 90% that most people would never need. Decades later, people still don't like that phone system, but they were captive users.

32.10.2 Overall Consistency

Historically, "be consistent" is one of the earliest UX design guidelines ever and probably the most often quoted. Things that work the same way in one place as they do in another just make logical sense.

But when HCI researchers have looked closely at the concept of consistency in UX design over the years, many have concluded that it's often difficult to pin it

down in specific designs. Grudin (1989) shows that the concept is difficult to define (p. 1164) and hard to identify in a design, concluding that it's an issue without much real substance. The transfer effects that support ease of learning can conflict with ease of use (p. 1166). In the context of designing an ecology, consistency across devices is trumped by more important issues such as maintaining usage context as users cross device boundaries (Pyla, Tungare, & Pérez-Quiñones, 2006). And blind adherence without interpretation within usage context to the rule can lead to foolish or undesirable consistency, as shown in the next example.

Be consistent by doing similar things in similar ways.

Example: And What Country Shall We Send It To?

Suppose that the menu choices in all pull-down menus in an application are ordered alphabetically for fast searching. But one pull-down menu is in a form in which the user enters a mailing address. One of the fields in the form is for "country," and the pull-down list contains dozens of entries. Because the majority of customers for this website are expected to live in the United States, ease of use will be better in a design with "United States" at the top of the pull-down list instead of near the bottom of an alphabetical list, even though that is inconsistent with all the other pull-down menus in the application.

Use consistent layout/location for objects across screens.
Maintain custom style guides to support consistency.

32.10.2.1 Structural consistency

We think Reisner (1977) helped clarify the concept of consistency, in the context of database query languages, when she coined the term "structural consistency." In referring to the use of query languages, structural consistency simply required similar syntax (wording or user actions) to denote similar or related semantics. So, in our context, the expression of cognitive affordances for two similar functions should also be similar.

However, in some situations, consistency can work against distinguishability. For example, if a design contains two different kinds of delete functions, one of which is used routinely to delete objects within an application, but the other is dangerous because it applies to files and folders at a higher level, the need to

distinguish these delete functions for safety may override this guideline for making them similar.

Use structurally similar names and labels for objects and functions that are structurally similar.

Example: Next and Previous

A simple example is seen in the common **Next** and **Previous** buttons that might appear, for example, for navigation among pictures in an online photo gallery. Although these two buttons are opposite in meaning, they both are a similar *kind* of thing; they are symmetric and structurally similar navigation controls. Therefore, they should be labeled in a similar way. For example, **Go Forward** and **Previous Picture** are not as symmetric and not as similar from a linguistic perspective.

32.10.2.2 Consistency is not absolute

Many design situations have more than one consistency issue, and sometimes, they trade off against each other. We have a good example to illustrate.

Example: May I Mix You a Screwdriver?

Consider the case of multi-blade screwdrivers that are handy for dealing with different sizes and types of screws. In particular, they each have both flat-head and Phillips-head driver bits, and each of these types comes in both small and large sizes.

Fig. 32-69 illustrates two of these so-called "4-in-1" screwdrivers. As part of a discussion of consistency, we bring screwdrivers like these to class for an in-class

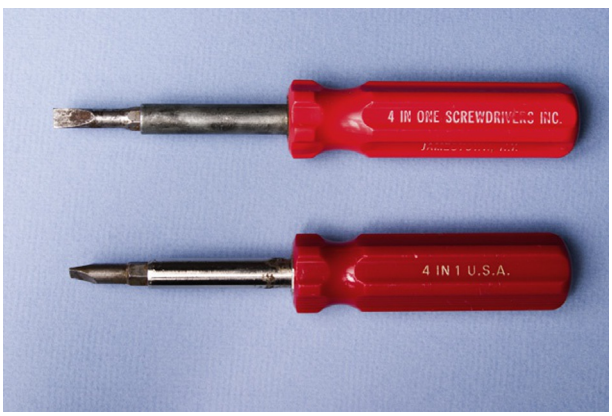


Fig. 32-69
Multipurpose screwdrivers.

exercise with students. We begin by showing the class the screwdrivers and explain how the bits are interchangeable to get the needed combination of head type and size.

Next, we pick a volunteer to hold and study one of these tools and then speak to the class about consistency issues in its design. They pull it apart, as shown in Fig. 32-70.

Fig. 32-70
Revealing the inner parts of the two screwdrivers.

The conclusion always is that it's a consistent design. We have another volunteer study the other screwdriver, always reaching the same conclusion. Then, we show the class that there are differences between the two designs, which become apparent when you compare the bits at the ends of each tool, as shown in Fig. 32-71.

One tool is consistent by head type, having both flat heads, large and small, on one insertable piece and both Phillips heads on the other piece. The other

Fig. 32-71
The two sets of screwdriver bits.

tool is consistent by size, having both large heads on one insertable piece and both small heads on the other piece.

We now ask them if they still think each design is consistent, and they do. They are each consistent; they each have intra-product consistency. Neither is more consistent than the other, but each is consistent in a different way and are not consistent with each other.

Consistency in design is supposed to aid predictability, but, because there is more than one way to be consistent in this example, you still lack interproduct consistency, and you don't necessarily get predictability. Such is one difficulty of interpreting and applying this seemingly simple design guideline.

32.10.2.3 Consistency can work against innovation

Final caveat: While a style guide works in favor of consistency and reuse, remember that it also can be a barrier to inventiveness and innovation (Kantrovich, 2004). Being the same all the time is not necessarily cool! When the need arises to break with consistency for the sake of innovation, throw off the constraints and barriers and be creative.

32.10.3 Reducing Friction

A recent term for poor usability is "friction." One definition is: "In user experience, friction is defined as interactions that inhibit people from intuitively and painlessly achieving their goals within a digital interface.[4]" "Frictionless UX has now become the new standard." Part of the requirement is to capture the full user/customer journey on your usage research. Perhaps the most important aspect is to understand and design for the entire ecology, not just tasks and devices.

Example: Frictionless Apple

In one of his *Scientific American* TechnoFiles column, David Pogue gave a good example of low friction in an ecology (Pogue, 2012): "A few months back I was at the main Apple Store in New York City. I wanted to buy a case for my son's iPod Touch—but it was December 23. The crowds were so thick, I envied sardines.

Fortunately, I knew something that most of these people didn't: I could grab an item off the shelf, scan it with my iPhone and walk right out. Thanks to the free

[4] https://www.dtelepathy.com/blog/business/strategic-ux-the-art-of-reducing-friction

Apple Store app, I didn't have to wait in line or even find an employee. The purchase was instantly billed to my Apple account. I was in and out of there in two minutes."

32.10.4 Humor

Avoid poor attempts at humor.

Poor attempts at humor usually don't work. It's easy to do humor badly, and it can easily be misinterpreted by users. You may be sitting in your office feeling good and want to write a cute error message, but users receiving it may be tired and stressed, and the last thing they need is the irritation of a bad joke, especially if this is not the first time they were subjected to it.

32.10.5 Anthropomorphism

Simply put, anthropomorphism is the attribution of human characteristics to nonhuman objects. We do it every day; it's a form of humor. You say, "my car is sick today," or "my computer doesn't like me," and everyone understands what you mean. In UX design, however, the context is usually about getting work done, and anthropomorphism can be less appreciated. A case can be made for and against anthropomorphism. We start with the case against.

32.10.5.1 Avoiding anthropomorphism

Avoid the use of anthropomorphism in UX designs.

Shneiderman and Plaisant (2005, pp. 80, 484) say that a model of computers that leads one to believe they can think, know, or understand in ways that humans do is erroneous and dishonest. When the deception is revealed, it undermines trust.

Avoid using first-person speech in system dialogue.

"Sorry, but I cannot find the file you need" is less honest and no more informative than something such as "File not found" or "File doesn't exist." If attribution must be given to what it is that cannot find your file, you can reduce anthropomorphism by using the third person, referring to the software, as in "Windows is unable to find the application that created this file." This guideline urges us to especially eschew chatty and overly friendly use of first-person cuteness, as we see in the next example.

Example: Who Is There?

Fig. 32-72 contains a message from a database system after a search request had been submitted. Ignoring other obvious UX problems with this dialogue box and message, most users find this kind of use of first person as dishonest, demeaning, and unnecessary.

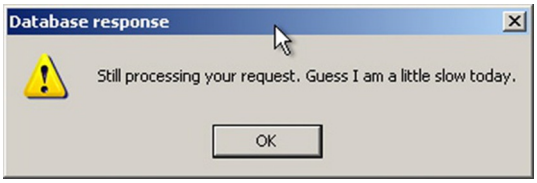


Fig. 32-72

Message tries to make computer seem like a person.

Avoid condescending offers to help.

Just when you think all hope is lost, then along comes Clippy or Bob, your personal office assistant or helpful agent. How intrusive and ingratiating! Most users dislike this kind of pandering and insinuating into your affairs, offering blandishments of hope when real help is preferred.

People expect other humans to be able to solve problems better than a machine. If your interaction dialogue portrays the machine as a human, users will expect more. When you cannot deliver, however, it's overpromising. The example that follows is ridiculous and cute, but it also makes our point.

Example: Come on, Clippy, You Can Do Better

Clearly the pop-up "help" in Fig. 32-73 is not a real example, but this kind of pop-up in general can be intrusive. In real usage situations, most users expect better.

32.10.5.2 The case in favor of anthropomorphism

On the affirmative side, Murano (2006) shows that, in some contexts, anthropomorphic feedback can be more effective than the equivalent nonanthropomorphic feedback. He also makes the case for why users sometimes prefer anthropomorphic feedback, based on subconscious social behavior of humans toward computers.

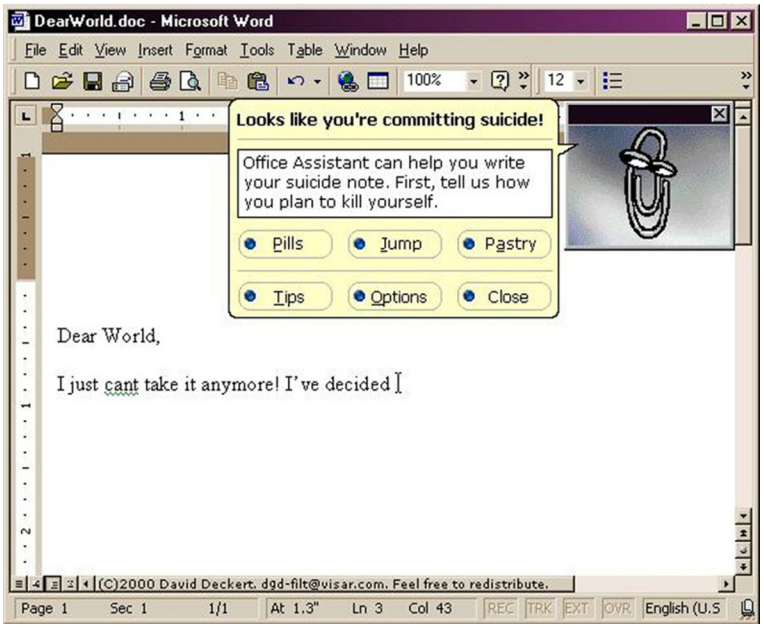


Fig. 32-73
Only too glad to help.

In his first study, Murano (2006) explored user reactions to language-learning software with speech input and output using speech recognition. Users were given anthropomorphic feedback in the form of dynamically loaded and software-activated video clips of a real language tutor giving feedback.

In this kind of software usage situation, where the objectives and interaction are very similar to what would be expected from a human language tutor, "the statistical results suggested the anthropomorphic feedback to be more effective. Users were able to self-correct their pronunciation errors more effectively with the anthropomorphic feedback. Furthermore, it was clear that users preferred the anthropomorphic feedback." The positive results are not surprising because this kind of application naturally uses human-computer interaction that is very close to natural human-to-human interaction.

In another study, Murano (2006) determined that, for direction-finding tasks, a map plus some guiding text was more effective than anthropomorphic feedback using video clips of a human giving directions verbally, with user preferences nearly evenly divided. The bottom line for Murano is that some application domains are more suited for anthropomorphic interaction than others.

Well-known studies by Reeves and Nass (1996) attempted to answer the question why anthropomorphic interaction might be better for some users in

some kinds of applications. They concluded that people naturally tend to interact with a computer the same social way we interact with people, in cases where feedback is given as natural language speech (Nass, Steuer, & Tauber, 1994). People treat computers in a social manner if the output of computers treats them in a social manner. And, of course, the case in favor of anthropomorphism is strong for systems specifically designing to carry out conversations and otherwise behave as a living being, such as Apple's Siri and Amazon's Alexa.

While a social manner of interaction did seem to be effective and desired by users of tasks that have a human-to-human counterpart, including tasks such as natural language learning and tutoring in a teacher-student kind of interaction, it's unlikely that a mutually social style and anthropomorphic interaction would have a place in the thousands of other kinds of tasks that make up a large portion of real computer usage—installing a device driver software, creating a text document, updating a data spreadsheet,

The bottom line is to tread carefully with anthropomorphism: A computer is not human. Eventually, expectations will not be met. Especially for the use of computers to get things done in business and work environments, we expect users to tire quickly of anthropomorphic feedback, particularly if it soon becomes boring by a lack of variety over time.

32.10.6 Tone and Psychological Impact

Use a tone in dialogue that supports a positive psychological impact.
Avoid violent, negative, demeaning terms.
Avoid use of psychologically threatening terms, such as "illegal," "invalid," and "abort."
Avoid use of the term "hit"; instead use "press" or "click."

32.10.7 Use of Sound and Color

"Color is one of the most powerful tools in the designer's toolkit. You can use color to impact users' emotions, draw their attention, and put them in the right frame of mind to make a purchase. It's also one of the main factors in customers' perception of a brand," (Alvarez, 2014).

Avoid irritation with annoying sound and color in displays.

Glaring colors, blinking graphics, and harsh audio not only are annoying but can have a negative effect on user productivity and the user experience over the long-term.

Use color conservatively.

Don't count on color to convey much information in your designs. It's good advice to render your design in black and white first so that you know it works without reliance on color. That will rule out usability problems some users may have with color perception due to different forms of color blindness, for example.

Use pastels, not bright colors.

Bright colors seem attractive at first, but soon lead to distraction, visual fatigue, and distaste. Exception: Bright colors on websites are memorable (Alvarez, 2014).

Be aware of color conventions (e.g., avoid red, except for urgency).

Again, color conventions are beyond our scope. They are complicated and differ with international cultural conventions. One clear-cut convention in our Western culture is about the use of red. Beyond very limited use for emergency or urgent situations, red, especially blinking red, is alarming as well as irritating and distracting.

Complementary color schemes (e.g., red/green, blue/orange, purple/yellow) are energizing (Alvarez, 2014).
Red is associated with danger, hunger, speed. So a website for fast food would include both latter factors (Alvarez, 2014).

Shades of blue and green are calming (Alvarez, 2014).
Low color contrast can be aesthetic, but high color contrast is easier to read (Alvarez, 2014).

Example: Help, Am I at Sea?

Fig. 32-74 is a map of the Outer Banks in North Carolina. We have never been able to use this map easily because it violates deeply ingrained color conventions used in maps. Blue is almost always used to denote water in maps, while gray, brown, green, or something similar is used for land. In this map, however, a deep blue is used to represent land and because there is about as much land as sea in this map, users experience a cognitive disconnect that confuses and makes it difficult to get oriented.

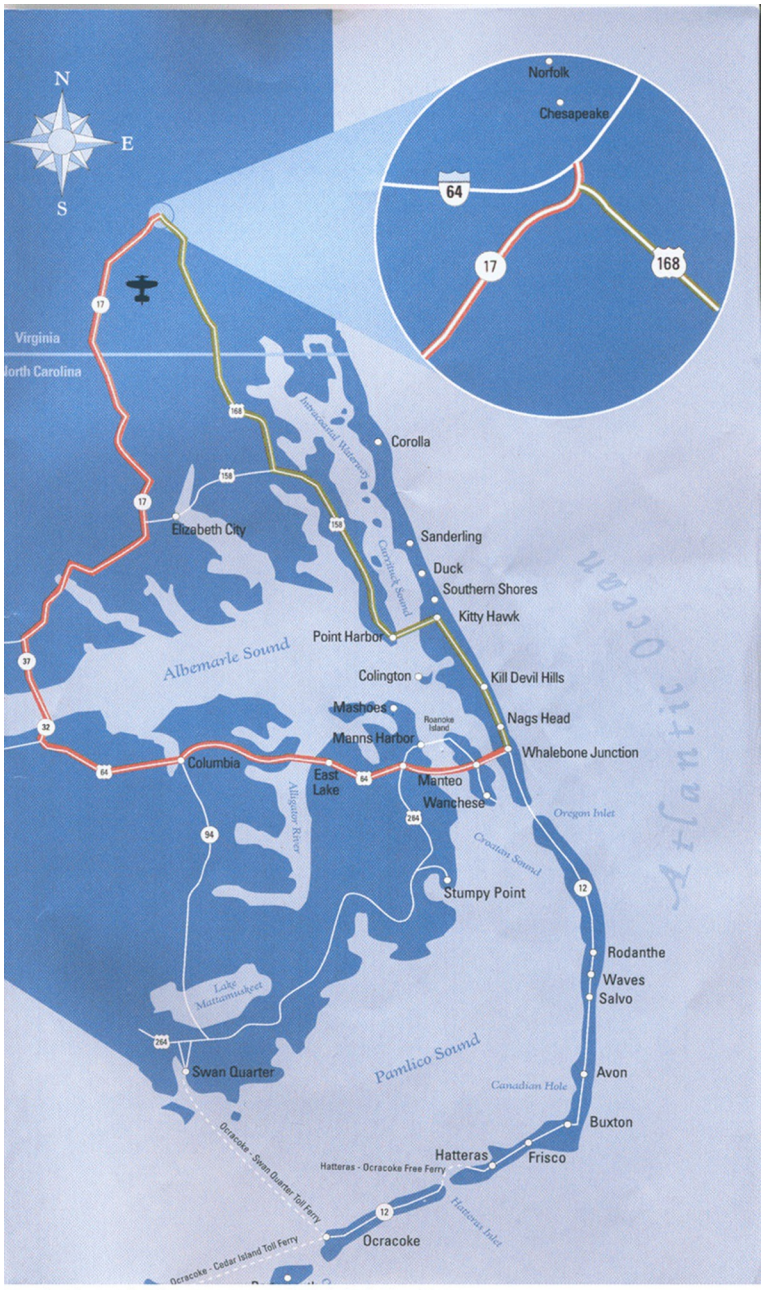


Fig. 32-74

Map of Outer Banks, but which is water and which is land?

Watch out for focusing problem with red and blue.

Chromostereopsis is the phenomenon humans face when viewing an image containing significant amounts of pure red and pure blue. Because red and blue are at opposite ends of the visual light spectrum and occur at different frequencies, they focus at slightly different depths within the eye and can appear to be at different distances from the eye. Adjacent red and blue in an image can cause the muscles used to focus the eye to oscillate, moving back and forth between the two colors, eventually leading to blurriness and fatigue.

Example: Roses Are Red; Violets Are Blue

In Fig. 32-75 we placed adjacent patches of blue and red. If the color reproduction in the book is good, some readers may experience chromostereopsis while viewing this figure.

Fig. 32-75
Chromostereopsis: humans focus at different depths in the eye for red and blue.

Get professional help with color in design.

The use of color in design is a complex topic that fills volumes of publications for research and practice, well beyond the scope of this book. Read more about it in some of these classic works (Albers, 1974; Christ, 1975; Nowell, Schulman, & Hix, 2002; Rice, 1991a, 1991b).

Bera (2016) points out the dangers of misusing color in UX designs. Too often color choices are left to software developers, who frequently select them indiscriminately. “Business dashboards that overuse or misuse colors cause

cognitive overload for users who then take longer to make decisions." "Use of colors can needlessly attract viewers' attention, causing them to search for meaning that is not there." One possible solution might be to build smart color capabilities into UX designer and software developer tools (Webster, 2014).

Color and branding. Branding is about eliciting emotion in favor of a product or brand.

Beyond this kind of commercial or organizational constraint, there are too many considerations about the use of color in design to cover completely here. The use of color in visual design involves many complicated psychological factors, and color "standards" can be complicated and will differ by international cultural conventions. The topic deserves a book or a course all on its own.

32.10.8 Text Legibility

It's obvious that text cannot convey the intended content if it's illegible.

Make presentation of text legible.
Make font size large enough for all users.
Use good contrast with background.

Use both color and intensity to provide contrast.

Use mixed case for extensive text.
Avoid too many different fonts, sizes.
Use legible fonts.
Use color other than blue for text.

It's difficult for the human retina to focus on pure blue for reading.

Accommodate sensory disabilities and limitations.

Support visually challenged, color blind users.

32.10.9 User Preferences

Allow user settings, preference options to control presentational parameters.

Afford users control of sound levels, blinking, color, and so on. Vision-impaired users, especially, need preference settings or options to adjust the text size in application displays and possibly to hear an alternative audio version of the text.

32.10.10 Accommodation of User Differences

As we have said, a treatise on accessibility is outside our scope and is treated well in the literature. Nonetheless, all UX designers should be aware of the requirement to accommodate users with special needs.

Accommodate different levels of expertise/experience with preferences.

Most of us have seen this sign in our offices or on a bumper sticker: Lead, follow, or get out of the way. In UX design, we might modify that slightly to: Lead, follow, *and* get out of the way.

- Lead novice users with adequate cognitive affordances.
- Follow intermittent or intermediate users with lots of feedback to keep them on track.
- Get out of the way of expert users; keep cognitive affordances from interfering with their physical actions.

Constantine (1994b) has made the case to design for intermediate users, which he calls the most neglected user segment. He claims that there are more intermediate users than beginners or experts.

Don't let affordances for new users be performance barriers to experienced users.

Although cognitive affordances provide essential scaffolding for inexperienced users, expert users interested in pure productivity need effective physical affordances and few cognitive affordances.

32.10.11 Helpful Help

Be helpful with Help.

Don't send your users to Help in a handbasket. For those who share our warped sense of humor, we quote from the manual for Dirk Gently's (Adams, 1990, p. 101) electronic *I Ching* calculator as an example of perhaps not so helpful help. As the protagonist consults the calculator for help to a burning personal question,

> The little book of instructions suggested that he should simply concentrate "soulfully" on the question which was "besieging" him, write it down, ponder on it, enjoy the silence, and then once he had achieved inner harmony and tranquility,

> he should push the red button. There wasn't a red button, but there was a blue button marked "Red" and this Dirk took to be the one.

Entertaining, yes; helpful, no. Note that it also makes reference at the end to an amusing little problem with cognitive affordance consistency.

32.11 CONCLUSIONS

Be cautious using guidelines.
Use careful thought and interpretation when using guidelines.
In application, guidelines can conflict and overlap.
Guidelines don't guarantee a quality user experience.
Using guidelines doesn't eliminate need for UX testing.
Design by guidelines, not by politics or personal opinion.